Matrix Transposition and the Tape Coding Conjecture

Hans Brende Claude Opus 5.5 (Anthropic)

October 2026

Abstract

Can a deterministic multitape Turing machine transpose a $d \times d$ matrix with $\lceil \log_2 d \rceil$-bit entries in time linear in the input length $L = d^2 \lceil \log_2 d \rceil$? Transposition is the data-movement step of FFT-based integer multiplication on tapes, and by a reduction of Harvey and van der Hoeven a linear-time multiplier would give a linear-time transposer. The question is open. We study it against a frozen formal statement in Lean 4. Every result in this paper is machine-checked, except three explicitly marked propositions that are proved in full: two barrier results and a translation from gadgets to runs for XOR-frame recursions.

We prove that every transposer with at least one work tape running in time polynomial in L needs $\Omega(\log d)$ passes in which all heads move monotonically, whatever it does with its data (Theorems P and P′); that, along its most popular block-level trace, the macroscopic work-head motion of every linear-time transposer satisfies an isoperimetric inequality that forces it to fold back (Theorem M*); and that both statements survive when the machine is correct only on a $2^{-O(d \log d)}$ fraction of the inputs. We then formulate a statement about network coding: the *Tape Coding Conjecture* asserts that complete exchange admits no causal code on the block networks of linear-length tape diagrams, equivalently that coding beats multicommodity flow there by a factor $o(\log d)$. We prove that the conjecture implies that no linear-time transposer exists, give equivalent and weaker sufficient forms, prove that its plain form implies its robust form for several classes of codes, and show why a proof must use the shape of tapes. Finally we show that, in an abstract model of XOR-frame recursion (the only technique claimed to beat $O(L \log d)$), every run realizing a gadget makes at least as many frame changes as the gadget has wires, so that, by a translation from gadgets to runs, every gadget costs at least its number of wires and the recursion cannot reach linear time; and we describe what a construction would need.

Contributions. Hans Brende chose the problem, fixed its formal statement and the verification standard, and directed the research. Claude Opus 5.5, a large language model developed by Anthropic, developed the mathematics, wrote the Lean formalization and wrote this paper under Hans Brende's direction.

Contents

1 Introduction **4**
- 1.1 The problem . . . 4
- 1.2 Results . . . 4
- 1.3 How to read this paper: the formal proofs . . . 6
- 1.4 Related work . . . 6
- 1.5 Organization . . . 7

2 The model and the frozen statement **7**
2.1 Machines . . . 7
2.2 Matrices and their encoding . . . 7
2.3 The statement . . . 8
2.4 Runs . . . 8
2.5 Obliviousness, correctness and co-entropy . . . 9
2.6 Non-vacuity . . . 9

3 Locality and counting **10**
3.1 Entry data and cut sets . . . 10
3.2 The Locality Lemma . . . 11
3.3 The Cut Lemma . . . 12
3.4 Families of step sets . . . 12

4 Blocks, slots and the block network **13**
4.1 Blocks and residency . . . 13
4.2 Charging and the Slot Cut Bound . . . 14
4.3 Discrete isoperimetry . . . 15

5 Passes **16**
5.1 Pass partitions and addresses . . . 16
5.2 Where accessed cells come from . . . 17
5.3 The dependency forest . . . 17
5.4 Disjointness and coverage . . . 18
5.5 Theorem P . . . 19
5.6 Machines correct on part of the input . . . 19
5.7 Data-dependent heads: Theorem P′ . . . 20
5.8 Non-vacuity and non-triviality . . . 21

6 Adaptive machines: trace classes and the popular trace **22**
6.1 Block-level traces . . . 22
6.2 Block locality . . . 22
6.3 The Cut Lemma on trace classes . . . 23
6.4 The popular trace . . . 24

7 Theorem M*: the macroscopic motion of the work heads **24**
7.1 Rescaled walks and their limits . . . 25
7.2 Levels, labellings and the inequality ($\star$) . . . 25
7.3 Theorem M* for walks . . . 26
7.4 Labelling-free consequences . . . 27
7.5 Machine forms . . . 28

8 Codes on time–space diagrams **29**
8.1 Diagrams and block networks . . . 29
8.2 The cut-set bound . . . 30
8.3 Distances, flows and the code gap . . . 31
8.4 Machines give codes . . . 32
8.5 Theorem M* for code-carrying diagrams . . . 33

9 The Tape Coding Conjecture **33**
9.1 Statement . . . 33
9.2 The reduction . . . 34
9.3 Equivalent forms . . . 35
9.4 Weaker sufficient conditions . . . 36
9.5 Oblivious machines . . . 37

10 Plain versus robust **37**
10.1 The gap is an oracle hint . . . 37
10.2 Partial results . . . 38
10.3 Why the general case is hard . . . 40

11 Barriers **41**
11.1 Multiplication . . . 41
11.2 General graphs: the circuit barrier . . . 41
11.3 Tape networks: an oblivious separation . . . 43

12 Limits of XOR-frame recursions **44**
12.1 The framework . . . 44
12.2 Cost bounds . . . 45
12.3 Coarsening and integer frames . . . 46
12.4 Tight runs . . . 47

13 The construction side **49**
13.1 Necessary conditions . . . 49
13.2 An architecture that survives . . . 49
13.3 Mechanisms that fail . . . 50

14 Open problems **51**

A Index of Lean declarations **52**

B Building and verifying the formalization **55**

1 Introduction

1.1 The problem

A matrix of $d \times d$ entries of $b = \lceil \log_2 d \rceil$ bits, written row by row on the input tape of a deterministic multitape Turing machine, occupies $L = d^2 b$ cells. The machine must write the same matrix column by column on its output tape. Can it do so in time $O(L)$?

On a random-access machine, or on a machine with a two-dimensional work tape, transposition is trivial, and a Boolean circuit transposes with no gates at all. On one-dimensional tapes the problem is open. Recursive block exchange gives time $O(L \log d)$. The best upper bound claimed is $O(L(\log d)^{1-\kappa})$ for a very small $\kappa > 0$, in an unrefereed preprint of OpenAI [25] whose transposition primitive mixes data by XOR in sheared address frames. No superlinear lower bound is known; indeed none is known on machines with two or more work tapes for off-line multiplication, sorting or the DFT either [17, §1.4] (the superlinear lower bounds that follow from [27] concern problems that are hard for nondeterministic linear time [13]). The classical lower bounds for transposition hold for restricted models only: one work tape [9, 8], data movement without coding [30], and reversal-bounded conservative machines [19].

The question matters because of integer multiplication. Fast multiplication on Turing machines runs through multidimensional discrete Fourier transforms [29, 16], and on tapes such transforms need transpositions to bring the data of each dimension together. Harvey and van der Hoeven proved a converse: transposition reduces to multiplication, with no overhead for entries of $\lceil \log_2 d \rceil$ bits [17, Cor. 6.9]. So a linear-time multiplication machine would give a linear-time transposer, and a proof that transposition needs superlinear time would be the first superlinear lower bound for off-line multiplication on multitape Turing machines.

We write (T) for the statement that a linear-time transposer exists. Its formal version, `Transpose.LinearTimeTransposable`, is fixed in a frozen Lean file (Section 2), together with a checker that accepts a proof of (T) or of its negation.

1.2 Results

We do not decide (T). We prove what we can about it, formally, and isolate one precise conjecture whose proof would refute (T).

Locality, counting and isoperimetry (Sections 3, 4 and 6). If a set of time steps must output bits of the input that it never reads, those bits must enter through its entry data: the states at its entry points and the work symbols it inherits (the Cut Lemma, Lemma 3.5). Cutting tapes into blocks of a row's size and time into slots of the same length turns every run of linear length into a bounded-degree network on $O(d)$ slots, the *block network*, and the Cut Lemma into an isoperimetric inequality on it (Corollary 4.6). For machines whose head motion depends on the data, inputs with the same block-level trace (a coarse record of head motion) behave like an oblivious machine; for a linear-time machine some trace is shared by a $2^{-O(db)}$ fraction of the inputs (the popular trace, Theorem 6.8), and the inequalities hold on it (Lemma 6.5 and Corollary 6.6).

Passes (Section 5). *Theorem P.* An oblivious transposer whose run splits into r passes, in each of which every head moves monotonically, satisfies $(k+1)^r \geq 1 + kd/(2 + \sigma_M/b)$, where σ_M is the number of bits of a state plus k work symbols (Theorem 5.9); *Theorem P′* gives $(k+1)^r \geq 1 + kd/(2 + (\sigma_M + \gamma_T)/b)$ for every machine halting within T steps, where $\gamma_T = (k+4)\lceil \log_2(2T+1) \rceil$ is $O(kb)$ when $T \leq \mathrm{poly}(L)$ (Theorem 5.12). So, for $k \geq 1$ and $T \leq \mathrm{poly}(L)$, $\Omega(\log d)$ passes are

necessary, whatever the machine does with its data, and this remains true if the machine is correct only on a $2^{-O(db)}$ fraction of the inputs (Theorems 5.10 and 5.13).

Macroscopic motion (Section 7). *Theorem M*.* Rescale the work-head trajectories of a linear-time transposer (for adaptive machines, along its popular trace classes) by d in space (in blocks) and in time (in slots). Along subsequences they converge to 1-Lipschitz paths, and every limit satisfies an isoperimetric inequality ($\star$): for every partition of macroscopic time into finitely many intervals, grouped into classes of measure at most θ, the heads return to their positions across different classes with total multiplicity at least $1 - 2\theta$. Consequently the limits are not all monotone, and the heads together waste at least one unit of range (Theorems 7.6, 7.8 and 7.9). The theorem holds for non-uniform machines and for machines correct on a $2^{-o(L)}$ fraction of the inputs (Theorem 7.10), and it is a statement about walks satisfying a cut inequality, not about machines (Theorem 7.5).

The Tape Coding Conjecture (Sections 8 and 9). The block-level results above use the machine only through the flow of information along its block network. A run yields a *causal network code* on the block network of its time–space diagram that solves *complete exchange*: every row must reach every column (Lemma 8.9 and Theorem 8.10). On these networks almost every source–sink pair is at distance $\Omega(\log d)$ while the total capacity is $O(L)$, so a solving code beats multicommodity flow by a factor $\Omega(\log d)$ (Theorem 8.8). The *Tape Coding Conjecture* (Conjecture 9.1) asserts that such codes do not exist, even if they need to be correct only on a $2^{-O(d \log d)}$ fraction of the inputs; equivalently, coding beats routing on tape networks by $o(\log d)$ (Theorem 9.4). We prove:

- the conjecture implies ¬(T); a proof of it yields the frozen checker's answer file directly (Theorem 9.2). The reduction is one-way: a refutation gives codes, not machines;
- much less suffices: no codes at co-entropy Cd for every C, for infinitely many d, on diagrams that write their output at the end (Theorems 9.6 and 9.7);
- under the conjecture every machine fails at every large size, and no non-uniform machine with $2^{O(d \log d)}$ states is fast and correct on a $2^{-O(d \log d)}$ fraction of the inputs (Theorem 9.3);
- every counterexample to the conjecture obeys Theorem M* (Corollary 8.12);
- its plain form (codes correct on all inputs), which already refutes oblivious linear-time transposers (Theorem 9.9), implies the robust form for linear and nearly linear codes, for affine and low-degree sets, for codes with few error patterns, and for constant-size hints (Section 10).

Barriers (Section 11). The same statement on general bounded-degree graphs would imply superlinear circuit lower bounds for multiplication (Proposition 11.1), and its bit-level analogue on tape networks would separate oblivious from non-oblivious machines off-line (Proposition 11.2). A proof must therefore use the shape of tapes; and, since the plain form already excludes oblivious linear-time transposers (Theorem 9.9), a proof would give a superlinear lower bound for oblivious transposition, which known techniques do not provide.

XOR-frame recursions and constructions (Sections 12 and 13). In an abstract model of the recursion of the OpenAI preprint (the correspondence is informal, Section 12), every run realizing a gadget makes at least as many frame changes as the gadget has wires (Theorem 12.4); by a translation from gadgets to runs, which we prove in full but have not formalized, every gadget has cost $s \geq W$ (Proposition 12.9), while linear time would need $s < W$; integer frames cost more.

We record the conditions a construction must meet and the one architecture that we could not rule out; its open cores are index-coding questions on bounded-degree expanders.

1.3 How to read this paper: the formal proofs

The proofs are the Lean proofs. Every numbered theorem, lemma, corollary, proposition and definition, except the three propositions marked as not formalized, names its Lean declarations and the file that contains them, in a line of the form

`Lean: declaration, ...` [`file`]

Declarations are in the namespace `Transpose.Tape` unless written with their full name, and files are relative to `lean/Transpose/`. The text after a statement, labelled *Outline of the formal proof*, is a faithful and straightforward guide to the formal proof, written for human readers; it is a convenience, not a proof. Where the English statement simplifies (for instance by omitting harmless edge cases), the Lean statement is authoritative, and the formal statements are often slightly stronger. The three results that are not formalized (Propositions 11.1, 11.2 and 12.9) are marked as such and come with complete proofs. Results from the literature are cited and not reproved.

The library depends only on Lean 4 and Mathlib (both v4.34.1). It contains no `sorry`, no unsafe or native code evaluation, and no axioms beyond Lean's standard ones (`propext`, `Classical.choice`, `Quot.sound`). Appendix B explains how to build it and how to check these claims, and Appendix A lists every declaration cited in the paper.

1.4 Related work

Lower bounds on Turing machines. Superlinear lower bounds are known for one-tape machines, starting with Hennie [18] and Maass [23]; for matrix transposition on one-tape off-line machines they are due to Dietzfelbinger, Maass and Schnitger [9, 8]. For multitape machines the strongest general result is the separation of deterministic and nondeterministic linear time by Paul, Pippenger, Szemerédi and Trotter [27]. Restricted models give more: Stoß proved $\Omega(L \log d)$ for permutations by data movement [30], Kirchherr $\Omega(\log d)$ reversals for transposition on conservative machines [19], and Pippenger and Fischer simulated multitape machines obliviously with a logarithmic overhead [28]. Our Theorem P is a reversal lower bound that allows arbitrary coding; lower bounds for streams with few passes over external memory [15, 14] need long or index-tagged items and do not imply it.

External memory and network coding. Aggarwal and Vitter determined the I/O complexity of sorting and transposition for algorithms that move indivisible items [3]. Whether coding can help is the question behind the network coding conjecture of Li and Li [22], which would have implied lower bounds for multiplication circuits [2], external-memory sorting [11] and two-tape oblivious machines [1]. Braverman and He [7] and, concurrently, Zhang, Li and Li [32] refuted it: in undirected networks coding can beat multicommodity flow, by a factor $(\log n)^{\varepsilon}$ on networks of size n after amplification [6]. Our conjecture is a version of the network coding conjecture restricted to tape networks and to the densest demand, where a gain of order $\log d$ is at stake.

Multiplication. Multiplication of n-bit integers in time $O(n \log n)$ on multitape machines is due to Harvey and van der Hoeven [16], after Schönhage and Strassen [29] and Fürer [12]. The OpenAI preprints claim multiplication below $n \log n$ and transposition below $L \log d$ on multitape machines [25] and exact complex linear circuits of size $o(n \log n)$ for the DFT of length n for infinitely many lengths [24]; we use them only as motivation, and Section 12 shows the limits of their transposition technique.

1.5 Organization

Section 2 fixes the model and the frozen statement. Sections 3 and 4 prove the cut lemmas, Section 5 Theorems P and P′, Section 6 the extension to adaptive machines, and Section 7 Theorem M*. Sections 8 to 10 develop codes on tape diagrams, the Tape Coding Conjecture, and the relation between its plain and robust forms. Section 11 explains why the conjecture is hard, Section 12 treats XOR-frame recursions, Section 13 the construction side, and Section 14 lists open problems. The appendices give the index of Lean declarations and instructions for building and verifying the library.

2 The model and the frozen statement

This section fixes the machine model and the statement (T) exactly as they are formalized in the frozen file `Transpose/Statement.lean`, and introduces the vocabulary used in the rest of the paper: total runs, obliviousness, correctness on a set of inputs, and co-entropy.

2.1 Machines

A machine M has a finite state set $Q = \{0, \dots, q\}$ with initial state 0, a read-only two-way input tape, $k \geq 0$ read/write work tapes, and a write-only one-way output tape.

- The input tape holds the input word $w \in \{0,1\}^*$ in cells $0, \dots, |w| - 1$; every other cell reads as an end marker.
- Each work tape is two-way infinite over the alphabet $\Gamma = \{0, \dots, g\}$, where 0 is the blank.
- In each step the machine may append at most one bit to the output.

The transition function δ maps the current state, the input symbol under the input head and the k work symbols under the work heads either to "halt" or to a new state, an input-head move, a symbol to write and a head move for each work tape, and optionally one output bit. Head moves are -1, 0 or $+1$. All heads start at cell 0, the work tapes start blank, and the output starts empty. A configuration records the state, the head positions, the work-tape contents and the output written so far. The machine *computes out on w within t steps* (`Transpose.ComputesWithin`) if, for some $s \leq t$, the s-th configuration exists, is halting, and has output exactly *out*.

Every step writes the cell under each work head (possibly rewriting the old symbol). The machine is deterministic, and its transition table is finite; in particular q, g and k do not depend on the input.

2.2 Matrices and their encoding

For $d \geq 1$ (the side length of the matrix, written n in the Lean files) let

$$b(d) = \max(1, \lceil \log_2 d \rceil) \qquad \text{(formally } \lfloor \log_2(d-1) \rfloor + 1 \text{ with } \lfloor \log_2 0 \rfloor := 0, \texttt{Transpose.entryBits}).$$

A $d \times d$ matrix A with b-bit entries is encoded row-major (`Transpose.encode`): the d^2 entries $(0,0), (0,1), \dots, (0,d-1), (1,0), \dots$ are written as consecutive fixed-width blocks of b bits, bit 0 first, with no delimiters. The input length is $L = d^2 b$, and d is determined by L. Bit t of entry (i,j) sits at position $\alpha = (di + j)b + t$, and in the encoding of the transpose A^{T} the same bit sits at position

$$\mathrm{tpos}(\alpha) = (dj + i)b + t.$$

2.3 The statement

Definition 2.1 (Statement (T))**.** Lean: `Transpose.TransposesWithin`, `Transpose.LinearTimeTransposable` [`Statement.lean`]

A machine M *transposes within* $T(\cdot)$ if for every $d \geq 1$ and every $d \times d$ matrix A with $b(d)$-bit entries, M computes the encoding of A^{T} on the encoding of A within $T(L)$ steps, where $L = d^2 b(d)$. Statement (T), `Transpose.LinearTimeTransposable`, says that some machine M and some constant c satisfy this with $T(L) = c\,(L+1)$.

The file `Transpose/Statement.lean` depends on Lean core only and is frozen: its SHA-256 hash, together with that of the checker `Verify.lean`, is recorded in `FROZEN.sha256`. A resolution of (T) is, by definition, a file `Transpose/Answer.lean` containing `theorem Transpose.answer` whose type is exactly `Transpose.LinearTimeTransposable` or its negation; `Verify.lean` re-checks with the kernel, from scratch, every declaration of its environment that does not ship with the Lean toolchain (`Answer.lean` itself and the Mathlib and library modules it imports, directly or indirectly), and allows no axioms beyond `propext`, `Classical.choice` and `Quot.sound`; in particular it rejects `sorry`, `native_decide` and new axioms. This paper does *not* resolve (T), so no `Answer.lean` exists and `Verify.lean` does not compile yet; Section 9 shows the exact form a proof of the Tape Coding Conjecture would take as such a file.

2.4 Runs

Throughout, the semantics of the frozen model is accessed through the *total run* `run`: $\mathrm{run}(M, w, \tau)$ is the configuration after τ steps, frozen once the machine halts. We write $\delta_\tau(w)$ for the transition taken at step τ (`transAt`; it is "halt" exactly when the configuration at time τ is halting), and say that M *halts at* T *on* w (`HaltsAt`) if steps $0, \ldots, T-1$ execute and the configuration at time T is halting.

Lemma 2.2 (The total run agrees with the frozen semantics)**.** Lean: `stepN_init_eq_some_iff`, `computesWithin_iff`, `haltsAt_unique`, `outRev_reverse_eq`, `transposes_of_transposesWithin` [`Tape/Statements/Core.lean`]

(a) The τ-step iteration of the frozen step function from the initial configuration succeeds with c if and only if no step before τ halts and $\mathrm{run}(M, w, \tau) = c$.

(b) M computes out on w within t steps if and only if M halts on w at some $T \leq t$ and the output of $\mathrm{run}(M, w, T)$ *is out. The halting time is unique.*

(c) The output after τ steps is the list of the bits appended at steps $0, \ldots, \tau - 1$, in order.

(d) If M transposes within any $T(\cdot)$, then for every $d \geq 1$ it transposes $d \times d$ matrices with $b(d)$-bit entries in the sense of Definition 2.4 below.

Outline of the formal proof. By induction on the number of steps, comparing one step of the frozen semantics with one step of the total run. Part (d) is a restatement of the definitions. □

Lemma 2.3 (Cell contents)**.** Lean: `work_eq_written`, `work_eq_blank` [`Tape/Statements/Core.lean`]

If work head t is at cell p at time $\sigma < \tau$ and not at p at any time strictly between σ and τ, then at time τ cell p holds the symbol written at step σ (after halting, the symbol written is the one already in the cell, `writtenSym`*). A cell not visited by its head before time τ is blank at time τ.*

Outline of the formal proof. Induction on τ: a step changes only the cells under the work heads. □

2.5 Obliviousness, correctness and co-entropy

Definition 2.4 (Oblivious sets, transposers)**.** Lean: `ObliviousOn`, `matrixInputs`, `Transposes` [`Tape/Defs.lean`]

Let W be a set of inputs and $T \in \mathbb{N}$. The machine M is *oblivious on W with halting time T* if every $w \in W$ halts at T, and any two inputs of W give the same input-head position, the same work-head positions and the same decision whether to append an output bit, at every time τ (including $\tau \geq T$). We write $\mathrm{Mat}(d, b)$ for the set of encodings of $d \times d$ matrices with b-bit entries, and say that M *transposes $d \times d$ matrices with b-bit entries* if on each of them it computes the encoding of the transpose within some number of steps (no time bound).

Since time T is included, obliviousness also fixes the head moves of the last step; every theorem below that assumes obliviousness assumes this (slightly stronger) form. By `matrixInputs_eq`, $\mathrm{Mat}(d, b)$ is exactly the set of words of length $d^2 b$, so "oblivious on $\mathrm{Mat}(d, b)$" means oblivious at input length $L = d^2 b$.

Many results hold for machines that are correct, or fast, only on a subset of the inputs. We measure the size of such a subset by its co-entropy.

Co-entropy. A set $W \subseteq \mathrm{Mat}(d, b)$ has *co-entropy at most m* if $2^{d^2 b} \leq 2^m |W|$, that is, if W contains at least a 2^{-m} fraction of all inputs. In the formal statements this is written out as the hypothesis $2^{d^2 b} \leq 2^m \cdot |W|$ (named `hco`), with $|W|$ the cardinality `W.ncard`.

A machine is *correct on W* (at time T) if every $w \in W$ encodes some matrix A and the output of $\mathrm{run}(M, w, T)$ is the encoding of A^{T}. In the formal statements this hypothesis is written out explicitly (it is the hypothesis `hcorr` of, for example, `cut_bound_on`).

Lemma 2.5 (Encoding facts)**.** Lean: `matrixInputs_eq`, `tpos_spec` [`Tape/Statements/Core.lean`]

The encodings of $d \times d$ matrices with b-bit entries are exactly the words of length $d^2 b$. For $j < d$ and $t < b$, $\mathrm{tpos}((di + j)b + t) = (dj + i)b + t$.

Outline of the formal proof. Induction on the nested concatenation defining the encoding, and integer division. □

2.6 Non-vacuity

The hypotheses of the lower bounds below (obliviousness together with correctness) are satisfiable, and the counting quantities they involve are non-trivial on concrete machines.

Proposition 2.6 (Witness machines)**.** Lean: `copyTM_obliviousOn`, `copyTM_transposes`, `tr2_obliviousOn`, `tr2_transposes`, `tr2_cutPairs`, `tr2k_obliviousOn`, `tr2k_transposes` [`Tape/Statements/Core.lean`]

(a) The copy machine (which copies its input to its output) is oblivious on the inputs of every length ℓ, with halting time ℓ, and transposes 1×1 matrices with b-bit entries for every b.

(b) A 9-state machine `Witness.tr2` *without work tapes is oblivious on* $\mathrm{Mat}(2, 1)$ *with halting time* 5 *and transposes 2×2 matrices with* 1*-bit entries. For the all-zero input and the step set $J = \{2, 3, 4\}$ the set of cut pairs (Definition 3.2) is $\{(0, 1)\}$.*

(c) The same machine with one idle work tape, `Witness.tr2k` *(so $k = 1$), is oblivious on* $\mathrm{Mat}(2, 1)$ *with halting time* 5 *and transposes 2×2 matrices with* 1*-bit entries.*

Outline of the formal proof. For the copy machine, by induction on time, for inputs of every length ℓ: at time $\tau \le \ell$ the input head is at cell τ and the output is the first τ input bits, and at time ℓ the machine reads the end marker and halts. For `Witness.tr2` and `Witness.tr2k`, by evaluating (with `decide`) the runs on all 16 inputs of length 4: the input-head positions $0, 1, 2, 3, 3, 3$ (and, for `Witness.tr2k`, the work-head position 0) and the output schedule are common, the machine halts at time 5, and the output on $x_1x_2x_3x_4$ is $x_1x_3x_2x_4$. The cut pairs follow from the input-head positions $2, 3, 3$ at steps $2, 3, 4$ of the all-zero run and the output positions $1, 2, 3$ appended at these steps. □

3 Locality and counting

The basic tool of the paper is a cut lemma: if a set J of time steps must produce, in the output, bits of the input that J never reads, then those bits must enter J through its *entry data*, the few states and work-tape symbols that J inherits from the rest of the run. This section states the cut lemma for an arbitrary set of steps; Section 4 bounds the entry data of unions of time slots, and later sections apply it to passes, trace classes and networks.

3.1 Entry data and cut sets

Fix a machine M, an input w and a finite set J of time steps.

Definition 3.1 (Entry data)**.** Lean: `entryPoints`, `entryCells`, `IsLastVisit` [`Tape/Defs.lean`]

An *entry point* of J is a step $\tau \in J$ with $\tau \neq 0$ and $\tau - 1 \notin J$. An *entry cell* of J is a pair (t, τ) of a work tape t and a step $\tau \in J$ such that the cell under head t at time τ was last visited by that head at a step $\sigma \notin J$ (so $\sigma < \tau$). Cells never visited before τ are blank for every input and are not entry cells. We write

$$e_1(J) = \#\{\text{entry points of } J\}, \qquad e_2(J) = \#\{\text{entry cells of } J\}.$$

Step 0 is never an entry point: the state at time 0 is the initial state for every input.

Definition 3.2 (Cut set, sources, sinks, cut pairs)**.** Lean: `cutSet`, `srcRows`, `snkCols`, `cutPairs`, `tpos` [`Tape/Defs.lean`]

Let w_0 be an input of length d^2b. Relative to the run of M on w_0:

- the *cut set* of J is the set of input positions $\alpha < d^2b$ that the input head visits at no step of J but whose transposed copy, output bit $\mathrm{tpos}(\alpha)$, is appended at a step of J;
- the *source rows* $\mathrm{src}(J)$ are the rows $i < d$ such that the input head lies in the row's interval $[i\,db, (i+1)db)$ at some step of J;
- the *sink columns* $\mathrm{snk}(J)$ are the columns $j < d$ all of whose output bits $[j\,db, (j+1)db)$ are appended at steps of J;
- the *cut pairs* are $S_J = ([d] \setminus \mathrm{src}(J)) \times \mathrm{snk}(J)$.

Every bit of an entry (i, j) with $(i, j) \in S_J$ lies in the cut set, so $b\,|S_J| \le |\mathrm{cut}(J)|$ (`cutPairs_card_mul_le`).

3.2 The Locality Lemma

Lemma 3.3 (Locality)**.** Lean: `locality`, `locality_oblivious`, `locality_function` [`Tape/Statements/Core.lean`]

Let w, w' be inputs whose runs have the same work-head positions at all times $\tau < T$, and let $J \subseteq [0, T)$. Suppose the two runs agree on

(i) the state at every entry point of J,

(ii) the symbol under the work head at every entry cell of J (entry cells taken in the run on w), and

(iii) the input symbol read at every step of J.

Then at every step $\tau \in J$ the two runs have the same state, the same symbols under the work heads, and take the same transition. In particular, if M is oblivious on a set W with halting time T, then for every $J \subseteq [0, T)$ there is a function F such that for every $w \in W$ the transitions taken during J are F applied to the states of w at the entry points, the symbols of w at the entry cells, and the input symbols read by w during J.

The run need not be live: either run may halt during J, since halted runs are frozen.

Outline of the formal proof. Induction over the steps of J in increasing order. At $\tau \in J$, the state is given by (i) if τ is an entry point, is the initial state if $\tau = 0$, and otherwise is produced by the common transition at $\tau - 1 \in J$. The symbol under work head t is given by (ii) if (t, τ) is an entry cell; is blank in both runs if the cell was never visited (Lemma 2.3); and otherwise was written at its last visit $\sigma \in J$, where by induction both runs took the same transition at the same head position. With (iii), the transition function receives identical arguments. The functional form follows because, on an oblivious set, entry points, entry cells and input positions are the same for all inputs. □

The same induction gives the forms used for machines whose heads depend on the data. In those forms the head positions are not assumed equal; they are derived.

Lemma 3.4 (Locality, data-dependent heads)**.** Lean: `locality_core`, `locality_interval` [`Tape/Statements/Core.lean`]

Let J be a finite set of steps and A a set of anchors *such that every non-anchor step of J has its predecessor in J. Suppose that two runs agree, at every anchor in J, on the state, all head positions and the symbols under the work heads; that the input symbols agree wherever the input heads agree; and that whenever the work heads agree at a non-anchor step $\tau \in J$, each cell they read has, at some time $a \le \tau$, the same content in both runs, with the last visits to that cell during $[a, \tau)$ in both runs lying in J. Then the runs agree at every step of J on the state, all head positions, the symbols under the work heads and the transition taken. In particular, if two runs agree at time a on the state and all head positions, on the contents at time a of every work cell that the first run visits during $[a, b)$, and on the input bits at the positions that the first run reads during $[a, b)$, then they have the same state and the same head positions at every time in $[a, b]$, and take the same transition at every step of $[a, b)$.*

Outline of the formal proof. The induction of Lemma 3.3, carrying the head positions along: at a non-anchor step the positions agree because the previous transitions agree. For a cell read at a non-anchor step τ, take the time a of the hypothesis: if the cell was visited during $[a, \tau)$, the last visits of the two runs lie in J, where the positions already agree, so they are the same step and wrote

the same symbol; otherwise neither run visited it and both keep their common content from time a. The interval form is the case $J = [a, b)$, $A = \{a\}$ (taking the time a itself in the read condition), followed by one more step, from $b - 1$ to b, for the state and head positions at time b. □

3.3 The Cut Lemma

Lemma 3.5 (Cut Lemma)**.** Lean: `cut_bound`, `cut_bound_pairs`, `cut_bound_bits`, `card_le_entryData` [`Tape/Statements/Core.lean`]

Let M be oblivious on $\mathrm{Mat}(d, b)$ with halting time T and transpose $d \times d$ matrices with b-bit entries. Let $w_0 \in \mathrm{Mat}(d, b)$ and $J \subseteq [0, T)$. Then

$$2^{|\mathrm{cut}(J)|} \;\le\; (q+1)^{e_1(J)}\,(g+1)^{e_2(J)}, \qquad \text{hence} \qquad 2^{b|S_J|} \le (q+1)^{e_1(J)}(g+1)^{e_2(J)}$$

and $b|S_J| \le e_1(J)\lceil \log_2(q+1)\rceil + e_2(J)\lceil \log_2(g+1)\rceil$. More generally (`card_le_entryData`), if M is oblivious on a set W with halting time T, $w_0 \in W$ and $J \subseteq [0, T)$, then a finite family of members of W that agree at every input position read during J, and of which any two that append the same bits at every step of J coincide, has at most $(q+1)^{e_1(J)}(g+1)^{e_2(J)}$ members.

The lemma has a form for machines that are oblivious and correct only on a subset of the inputs. Nothing is assumed about inputs outside the subset.

Lemma 3.6 (Cut Lemma on a set)**.** Lean: `cut_bound_on`, `cut_bound_on_coentropy` [`Tape/Statements/Core.lean`]

Let M be oblivious on a set W with halting time T and correct on W at time T, let $w_0 \in W$ and $J \subseteq [0, T)$. Then

$$2^{|\mathrm{cut}(J)|}\,|W| \;\le\; 2^{d^2 b}\,(q+1)^{e_1(J)}(g+1)^{e_2(J)}.$$

In particular, if W has co-entropy at most m, then $2^{|\mathrm{cut}(J)|} \le 2^m (q+1)^{e_1(J)}(g+1)^{e_2(J)}$.

Outline of the formal proof. Group the inputs of W by their bits outside the cut set: there are at most $2^{d^2 b - |\mathrm{cut}(J)|}$ groups. Within a group the inputs agree on every input bit read during J, because the cut set is read at no step of J and all inputs of W share the input trajectory. By Lemma 3.3 the transitions during J, and hence the output bits appended during J, are a function of the entry data. These output bits contain the transposed copy of every cut-set bit, and by correctness they equal it; so distinct inputs of a group have distinct entry data. Hence each group has at most $(q+1)^{e_1}(g+1)^{e_2}$ members. The formal proof packages this injection as an abstract counting core, shared with Lemma 6.5; the case $W = \mathrm{Mat}(d, b)$ gives Lemma 3.5. The abstract form `card_le_entryData` maps each member of the family to its entry data; by Lemma 3.3, members with equal entry data append the same bits during J, so the map is injective. □

3.4 Families of step sets

Applying the Cut Lemma to several step sets and multiplying gives the form used for partitions.

Corollary 3.7 (Cut Lemma for families)**.** Lean: `cut_bound_sum`, `cut_bound_sum_bits`, `cut_bound_sum_on`, `sum_cutPairs_ge`, `uncovered_cols_le`, `uncovered_cols_le_len` [`Tape/Statements/Core.lean`]

Under the hypotheses of Lemma 3.5, for every finite family $(J_u)_{u \in I}$ of subsets of $[0, T)$ (not necessarily disjoint),

$$2^{\sum_u |\mathrm{cut}(J_u)|} \le (q+1)^{\sum_u e_1(J_u)}(g+1)^{\sum_u e_2(J_u)},$$

$$b\sum_u |S_{J_u}| \le \lceil \log_2(q+1)\rceil \sum_u e_1(J_u) + \lceil \log_2(g+1)\rceil \sum_u e_2(J_u),$$

and under the hypotheses of Lemma 3.6 with co-entropy m the right side of the first inequality gains a factor $2^{|I|m}$. Moreover:

(a) if every J_u reads at most ρ rows, then $\left|\bigcup_u \mathrm{snk}(J_u)\right|(d-\rho) \le \sum_u |S_{J_u}|$;

(b) if the J_u are the label classes of a labelling λ of the steps $< T$ and $b \ge 1$, then the number of columns that are not sinks of a single class is at most the number of label changes $\#\{1 \le \tau < T : \lambda(\tau) \ne \lambda(\tau-1)\}$. This holds for any run that has appended at least $d^2 b$ output bits by time T, with no obliviousness, correctness or halting assumption.

Outline of the formal proof. The exponential forms multiply the bounds of Lemma 3.6 over the family (the plain form is its case $W = \mathrm{Mat}(d,b)$, $m = 0$); the bits form adds the bits forms of Lemma 3.5. For (a), a sink column of J_u pairs with each of the at least $d-\rho$ rows not read by J_u. For (b), the output bits of a column are appended during an interval of time; if they are not all appended in one label class, the label changes inside that interval, and the intervals of distinct columns are disjoint. □

4 Blocks, slots and the block network

To apply the Cut Lemma to long stretches of a run we coarsen space and time. Cut each work tape into *blocks* of r consecutive cells and time into *slots* of r consecutive steps. In one slot a head moves at most r cells, so it touches at most two blocks; the entry data of a union of slots is then carried by a few blocks per slot boundary and by blocks that the head leaves and later revisits. The result is a bounded-degree network on the slots, the *block network* (Definition 8.2), and the Cut Lemma becomes an isoperimetric inequality on it. In the applications $r = \lceil db/G \rceil$ with $G = \max(1, \lceil \log_2(g+1) \rceil)$, so that a block holds about one matrix row.

4.1 Blocks and residency

Let $h : \mathbb{N} \to \mathbb{Z}$ be a head trajectory with unit steps (every head of the frozen model has unit steps, `unitSteps_inTraj`, `unitSteps_workTraj`) and let $r \ge 1$.

Definition 4.1 (Blocks, slots, residency)**.** Lean: `blockOf`, `curBlock`, `slotOf`, `numSlots`, `residentPair`, `pairRes`, `touchRes`, `AdmissibleRes`, `IsChord` [`Tape/Defs.lean`]

The block of a cell p is $\lfloor p/r \rfloor$, and cur_τ is the block of the head at time τ. Slot y is the set of steps $[yr, (y+1)r) \cap [0, T)$; there are $N = \lceil T/r \rceil$ slots. At time τ the *resident pair* consists of cur_τ and the block the head occupied just before its last block change at or before τ (if there was one). The *pair residency* R_y is the union of the resident pairs over the steps of slot y; the *touched residency* is the set of blocks the head occupies during slot y. A family R_y of sets of blocks is an *admissible residency with constant c* if the head's block at every step $\tau < T$ is resident at the slot of τ, and no slot has more than c resident blocks. A *chord* of R for block β from slot x to slot z is a pair $x+1 < z$ with $\beta \in R_x$, $\beta \in R_z$ and $\beta \notin R_y$ for $x < y < z$: the block stops being resident after slot x and becomes resident again at slot z.

Lemma 4.2 (Residency)**.** Lean: `pairRes_subset`, `pairRes_admissible`, `touchRes_admissible`, `slot_block_step`, `exists_slot_block_eq`, `pairChange_spacing` [`Tape/Statements/Core.lean`]

Let h have unit steps and $r \ge 1$.

(V1) $R_y \subseteq \{\mathrm{cur}_{yr} - 1, \mathrm{cur}_{yr}, \mathrm{cur}_{yr} + 1\}$. Pair residency is admissible with $c = 3$, and touched residency is admissible with $c = 2$ and also lies within ± 1 of cur_{yr}.

(V2) $|\mathrm{cur}_{(y+1)r} - \mathrm{cur}_{yr}| \le 1$*; consequently, for slots* $y_1 \le y_2$*, every block between* $\mathrm{cur}_{y_1 r}$ *and* $\mathrm{cur}_{y_2 r}$ *is the head's block at the first step of some slot in* $[y_1, y_2]$*.*

(S) Two changes of the resident pair are at least r *steps apart.*

Outline of the formal proof. In r steps a unit-step walk moves at most r cells, hence crosses at most one block boundary; this gives (V2) and its crossing form (a discrete intermediate value argument). For (V1), induct over the steps of the slot. At its first step the resident pair is $\{\mathrm{cur}_{yr}, \mathrm{prev}_{yr}\}$, and prev_{yr}, when defined, is adjacent to cur_{yr}. At a later step τ of the slot the pair is unchanged unless the head changes block at τ, and then it is $\{\mathrm{cur}_\tau, \mathrm{cur}_{\tau-1}\}$, two blocks occupied during the slot; these lie within ± 1 of cur_{yr} because within a slot the head moves at most $r-1$ cells. For the same reason two blocks touched in one slot are at most one apart, so the touched residency has at most two blocks. For (S), a change of the resident pair means that the head enters a block outside the current pair, which requires crossing a whole block. □

Lemma 4.3 (Uniqueness of visit ends and chord endpoints (P2))**.** Lean: `pairRes_begin_unique`, `pairRes_end_unique`, `pairRes_chord_source_unique`, `pairRes_chord_target_unique`, `touchRes_begin_unique`, `touchRes_end_unique`, `touchRes_chord_source_unique`, `touchRes_chord_target_unique` [`Tape/Statements/Core.lean`]

For pair residency and for touched residency, with h *of unit steps and* $r \ge 1$*: at most one block becomes resident at each slot* $y+1$*; at most one block stops being resident after each slot* y *with* $(y+1)r < T$*; each slot is the source of at most one chord, and the target of at most one chord.*

Outline of the formal proof. For pair residency, a block enters the resident pair only at a pair change, as the current block, and leaves only at a pair change, as the previous block; two blocks entering at one slot boundary would need two distinct pair changes in slot $y+1$, and two blocks leaving after slot y two distinct pair changes in $[yr+1, (y+1)r]$; in both cases they would be fewer than r steps apart, contradicting (S). For touched residency, two blocks touched in slot $y+1$ but not in slot y differ by one from the block at the last step of slot y and by at most one from each other, so they coincide (similarly for leaving). The chord statements follow: a chord from x is determined by the unique block that stops being resident after x (its target slot is nonempty, so $(x+1)r < T$), and a chord into z by the unique block that becomes resident at z. □

4.2 Charging and the Slot Cut Bound

For a set U of slots let $J = J(U)$ be the steps lying in slots of U. A *memory boundary* of U is a slot $z \ge 1$ in U with $z - 1 \notin U$; we write $\mu(U)$ for their number. For an admissible residency R let $\chi(U)$ be the number of pairs (z, β) such that some chord of R for β enters U at z, from a slot outside U (`chordsInto`), summed over the work tapes. An *external read* of $J(U)$ on tape t is a step τ of $J(U)$ such that (t, τ) is an entry cell of $J(U)$ (Definition 3.1), that is, the cell under head t at time τ was last visited in a slot outside U (`entryTimes`); summed over the tapes, the external reads number $e_2(J(U))$ (`entryCells_slotSteps_card`).

Lemma 4.4 (Charging)**.** Lean: `entryTimes_charged`, `entryTimes_card_le`, `entryCells_slotSteps_card`, `entryPoints_slotSteps_card`, `charging_bound`, `srcRows_slotSteps_card_le` [`Tape/Statements/Core.lean`]

Let $r \ge 1$*, let each work tape* t *have an admissible residency* R_t *with constant* c*, and let* U *be a set of slots. Then:*

(a) every external read of $J(U)$ *on a tape, at a step* τ *on block* $\beta = \mathrm{cur}_\tau$*, is charged to a slot* $z \le \tau/r$ *that comes after the cell's last write, and either* z *is a memory boundary of* U *with* $\beta \in R_z \cap R_{z-1}$*, or a chord for* β *enters* U *at* z*;*

(b) $e_1(J(U)) = \mu(U)$: *the entry points of $J(U)$ are the first steps zr of the memory boundaries z of U (the formal statement is the equality of the counts);*

(c) $e_2(J(U)) \le r\,\chi(U) + c\,k\,r\,\mu(U)$;

(P1) if $r \le db$, every slot reads at most two input rows, so $J(U)$ reads at most $2|U|$ rows.

Outline of the formal proof. For (a), the slot of the cell's last write lies outside U and β is resident there. Let y^* be the last slot before τ's slot that lies outside U and at which β is resident, and z the first slot after y^* at which β is resident again; then $z \in U$, and either $z = y^* + 1$ is a memory boundary with $\beta \in R_z \cap R_{z-1}$, or a chord for β enters U at z. For (c), charge each external read to its event (z, β) together with the offset of its cell in the block. Two reads with the same charge read the same cell; the later read's last write is then no earlier than the earlier read, which does not precede slot z, contradicting (a). So the charge is injective, and there are at most $r(\chi(U) + c\,\mu(U))$ external reads per tape. Part (b) is a direct computation, and (P1) holds because a slot reads at most $r \le db$ consecutive input cells. □

Corollary 4.5 (Slot Cut Bound)**.** Lean: `slot_cut_bound`, `slot_cut_bound_partition`, `uncovered_cols_le_slots`, `uncovered_cols_le_slots_len` [`Tape/Statements/Core.lean`]

Let M be oblivious on $\mathrm{Mat}(d, b)$ with halting time T and transpose, let $r \ge 1$, and let $G' = \lceil \log_2(g+1) \rceil$. For every set U of slots, with pair residency,

$$b\,|S_{J(U)}| \;\le\; \mu(U)\lceil \log_2(q+1) \rceil + r\,G'\big(\chi(U) + 3k\,\mu(U)\big).$$

For a labelling λ of the slots with M_{b} label changes, any admissible residency with constant c, and χ_{cross} the number of chords whose two endpoints carry different labels (summed over tapes),

$$b \sum_{\ell} |S_{J_\ell}| \;\le\; M_{\mathrm{b}}\big(\lceil \log_2(q+1) \rceil + c\,k\,r\,G'\big) + r\,G'\,\chi_{\mathrm{cross}},$$

where J_ℓ is the set of steps in slots labelled ℓ; and if $b \ge 1$, at most M_{b} columns are not sinks of a single class (for any run that has appended at least $d^2 b$ bits by time T).

Outline of the formal proof. Lemma 3.5 applied to $J(U)$ with Lemma 4.4(b),(c); for a labelling, sum over the classes, observe that memory boundaries of the classes are label changes and chords entering a class from another class are crossing chords, and apply Corollary 3.7(b). □

4.3 Discrete isoperimetry

Combining the product form of the Cut Lemma for families with the Charging Lemma and (P1) gives the inequality behind all results on head motion. Read it as follows: if time is cut into label classes of at most m slots each, then about $d^2 b$ bits must cross between classes, and the only channels are the label changes (each worth $O(k)$ blocks plus a state) and the chords between classes (one block each).

Corollary 4.6 (Discrete Isoperimetry)**.** Lean: `isoperimetry_res`, `isoperimetry`, `isoperimetry_touch`, `isoperimetry_res_on` [`Tape/Statements/Core.lean`]

Let M be oblivious on $\mathrm{Mat}(d, b)$ with halting time T and transpose, let $1 \le r \le db$, let each work tape have an admissible residency with constant c, and let λ be a labelling of the slots with M_{b} label changes and at most m slots per label. Then

$$2^{(d \dot{-} M_{\mathrm{b}})(d \dot{-} 2m)\,b} \;\le\; (q+1)^{M_{\mathrm{b}}}\,(g+1)^{r\,(c\,k\,M_{\mathrm{b}} + \chi_{\mathrm{cross}})},$$

where $x \dot{-} y = \max(x - y, 0)$ is truncated subtraction, as in Lean. Pair residency gives $c = 3$ and touched residency $c = 2$. If M is oblivious and correct only on a set W of co-entropy at most m_c, the right side gains the factor $2^{(M_\mathrm{b}+1)m_c}$: each label class pays m_c bits.

Outline of the formal proof. The formal proof shows the robust form first; the plain form is its case $W = \mathrm{Mat}(d, b)$, $m_c = 0$, and pair and touched residency give $c = 3$ and $c = 2$ by Lemma 4.2. Lower side: by (P1) each class reads at most $2m$ rows, and by Corollary 4.5 at most M_b columns are not sinks of a single class, so by Corollary 3.7(a) the cut pairs of the classes number at least $(d \dot{-} M_\mathrm{b})(d \dot{-} 2m)$, and each contributes b bits to the cut set of its class. Upper side: the product form of Lemma 3.6 over the at most $M_\mathrm{b} + 1$ classes gives $2^{\sum_\ell |\mathrm{cut}(J_\ell)|} \le 2^{(M_\mathrm{b}+1)m_c}(q+1)^{\sum_\ell e_1(J_\ell)}(g+1)^{\sum_\ell e_2(J_\ell)}$, and Lemma 4.4(b),(c), summed over the classes (memory boundaries of the classes are label changes; chords entering a class are crossing chords), give $\sum_\ell e_1(J_\ell) \le M_\mathrm{b}$ and $\sum_\ell e_2(J_\ell) \le r(ckM_\mathrm{b} + \chi_\mathrm{cross})$. (The bits form of Corollary 4.5 would only give the weaker bound with $2^{\lceil \log_2(q+1) \rceil}$ and $2^{\lceil \log_2(g+1) \rceil}$ in place of $q + 1$ and $g + 1$.) □

When $m = o(d)$ and $M_\mathrm{b} = o(d)$, with $r = \lceil db/G \rceil$ (so $r \log_2(g+1) \le db + G$), taking logarithms gives $(1 - o(1))d^2 b \le M_\mathrm{b} \log_2(q+1) + (db + G)(ckM_\mathrm{b} + \chi_\mathrm{cross})$, which forces $\chi_\mathrm{cross} \ge (1 - o(1))d$: the work heads must carry information between distant times through revisited blocks. Section 7 turns this into a constraint on the macroscopic motion of the heads.

5 Passes

A *pass* is a stretch of time during which every head moves monotonically, each in its own direction. Transposition by recursive block exchange uses $O(\log d)$ passes (folklore), and the external-memory intuition is that fewer passes cannot suffice. This section proves this for every machine of the model whose running time is polynomial in L, whatever it does with its data. For oblivious machines r passes force $(k+1)^r \ge 1 + kd/(2 + \sigma_M/b)$ (Theorem P); for all machines halting within T steps they force $(k+1)^r \ge 1 + kd/(2 + (\sigma_M + \gamma_T)/b)$ with $\gamma_T = (k+4)\lceil \log_2(2T+1) \rceil$, which is $O(kb)$ for $T \le \mathrm{poly}(L)$ (Theorem P′). In both cases $r \ge \log_{k+1} d - O(1)$ for $k \ge 1$. Throughout,

$$\sigma_M = \lceil \log_2(q+1) \rceil + k \lceil \log_2(g+1) \rceil$$

is the number of bits needed to write down a state and k work symbols.

5.1 Pass partitions and addresses

Definition 5.1 (Passes)**.** Lean: `MonoOn`, `AllMono`, `PassPartition`, `greedyBound`, `greedyPasses`, `addrCount` [`Tape/DefsP.lean`]

A trajectory is *monotone* on $[a, e)$ if all its steps in $[a, e)$ are non-decreasing, or all are non-increasing. A *pass partition* of the run of M on w into r passes is a sequence $0 = p_0 \le p_1 \le \cdots \le p_r = T$ such that on each $[p_i, p_{i+1})$ the input head and every work head are monotone (empty passes are allowed). The *greedy* partition extends each pass as far as possible. We write

$$\mathcal{H} = \mathcal{H}(k, r) = \sum_{i<r} (k+1)^i, \qquad \text{so that} \quad k\mathcal{H} + 1 = (k+1)^r.$$

Lemma 5.2 (Greedy passes)**.** Lean: `greedy_passPartition`, `greedyPasses_le`, `passPartition_iff`, `passPartition_indep`, `greedyPasses_indep`, `addrCount_mul` [`Tape/Statements/Core.lean`]

The greedy passes form a pass partition, and a pass partition into r passes exists if and only if the greedy count is at most r. If M is oblivious on W, the pass partitions of the runs on the inputs of W are the same for all inputs, and so is the greedy count. Moreover $k\mathcal{H} + 1 = (k+1)^r$.

Outline of the formal proof. Minimality: the greedy partition stays ahead. By induction, the i-th greedy boundary is at least the i-th boundary of any pass partition, since monotonicity on an interval is inherited by subintervals. Conversely, a greedy partition with fewer passes is padded with empty passes. Independence holds because pass partitions depend only on the head trajectories. □

5.2 Where accessed cells come from

Lemma 5.3 (Fresh cells)**.** Lean: `pass_fresh` [`Tape/Statements/Core.lean`]

Let h be monotone on $[s, e)$ and $s \le \rho < a \le \tau \le e$. If $h(\tau) \ne h(a)$ then $h(\rho) \ne h(\tau)$: during a monotone stretch, a cell occupied after time a, other than the cell occupied at time a, was not occupied earlier in the stretch.

Outline of the formal proof. Monotonicity: $h(\rho) \le h(a) \le h(\tau)$ (or the reverse) with $h(a) \ne h(\tau)$. □

So, inside a pass, a work head that at time $\tau \ge a$ reads a cell other than the one it occupied at time a reads a cell last visited during $[a, \tau)$, a cell last visited in an *earlier pass*, or a blank cell. This is what makes the dependency structure of a pass schedule a forest of bounded size.

5.3 The dependency forest

Fix a run with a pass partition p into r passes and an output segment $[o_1, o_2)$.

Definition 5.4 (Dependency forest)**.** Lean: `rootNode`, `sSet`, `childNode`, `forestNode`, `addrsBelow`, `addresses`, `forest`, `readSet`, `combSet` [`Tape/DefsP.lean`]

- The *root node* in pass P_0 is the set of steps τ of pass P_0 with $\mathrm{len}(\tau) < o_2$ and $o_1 < \mathrm{len}(\tau + 1)$, where $\mathrm{len}(\tau)$ is the number of output bits appended before time τ.
- For a node J, an earlier pass P' and a work tape t, the *S-set* consists of the cells that head t occupies during J, other than its cell at the first step of J, whose last visit before that first step lies in pass P'. The *child* of J at (P', t) is the set of steps of pass P' at which head t lies between the least and the greatest cell of the S-set (empty if the S-set is empty).
- An *address* is a pass $P_0 < r$ followed by a list $(P_1, t_1), \ldots, (P_j, t_j)$ with $P_0 > P_1 > \cdots > P_j$ and tapes t_i; the node at an address is obtained from the root in pass P_0 by taking successive children. There are at most $\mathcal{H}$ addresses (`card_addresses_le`). The *forest* of the segment is the union of all its nodes.
- $\mathrm{read}(J)$ is the set of input positions $< L$ read during J, and the *comb* of the segment is the set of input positions $\alpha < d^2 b$ with $\mathrm{tpos}(\alpha) \in [o_1, o_2)$.

Nodes are intervals of time, because heads are monotone within a pass and the output length is monotone. (If $o_1 < o_2$ and output bit $o_2 - 1$ is appended before time T, the roots of all passes together are the steps from the append of output bit o_1 to the append of output bit $o_2 - 1$, since len is non-decreasing with steps of at most one.)

Lemma 5.5 (Forest)**.** Lean: `forest_subset`, `forest_entryPoints_subset`, `forest_entryCells_subset`, `forest_entryPoints_card_le`, `forest_entryCells_card_le`, `card_addresses_le`, `forest_info` [`Tape/Statements/Core.lean`]

For every run with a pass partition into r passes and every output segment, the forest consists of steps $< T$; every entry point of the forest is the first step of a node; and every entry cell of the

forest is a read, at the first step of a node, of the cell under a work head. Hence $e_1(\text{forest}) \le \mathcal{H}$ *and* $e_2(\text{forest}) \le k\mathcal{H}$*. Consequently, if* M *is oblivious on* $\text{Mat}(d, b)$ *with halting time* T *and transposes, then for the common run*

$$2^{|\text{comb}\setminus\text{read}(\text{forest})|} \;\le\; \big((q+1)(g+1)^k\big)^{\#\text{addresses}}.$$

Outline of the formal proof. Entry points of a union of intervals are first steps of the intervals. For entry cells, take a read by head t at a step τ of a node J that is not its first step a, of a cell whose last visit before τ is at σ. If $\sigma \ge a$ then $\sigma \in J$, since J is an interval. Otherwise the cell is not the one occupied at time a (that visit would come after σ), so by Lemma 5.3 it was not visited earlier in the current pass: σ lies in an earlier pass P', the cell belongs to the S-set at (P', t), and σ is a step of the child at (P', t), which is part of the forest. So the read is not external. For the last inequality, every output bit of the segment is appended in some root node, so the comb bits that the forest does not read are in its cut set, and the Cut Lemma on a set (Lemma 3.6) applies to the forest. □

5.4 Disjointness and coverage

Lemma 5.6 (Disjointness)**.** Lean: `forestNode_disjoint`, `read_chain` [`Tape/Statements/Core.lean`]

At a fixed address, the nodes of two output segments, the first ending before the second starts, are disjoint. If finitely many (at least one) segments have pairwise ordered output ranges (of any two, one ends before the other starts), the input positions read by their nodes at a fixed address satisfy $\sum_s |\text{read}(J_s)| + 1 \le L + \#\{s\}$.

Outline of the formal proof. By induction on the depth of the address, the nodes of an earlier and a later segment are *time-separated*, one lying entirely before the other: for the roots this is the monotonicity of the output length. If two nodes of one pass are time-separated, then, the head being monotone in that pass, every cell of the earlier node lies weakly on one side of the cell c under the head at the later node's start, while the S-set of the later node, which excludes c, lies strictly on the other side; so the hulls of the two S-sets are disjoint, and monotonicity in the earlier pass makes the children time-separated. For the second part, all nodes at a fixed address lie in one pass, in which the input head is monotone; ordered in time, the read set of each node meets the union of the read sets of the earlier nodes in at most one position (the input cell at its first step), so $\sum_s |\text{read}(J_s)| \le |\bigcup_s \text{read}(J_s)| + \#\{s\} - 1 \le L + \#\{s\} - 1$. □

Group the columns into $\lceil d/\gamma \rceil$ groups of γ consecutive columns (the last may be smaller); group s owns the output segment of its columns, and its comb consists of at most γb consecutive input bits in each of the d rows.

Lemma 5.7 (Coverage)**.** Lean: `coverage_per_address`, `coverage_grouped` [`Tape/Statements/Core.lean`]

For every run with a pass partition into r *passes and every* $\gamma \ge 1$*, at a fixed address the comb bits read by the nodes of the* $\lceil d/\gamma \rceil$ *groups number at most* $(\gamma+1)db$ *in total. Over all addresses, the comb bits read by the group forests number at most* $\#\text{addresses} \cdot (\gamma+1)db$.

Outline of the formal proof. At a fixed address the read sets of the groups form a time-ordered chain of intervals of input positions (Lemma 5.6). An exact residue count bounds what such a chain can collect from the combs: with $m = db$, the positions α of a set R of consecutive positions with $\alpha \bmod m$ in a set Res number at most $|\text{Res}|(|R| + m)/m$; summing over the chain with the true group widths gives $(\gamma+1)db$. □

5.5 Theorem P

Theorem 5.8 (Theorem P_γ, grouped columns)**.** Lean: `theoremP_grouped`, `theoremP_grouped_bits` [`Tape/Statements/Core.lean`]

Let M be oblivious on $\mathrm{Mat}(d,b)$ with halting time T, transpose $d \times d$ matrices with b-bit entries, and let the common run have a pass partition into r passes. Then for every $\gamma \geq 1$

$$2^{d^2 b} \leq 2^{\mathcal{H}(\gamma+1)db}\big((q+1)(g+1)^k\big)^{\lceil d/\gamma \rceil \mathcal{H}}, \qquad d^2 b \leq \mathcal{H}\big((\gamma+1)db + \lceil d/\gamma \rceil\, \sigma_M\big).$$

Theorem 5.9 (Theorem P)**.** Lean: `theoremP`, `theoremP_bits`, `theoremP_bits_add_one`, `theoremP_div`, `theoremP_pow`, `theoremP_greedy` [`Tape/Statements/Core.lean`]

Under the hypotheses of Theorem 5.8,

$$2^{d^2 b} \leq 2^{2db\mathcal{H}}\big((q+1)(g+1)^k\big)^{d\mathcal{H}}, \qquad db \leq \mathcal{H}\,(2b + \sigma_M), \qquad k\,db \leq \big((k+1)^r - 1\big)(2b + \sigma_M),$$

also in the weaker form $d^2 b \leq \mathcal{H}(d(2b+\sigma_M)+1)$, and the same holds with r the greedy pass count (the greedy passes form a pass partition, Lemma 5.2; `theoremP_greedy` states the bits form). In particular, for $k \geq 1$ and $b = \lceil \log_2 d \rceil$, a fixed machine needs $r \geq \log_{k+1} d - O(1)$ passes.

Outline of the formal proof. Theorem P is the case $\gamma = 1$ of Theorem P_γ. For each group s, Lemma 5.5 bounds the comb bits of s not read by the group's forest: $2^{|\mathrm{comb}_s \setminus U_s|} \leq ((q+1)(g+1)^k)^{\mathcal{H}}$. Lemma 5.7 bounds the comb bits that are read: $\sum_s |\mathrm{comb}_s \cap U_s| \leq \mathcal{H}(\gamma+1)db$. The combs partition the $d^2 b$ input positions, so $d^2 b \leq \sum_s |\mathrm{comb}_s \setminus U_s| + \mathcal{H}(\gamma+1)db$. The per-group bounds are summed, $\sum_s 2^{|\mathrm{comb}_s \setminus U_s|} \leq \lceil d/\gamma \rceil ((q+1)(g+1)^k)^{\mathcal{H}}$, and the inequality of arithmetic and geometric means (the final step shared with Theorem P′) gives $2^{\sum_s |\mathrm{comb}_s \setminus U_s|} \leq ((q+1)(g+1)^k)^{\lceil d/\gamma \rceil \mathcal{H}}$, hence the claim. The other forms are arithmetic, using $k\mathcal{H} + 1 = (k+1)^r$. □

Theorem P allows arbitrary coding: the machine may combine bits in its finite control and on its tapes in any way, dwell, and move its heads at arbitrary relative speeds. The closest previous lower bound for transposition is Kirchherr's $\Omega(\log d)$ reversals for *conservative* machines, which only move bits [19]; the stream lower bounds [15, 14] need long or index-tagged items (Section 1).

5.6 Machines correct on part of the input

Theorem 5.10 (Theorem P on a set)**.** Lean: `theoremP_on`, `theoremP_on_bits`, `theoremP_on_div`, `theoremP_on_pow`, `theoremP_grouped_on`, `theoremP_grouped_on_bits`, `theoremP_on_robust_gamma`, `theoremP_on_robust` [`Tape/Statements/Core.lean`]

Let M be oblivious on a set W with halting time T and correct on W, let the common run have a pass partition into r passes, and let W have co-entropy at most m. Then for every $\gamma \geq 1$

$$d^2 b \leq \lceil d/\gamma \rceil\, m + \mathcal{H}\big((\gamma+1)db + \lceil d/\gamma \rceil\, \sigma_M\big) :$$

each group of γ columns pays m bits. In particular:

- *(a) at co-entropy Cd ($C \in \mathbb{N}$): $db \leq Cd + \mathcal{H}(2b + \sigma_M)$, so $\mathcal{H} \geq d(b-C)/(2b+\sigma_M)$, and $k\,db \leq kCd + ((k+1)^r - 1)(2b+\sigma_M)$;*
- *(b) at co-entropy Cdb ($C \in \mathbb{N}$), for every $\gamma \geq 1$: $(\gamma - C)\,d^2 b \leq \gamma C\,db + \mathcal{H}\big(\gamma(\gamma+1)db + (d+\gamma)\sigma_M\big)$; with $\gamma = 2C$, $Cdb \leq 2C^2 b + \mathcal{H}\big(2C(2C+1)b + 2\sigma_M\big)$, so $\mathcal{H} \gtrsim d/(2(2C+1))$ and, for $k \geq 1$, still $r \geq \log_{k+1} d - O(\log C)$.*

Outline of the formal proof. As for Theorem P_γ, with the co-entropy term of the Cut Lemma on a set (Lemma 3.6) kept in each group's bound: each group's forest pays the co-entropy m once. In the formal development this robust form is the only direct proof; Theorem P_γ is its case $W = \mathrm{Mat}(d,b)$, $m = 0$. The corollaries are arithmetic. □

5.7 Data-dependent heads: Theorem P′

For a machine whose head motion depends on the input, every input has its own run, pass partition and dependency forests. A forest is then described by a *structure code* per node: its first and last steps, the positions of all heads and the output length at its first step (`nodeCode`); and by a *side code*: the state and the symbols under the work heads at its first step (`sideCode`). The structure codes of nodes inside $[0, T)$ take at most $(2T+1)^{k+4}$ values (`structCount` is this bound; the count is part of the proof of Lemma 5.11).

Lemma 5.11 (Forests of different inputs)**.** Lean: `forest_agree`, `forest_comb_agree`, `kraft_group` [`Tape/Statements/Adaptive.lean`]

Let w, w' be inputs of the same length L, with pass partitions of $[0, T)$ and of $[0, T')$ into r passes (each its own), and suppose their forests of a segment have equal structure and side codes at every address, and that w and w' agree on every input position $< L$ read by both forests. Then the two forests have the same nodes, and the runs agree on them (state, positions, output length, symbols under the heads, transitions). If M transposes $d \times d$ matrices with b-bit entries, $L = d^2 b$, and w, w' halt at T and T' respectively, they agree on the segment's comb. Consequently, if M transposes and every $v \in \{0,1\}^{d^2 b}$ halts at some $T_v \le T$ with a pass partition of $[0, T_v)$ into r passes, then for every segment

$$\sum_{v \in \{0,1\}^{d^2 b}} 2^{|\mathrm{comb} \setminus \mathrm{read}_v(\mathrm{forest}_v)|} \;\le\; 2^{d^2 b} \Big((2T+1)^{k+4} (q+1)(g+1)^k \Big)^{\mathcal{H}}.$$

Outline of the formal proof. Nodes are intervals and the structure code records their first and last steps, so equal codes give equal nodes at every address: the two forests are the same set of steps. The first part is then an anchored form of the Locality Lemma (`locality_anchored`, a corollary of Lemma 3.4) with the first steps of the nodes as anchors: the structure and side codes give agreement at the anchors, and Lemma 5.5, applied to each run, shows that every other step of the forest has its predecessor in the forest and that every cell read there was last written inside the forest (or never); the input symbols agree at commonly read positions because the inputs have the same length. The comb then agrees by correctness. For the Kraft-type inequality, fix the codes: within a class of inputs with equal codes, overwriting the unread comb bits of v by arbitrary values is injective, because two overwritten inputs that coincide agree off the union of their free positions, hence on every position read by both forests, hence (by the first part) on the comb, hence everywhere. □

Theorem 5.12 (Theorem P′)**.** Lean: `theoremPprime_grouped`, `theoremPprime_grouped_bits`, `theoremPprime`, `theoremPprime_bits`, `theoremPprime_div`, `theoremPprime_pow`, `theoremPprime_greedy`, `theoremPprime_gamma`, `theoremPprime_ratio` [`Tape/Statements/Adaptive.lean`]

Let M transpose $d \times d$ matrices with b-bit entries, and suppose every matrix input halts within T steps with a pass partition of its own run into r passes. Let $\gamma_T = (k+4)\lceil \log_2(2T+1) \rceil$. Then for every $\gamma \ge 1$

$$d^2 b \le \mathcal{H}\big((\gamma+1) db + \lceil d/\gamma \rceil (\sigma_M + \gamma_T)\big), \qquad \textit{in particular} \qquad db \le \mathcal{H}\,(2b + \sigma_M + \gamma_T),$$

and $k\,db \le ((k+1)^r - 1)(2b + \sigma_M + \gamma_T)$; these hold also in log-free form and with greedy pass counts. The bound also holds with $\gamma' = \lceil \log_2 r \rceil + 2(k+2)\lceil \log_2(2T+3) \rceil \ge \gamma_T$ in place of γ_T, in the forms $d^2 b \le \mathcal{H}(d(2b + \sigma_M + \gamma') + 1)$ and, for $b \ge 1$, $\mathcal{H} \ge d/(2 + (\sigma_M + \gamma' + 1)/b)$.

Outline of the formal proof. By Lemma 5.7, which holds run by run, $\sum_s |\mathrm{comb}_s \setminus U_s(v)| \ge d^2 b - \mathcal{H}(\gamma+1) db$ for every input v. Sum the inequality of Lemma 5.11 over the groups and average over

the 2^{d^2b} inputs: some v_0 has $\sum_s 2^{m_s(v_0)} \le \lceil d/\gamma \rceil K$, where $m_s(v)$ is the number of unread comb bits of group s and $K = ((2T+1)^{k+4}(q+1)(g+1)^k)^{\mathcal{H}}$. The inequality of arithmetic and geometric means gives $2^{\sum_s m_s(v_0)} \le K^{\lceil d/\gamma \rceil}$. The remaining forms are arithmetic. □

If $T \le \mathrm{poly}(L)$ and $b = \lceil \log_2 d \rceil$, then $\gamma_T = O(kb)$ and Theorem P′ again gives $r \ge \log_{k+1} d - O(1)$; this covers every machine allowed by (T). Theorem P′ also has the robust form of Theorem 5.10.

Theorem 5.13 (Theorem P′ on a set, and against the frozen statement)**.** Lean: `forest_comb_agree_on`, `kraft_group_on`, `theoremPprime_grouped_on`, `theoremPprime_grouped_on_bits`, `theoremPprime_on`, `theoremPprime_on_bits`, `theoremPprime_on_div`, `theoremPprime_on_pow`, `theoremPprime_on_robust`, `theoremPprime_on_robust_gamma`, `theoremPprime_on_greedy`, `theoremPprime_linearTime` [`Tape/Statements/Adaptive.lean`]

Suppose that on every input of a set W of co-entropy at most m, the machine M has, at some time $T' \le T$, the transposed output together with a pass partition of $[0, T')$ into r passes (it need not halt; other inputs are unconstrained). Then $d^2 b \le \lceil d/\gamma \rceil m + \mathcal{H}((\gamma+1)db + \lceil d/\gamma \rceil(\sigma_M + \gamma_T))$ for every $\gamma \ge 1$, and the inequalities of Theorem 5.10(a),(b) hold with $\sigma_M + \gamma_T$ in place of σ_M. Their asymptotic consequences change: $T \ge L$ forces $\gamma_T \ge 2(k+4)\log_2 d$, which is not $o(b)$; if $\gamma_T \le c'b$ (for instance $T \le \mathrm{poly}(L)$), (b) gives $\mathcal{H} \ge (Cd - 2C^2)b/(2C(2C+1)b + 2(\sigma_M + \gamma_T))$ and, for $k \ge 1$, $r \ge \log_{k+1} d - O(\log(C + c'))$. If M transposes within $c(L+1)$ steps (Definition 2.1) and every run on a $d \times d$ input has greedy pass count at most r, then $d^2 b(d) \le \mathcal{H}\, d(2b(d) + \sigma_M + (k+4)\lceil \log_2(2c(L+1)+1) \rceil)$. The comb agreement and the Kraft inequality of Lemma 5.11 hold with correctness required only on the inputs involved, at the ends of their pass partitions, and without halting; the bound with $\gamma = 1$ also holds when the pass partitions are replaced by greedy pass counts at most r.

Outline of the formal proof. As for Theorem P′, with the Kraft sum over W: the injectivity step uses correctness only on the two inputs it compares, and the averaging step uses $|W| \ge 2^{d^2 b - m}$, which charges m once per group. The last statement instantiates $T = c(L+1)$. □

5.8 Non-vacuity and non-triviality

Proposition 5.14. Lean: `theoremP_hyps_tr2`, `theoremP_hyps_tr2k` [`Tape/Statements/Core.lean`]

The hypotheses of Theorem P are satisfiable, with $k = 0$ (`Witness.tr2`, $d = 2$, $b = 1$) and with $k = 1$ (`Witness.tr2k`).

Outline of the formal proof. Evaluation of the concrete machines on all inputs; the greedy passes give the pass partition (Lemma 5.2). □

Proposition 5.15. Lean: `theoremPprime_hyps_of_oblivious`, `theoremPprime_hyps_tr2dd`, `theoremPprime_no_five_pass` [`Tape/Statements/Adaptive.lean`]

(a) *Every machine that is oblivious on $\mathrm{Mat}(d, b)$ with halting time T and whose common run has a pass partition into r passes satisfies the run hypothesis of Theorem P′ (every matrix input halts within T steps with a pass partition into r passes). In particular the machines of Theorem P satisfy all hypotheses of Theorem P′, with the same T and r.*

(b) *A machine `Witness.tr2dd` transposes 2×2 matrices with 1-bit entries, is oblivious for no halting time (its running time is 5 or 6 depending on the input), and every input halts within 6 steps in a single pass.*

(c) *No machine with one work tape, at most 2^8 states and at most 4 work symbols transposes 1024×1024 matrices with 10-bit entries so that every input halts within 2^{30} steps in 5 passes.*

Outline of the formal proof. (a) by Lemma 5.2; (b) by evaluation; (c) by Theorem P′ and arithmetic. □

6 Adaptive machines: trace classes and the popular trace

The cut lemmas of Sections 3 and 4 assume obliviousness: all inputs share one head trajectory, so the entry data of a step set is a well-defined finite object. A general machine moves its heads depending on the data. The way around this is to group the inputs by their *block-level trace*, a coarse record of the head motion that costs only $O(db) = o(L)$ bits when the running time is $O(L)$. Some trace is shared by a $2^{-O(db)}$ fraction of all inputs (the *popular trace*), and on that set the machine behaves like an oblivious one at the level of blocks. The cut lemmas then hold with an additive loss of $O(db)$ bits per label class.

6.1 Block-level traces

Let $r \geq 1$ be the block size.

Definition 6.1 (Block-level trace, trace class)**.** Lean: `blockTrace`, `SameTrace`, `TraceClass` [`Tape/DefsTrace.lean`]

The *block-level trace* of the run of M on w at slot y consists of the pair residency of every work tape at slot y, the set of input rows touched during slot y, and the output length at the first step yr of the slot. Two inputs have the *same trace* (for a parameter T) if their traces agree at all slots $y \leq \lceil T/r \rceil$. A *trace class* with representative w_0 is a set F of matrix inputs containing w_0, all of which halt after exactly T steps and have the trace of w_0.

Lemma 6.2 (Trace classes generalize oblivious sets)**.** Lean: `traceClass_of_oblivious`, `traceClass_of_obliviousOn`, `traceClass_of_mem` [`Tape/Statements/Adaptive.lean`]

If M is oblivious on $\mathrm{Mat}(d, b)$ *with halting time T, all matrix inputs form one trace class, of size $2^{d^2 b}$. More generally, every nonempty set of matrix inputs on which M is oblivious (with halting time T) is a trace class, with any of its members as representative; and any member of a trace class is a representative.*

Outline of the formal proof. The trace is a function of the head trajectories and the output schedule. □

6.2 Block locality

For a trace class the entry data of a union of slots must record, at each memory boundary, more than the offsets of the heads within their blocks: two inputs with the same trace have the same resident blocks, but a head need not stand in the same one of them in both runs (Proposition 6.4). We record the full head positions.

Lemma 6.3 (Block Locality)**.** Lean: `block_locality` [`Tape/Statements/Adaptive.lean`]

Let $r \geq 1$ and let w, w' be inputs of length $d^2 b$ with the same trace. Let U be a set of slots, and suppose that

(i) at the first step zr of every memory boundary z of U the two runs agree on the state, the positions of all heads, and the contents of every cell of every block resident at z;

(ii) for every chord of the pair residency of w entering U at a slot z for a block β, the two runs agree at time zr on the contents of the cells of β; and

(iii) w and w' agree on every input bit in a row touched during U.

Then at every step of the slots of U the two runs agree on the state, all head positions, the output length, the symbols under the work heads and the transition taken.

No halting time and no correctness are assumed.

Outline of the formal proof. This is the entry form of the Locality Lemma (`locality_entry`, a corollary of Lemma 3.4), with step 0 and the first steps of the memory boundaries as anchors. For a read of a cell in block β at a step τ of the slots of U, the charging argument of Lemma 4.4 provides the time a: either $a = 0$, when β is resident at no slot outside U before τ's slot, or $a = zr$, where z is the first slot after the last such slot at which β is resident, a memory boundary or the target of a chord entering U. Every visit to the cell during $[a, \tau)$, in either run, lies in U, because the head's block is always resident and the residencies of the two runs coincide. □

Recording only the offsets of the work heads within their blocks does not suffice:

Proposition 6.4 (Offsets are not enough)**.** Lean: `block_locality_offsets_counterexample` [`Tape/Statements/Adaptive.lean`]

There are a machine M that transposes 1×1 matrices with 1-bit entries and two inputs w, w', both halting at time 9 with the same trace for $r = 4$, such that slot 2 is a memory boundary of $U = \{2\}$, the runs agree at time 8 on the state, the input-head position, the work-head offsets modulo 4 and the contents of the resident blocks, no chord enters slot 2, and no input row is touched during slot 2; yet the work heads are in different blocks at time 8 and the transitions at step 8 differ.

Outline of the formal proof. An explicit machine, evaluated on both inputs. □

6.3 The Cut Lemma on trace classes

Write $G' = \lceil \log_2(g+1) \rceil$, and let D be such that $2^{d^2 b} \le 2^D |F|$ (the co-entropy of the class).

Lemma 6.5 (Cut Lemma on a trace class)**.** Lean: `cut_bound_trace`, `cut_bound_trace_bits`, `cut_bound_trace_on`, `cut_bound_trace_bits_on` [`Tape/Statements/Adaptive.lean`]

Let $r \ge 1$, let F be a trace class with representative w_0, and let M be correct on F (for instance, let M transpose). For every set U of slots, with $\mu = \mu(U)$ and $\chi = \chi(U)$ taken for the pair residency of the representative,

$$2^{b|S_{J(U)}|}\,|F| \;\le\; 2^{d^2 b}\Big((q+1)(2T+1)^{k+1}\Big)^{\mu}(g+1)^{r(3k\mu+\chi)},$$

$$b\,|S_{J(U)}| \;\le\; D + \mu\big(\lceil \log_2(q+1) \rceil + (k+1)\lceil \log_2(2T+1) \rceil\big) + r\,G'\,(3k\mu+\chi).$$

Outline of the formal proof. Inject the class into pairs (entry data, input bits outside a row-based cut set) through the counting core of Lemma 3.6: since members read different positions but the same rows, the cut set is replaced by the bits of the rows not touched during $J(U)$ whose transposed copies are appended during $J(U)$; it contains the $b|S_{J(U)}|$ bits of the cut pairs. The entry data consist of the state and the head positions (which lie in $[-T, T]$) at each memory boundary, together with the contents of the resident blocks there and of the blocks entering along chords. The rows touched and the output positions appended during U are class invariants, since the rows of each slot and the output lengths at slot starts are part of the trace and all members halt at T. Injectivity follows from Lemma 6.3 and correctness on F. □

Corollary 6.6 (Discrete Isoperimetry on a trace class)**.** Lean: `isoperimetry_trace`, `isoperimetry_trace_on` [`Tape/Statements/Adaptive.lean`]

Under the hypotheses of Lemma 6.5, with $1 \le r \le db$ and a labelling of the slots with M_{b} label changes and at most m slots per label,

$$(d \dot{-} M_{\mathrm{b}})(d \dot{-} 2m)\, b \;\le\; (M_{\mathrm{b}}+1)D + M_{\mathrm{b}}\big(\lceil \log_2(q+1)\rceil + (k+1)\lceil \log_2(2T+1)\rceil\big) + r\,G'\,(3kM_{\mathrm{b}} + \chi_{\mathrm{cross}}),$$

where $x \dot{-} y = \max(x-y, 0)$ and χ_{cross} is taken for the representative's pair residency.

Outline of the formal proof. Sum the bits form of Lemma 6.5 over the at most $M_{\mathrm{b}}+1$ label classes and bound the left side as in Corollary 4.6. □

6.4 The popular trace

Definition 6.7 (Slot length and horizon)**.** Lean: `cellBits`, `slotLen`, `horizon` [`Tape/DefsM.lean`]

For a machine M let $G = \max(1, \lceil \log_2(g+1)\rceil)$ be the number of bits per work cell. The slot length at size d is $r = \lceil db(d)/G \rceil$, so that a block of r cells holds about one matrix row. For a time constant c the *horizon* is $c_0 = cG + 1$; if $T \le cL$ there are at most $c_0 d$ slots (`numSlots_le_horizon`).

Theorem 6.8 (Popular trace)**.** Lean: `exists_popular_traceClass`, `traceBits_le` [`Tape/Statements/Adaptive.lean`]

Let $1 \le r \le db$ and suppose every matrix input halts within $T_{\max}$ steps. Then for some $T \le T_{\max}$ there is a trace class F with $2^{d^2 b} \le \mathcal{N} \cdot |F|$, where $\mathcal{N} = \mathcal{N}(k, d, \lceil T_{\max}/r \rceil, T_{\max})$ is an explicit bound on the number of pairs (halting time, trace) (`traceCount`). If $T_{\max} \le cL$ and r is the slot length, then $\lceil \log_2 \mathcal{N} \rceil \le C\, d\, b(d)$ for an explicit constant C depending only on M and c (`traceConst`). So the popular class has co-entropy $D = O(db) = o(L)$.

Outline of the formal proof. Pigeonhole over the pairs (halting time, trace): block indices lie in $[-N, N]$, each residency has at most three blocks, each slot touches at most two rows, and output lengths are at most $T_{\max}$. The bound on $\lceil \log_2 \mathcal{N} \rceil$ is arithmetic from $T \le cd^2 b$ and $b \le d \le 2^b$. □

The trace used here records absolute block indices, rows and output lengths per slot, which costs $O(d \log d)$ bits for running time $O(L)$. For machines that write their whole output at the end, a coarser trace that records only block and row moves and residency patterns, $O(1)$ bits per slot, suffices, and the popular class then has co-entropy $O(d)$; Section 9 uses this to show that the coding conjecture is needed only at co-entropy $O(d)$.

7 Theorem M*: the macroscopic motion of the work heads

Discrete Isoperimetry (Corollaries 4.6 and 6.6) says that, for every partition of time into small classes with few boundaries, $\Omega(d)$ revisited blocks must link different classes. This section passes to the limit $d \to \infty$. Rescaled, the head trajectories of a linear-time transposer converge (along subsequences) to 1-Lipschitz paths, and the counting inequality becomes an integral inequality ($\star$) for these paths: in every partition of macroscopic time into finitely many intervals, grouped into classes of measure at most θ, the heads must return to the same positions in different classes with total multiplicity at least $1 - 2\theta$. In particular the heads cannot all sweep monotonically: some head makes a macroscopic U-turn, and together the heads waste at least one unit of range.

The argument uses the cut inequality only, not the machine; we state it for arbitrary walks (Theorem M* for walks) and obtain the machine forms (oblivious, trace classes, adaptive, robust, non-uniform) and, in Section 8, the form for network codes as corollaries.

7.1 Rescaled walks and their limits

Fix $G \geq 1$. At size d the block size is $r = \lceil db(d)/G \rceil$ (`TSDiagram.blockSize`), so a block holds a row. For a walk $h : \mathbb{N} \to \mathbb{Z}$ the *rescaled walk* is

$$P^{(d)}(u) = \frac{1}{d} \Big\lfloor \frac{h(\lfloor ud \rfloor r)}{r} \Big\rfloor,$$

the block at the first step of slot $\lfloor ud \rfloor$, in units of d blocks (`blockTraj`). For a machine, $P_t^{(d)}$ is the rescaled trajectory of work head t on the all-zero input (`rescaledTraj`), or on an input w (`rescaledTrajOf`). If $T \leq cL$, macroscopic time runs over $[0, c_0]$ with $c_0 = cG + 1$.

Lemma 7.1 (Compactness and regularity)**.** Lean: `blockTraj_limit_lipschitz`, `exists_subseq_blockTraj`, `numSlots_le_of`, `frozen_V2`, `pairRes_congr` [`Tape/Statements/Coding.lean`]

Let $d_i \to \infty$ and let h_i be walks with unit steps and block sizes $r_i \geq 1$.

(a) Every uniform limit on $[0, c_0]$ of the rescaled walks is 1-Lipschitz.

(b) If the walks start at 0, then for any k walks per size some subsequence of the rescaled walks converges uniformly on $[0, c_0]$.

(c) If $T \leq cd^2b$ and a block holds a row ($db \leq rG$), there are at most $(cG + 1)d$ slots.

(V2′) For a walk frozen at time T, every block strictly between its blocks at the first steps of two slots $y_1 \leq y_2$ is pair-resident at some slot of $[y_1, y_2]$; and the pair residency depends only on the walk before T.

Outline of the formal proof. (a) A unit-step walk moves at most r cells per slot, that is at most one block per $1/d$ of macroscopic time, up to rounding that vanishes in the limit. (b) Arzelà–Ascoli in the form of a diagonal argument at rational times (a countable product of compact intervals is compact and metrizable) together with asymptotic equicontinuity; the classical statement does not apply directly because the rescaled walks are step functions. (c) Arithmetic. (V2′) is Lemma 4.2(V2) for frozen walks. □

Lemma 7.2 (Machine instances)**.** Lean: `pairRes_V2`, `numSlots_le_horizon`, `rescaledTraj_limit_lipschitz`, `exists_subseq_tendstoUniformlyOn` [`Tape/Statements/Core.lean`]

For every machine M, time constant c and $d_i \to \infty$: some subsequence of the rescaled work-head trajectories converges uniformly on $[0, c_0]$, every such limit is 1-Lipschitz, and if $T \leq cL$ there are at most $c_0 d$ slots. If M halts at T on w, its work heads satisfy (V2′).

Outline of the formal proof. Instances of Lemma 7.1. □

7.2 Levels, labellings and the inequality ($\star$)

Definition 7.3 (Levels and crossings)**.** Lean: `levelSet`, `ConsecIn`, `crossPairs`, `levelCross`, `levelExcess`, `FinUnionIcc`, `PiecewiseConstOn` [`Tape/DefsM.lean`]

Let $P : \mathbb{R} \to \mathbb{R}$ and $c_0 > 0$. The *level set* of p is $\Lambda(p) = P^{-1}(p) \cap [0, c_0]$. An *excised set* S is a finite union of closed intervals, and a *labelling* $\beta : \mathbb{R} \to \mathbb{N}$ is piecewise constant off S: constant on every interval of $[0, c_0] \setminus S$ that avoids a fixed finite set. For a level p with $\Lambda(p)$ finite, $\kappa(p)$ is the number of consecutive pairs $s < s'$ of $\Lambda(p)$ with $s, s' \notin S$ and $\beta(s) \neq \beta(s')$; for $\Lambda(p)$ infinite, $\kappa(p) = 0$. The *excess* of the level is $\#\Lambda(p) - 1$ (truncated at 0, and ∞ for an infinite level set).

So $\kappa(p)$ counts the returns of the path to height p that cross from one label class to another. For k paths P_t on $[0, c_0]$, a number $\theta \geq 0$, an excised set S and a labelling β whose classes off S have measure at most θ, the inequality is

$$(\star) \qquad 1 \;\leq\; \sum_{t<k} \int \kappa_t(p)\,\mathrm{d}p \;+\; 2\theta \;+\; 2k\,\big|S \cap [0, c_0]\big|.$$

Definition 7.4 (The cut inequality)**.** Lean: `CutIneq` [`Tape/DefsM.lean`]
Walks h_t $(t < k)$ of length T satisfy the *cut inequality* at size d with block size r, G bits per cell, slack D and boundary cost s, if for every labelling of the $N = \lceil T/r \rceil$ slots with at most m slots per label and M_{b} label changes,

$$(d \mathbin{\dot{-}} M_{\mathrm{b}})(d \mathbin{\dot{-}} 2m)\, b(d) \;\leq\; (M_{\mathrm{b}} + 1)D + M_{\mathrm{b}}\, s + rG\,\big(3kM_{\mathrm{b}} + \chi_{\mathrm{cross}}\big),$$

where χ_{cross} counts the chords of the pair residencies whose endpoints carry different labels, and $x \mathbin{\dot{-}} y = \max(x - y, 0)$.

For the work heads of a machine on a trace class on which it is correct, it follows from Corollary 6.6 (since $\lceil \log_2(g+1) \rceil \leq G$), with D the co-entropy of the class and $s = \lceil \log_2(q+1) \rceil + (k+1)\lceil \log_2(2T+1) \rceil$; the formal proofs obtain it through the class code and the cut-set bound for codes (Lemma 8.9, Corollary 8.5).

7.3 Theorem M* for walks

Theorem 7.5 (Theorem M* for walks)**.** Lean: `theoremMStar_walk`, `theoremMStar_level_walk` [`Tape/Statements/Coding.lean`]
Let $d_i \to \infty$ and, eventually, let k walks $h_{i,t}$ have unit steps before $T_i \leq cL_i$ and satisfy the cut inequality at size d_i with block size $\lceil d_i b(d_i)/G \rceil$, slack D_i and boundary cost s_i, where $(D_i + s_i)/L_i \to 0$. Let $c_0 = cG + 1$ and consider the rescaled walks frozen at T_i.

(a) If the walks start at 0, some subsequence converges uniformly on $[0, c_0]$.

(b) Every subsequential uniform limit $(P_t)_{t<k}$ consists of 1-Lipschitz paths and satisfies $(\star)$ for every θ, S and β. Moreover

$$\sum_t \int (\#\Lambda_t(p) - 1)\,\mathrm{d}p \;\geq 1, \qquad 1 + \sum_t \big|P_t([0, c_0])\big| \;\leq\; k\,c_0,$$

and the paths are not all monotone.

Outline of the formal proof. The discrete part works at one size. From S and β build a slot labelling: the label of a slot off S is determined by β, and slots in S are cut into short pieces. Its label changes and class sizes are bounded by counting good and bad slots. The *chord split* (`crossChords_split`) bounds the crossing chords by the label changes times $(\lceil \eta d \rceil + 1)$, plus twice the slots in S, plus the *long* crossing chords, those spanning at least ηd slots. Long chords are counted by levels (`levelChords_limsup_le`): for a good level p, the long crossing chords at block $\lfloor pd \rfloor$ inject, for large d, into the pairs counted by $\kappa(p)$, by uniform closeness, (V1) and (V2′). Reverse Fatou then gives $\limsup_d \#\{\text{long crossing chords}\}/d \leq \int \kappa(p)\,\mathrm{d}p$, with an explicit integrable majorant and no measurability assumption on κ. If $(\star)$ failed, the cut inequality would fail at a single large size for explicit η and δ; the arithmetic is isolated in one lemma. The remaining claims are the labelling-free consequences of Theorem 7.6 below. □

The interval labellings and excised sets avoid the area formula and Borel measurability of first returns: only finitely many breakpoints occur.

7.4 Labelling-free consequences

Theorem 7.6 (Excess and range)**.** Lean: `levelCross_le_levelExcess`, `one_le_excess_of_star`, `excess_add_range_le`, `range_add_one_le_of_star`, `lintegral_levelExcess_eq_zero_of_monotone`, `not_all_monotone_of_star` [`Tape/Statements/Core.lean`]

For every path, excised set and labelling, $\kappa(p) \le \#\Lambda(p) - 1$. Let P_t $(t < k)$ satisfy $(\star)$ for every $\theta \in (0, \frac{1}{2})$, S and β. Then:

(a) the total excess multiplicity is at least one: $\sum_t \int (\#\Lambda_t(p) - 1)\,\mathrm{d}p \ge 1$;

(b) if the P_t are 1-Lipschitz on $[0, c_0]$, then $\int (\#\Lambda(p) - 1)\,\mathrm{d}p + |P([0, c_0])| \le c_0$ for each t, and hence $1 + \sum_t |P_t([0, c_0])| \le k c_0$;

(c) a monotone path has total excess $\int (\#\Lambda(p) - 1)\,\mathrm{d}p = 0$, so the P_t are not all monotone.

Outline of the formal proof. For (a), take $S = \emptyset$ and the labelling $\max(0, \lfloor mu \rfloor)$ with $\theta = 1/m$, and let $m \to \infty$. Part (b) is the easy half of Banach's indicatrix [4] for 1-Lipschitz paths, proved by dyadic counting with the range split off. For (c), a level with two preimages under a monotone path is a value on an interval of constancy, and there are countably many such values. □

Lemma 7.7 (Analysis tools)**.** Lean: `countable_localExtr_values`, `levelSet_encard_majorant`, `lintegral_encard_levelSet_le`, `ae_finite_levelSet`, `levelChords_limsup_le`, `crossChords_split` [`Tape/Statements/Core.lean`]

For every $P : \mathbb{R} \to \mathbb{R}$ the values at local extrema form a countable set. For P that is K-Lipschitz on $[0, c_0]$, $\#\Lambda(p)$ has a measurable majorant with integral at most $K c_0$; hence $\int \#\Lambda(p)\,\mathrm{d}p \le K c_0$ and almost every level set is finite. Moreover:

(a) (Chord split.) Let a residency R satisfy (P2), let $\eta \ge 0$, and let a slot labelling λ satisfy $\beta(y/d) \ne \beta(y'/d)$ whenever $y/d, y'/d \notin S$ and $\lambda(y) \ne \lambda(y')$. Then the chords of R with target $< N$ whose ends carry different labels number at most $M_{\mathrm{b}}(\lceil \eta d \rceil + 1) + 2\,\#\{y < N : y/d \in S\} + \#\mathcal{X}$, where $\mathcal{X}$ is the set of long chords $x \to z$: $z < N$, $z - x \ge \eta d$, $x/d, z/d \notin S$, $\beta(x/d) \ne \beta(z/d)$.

(b) (Long chords by levels.) Let $d_i \to \infty$, $\eta > 0$ and $B \in \mathbb{N}$, let P be 1-Lipschitz on $[0, c_0]$, and for large i let $W_i : \mathbb{N} \to \mathbb{Z}$, residencies R_i and $N_i \le c_0 d_i$ satisfy $|W_i(y) - \sigma| \le B$ for $\sigma \in R_i(y)$ and (V2′) for W_i and R_i (if W_i passes strictly over σ between slots $y_1 \le y_2$, then $\sigma \in R_i(y)$ for some $y \in [y_1, y_2]$), with $u \mapsto W_i(\lfloor u d_i \rfloor)/d_i \to P$ uniformly on $[0, c_0]$. Then $\limsup_i \#\mathcal{X}_i/d_i \le \int \kappa(p)\,\mathrm{d}p$ for every excised set S and labelling β.

Outline of the formal proof. Each value at a local extremum is the minimum or maximum of P over a rational interval. The majorant is $\liminf_m N_m(p)$, where $N_m(p)$ counts the dyadic subintervals I of order m such that p lies within $K|I|/2$ of the value of P at the centre of I (an interval of values containing $P(I)$). Each N_m is measurable with integral $K c_0$; once $c_0 2^{-m}$ is below the gaps of a finite subset of $\Lambda(p)$, $N_m(p)$ is at least its size, so $\liminf_m N_m \ge \#\Lambda$ at every level, and Fatou's lemma bounds the integral. For (a), a crossing chord shorter than ηd has its source among the $\lceil \eta d \rceil$ slots before a label change; a chord touching S is charged to its source or target slot in S; by (P2) both charges are injective, and the remaining crossing chords are long. Part (b) is the level count in the outline of Theorem 7.5. □

7.5 Machine forms

Theorem 7.8 (Theorem M*, oblivious machines)**.** Lean: `theoremMStar_level`, `theoremMStar_level_sub`, `theoremMStar_level_subseq`, `theoremMStar_excess`, `no_stationary_limits` [`Tape/Statements/Core.lean`]

Let $d_i \to \infty$ and let M be, eventually, oblivious on $\mathrm{Mat}(d_i, b(d_i))$ with halting time $T_i \le cL_i$ and transpose. If the rescaled work-head trajectories converge uniformly on $[0, c_0]$ to P_t, then the P_t satisfy $(\star)$ for every θ, S, β (equivalently, $\sum_t \int \kappa_t \ge 1 - 2\theta - 2k|S \cap [0, c_0]|$ for $\theta \ge 0$), and the conclusions of Theorem 7.6. A convergent subsequence always exists. In particular the limits cannot all be identically 0.

Theorem 7.9 (Theorem M* for trace classes and adaptive machines)**.** Lean: `rescaledTrajOf_close`, `tendstoUniformlyOn_of_mem`, `mstar_discrete_class_on`, `theoremMStar_level_class`, `theoremMStar_level_class_on`, `theoremMStar_class_nonuniform`, `theoremMStar_level_adaptive`, `no_stationary_limits_class_on` [`Tape/Statements/Adaptive.lean`]

Let $d_i \to \infty$.

(a) Members of one trace class have rescaled trajectories within $4/d$ of each other, so limits along trace classes do not depend on the representatives.

(b) Along trace classes F_i with halting times $T_i \le cL_i$ and co-entropies $D_i = o(L_i)$ on which M is correct, the rescaled trajectories of the representatives satisfy the conclusion of Theorem 7.5 (the labelling-free parts by Theorem 7.6); the underlying finite-d inequality holds at every size. The same holds for non-uniform machines M_i with k tapes, G bits per cell and $o(L_i)$ state bits.

(c) If every matrix input halts within cL_i steps and M transposes, then the popular classes (Theorem 6.8) have $D_i \le Cd_ib(d_i)$, with $C = C(M, c)$ the constant of Theorem 6.8 (`traceConst`), a convergent subsequence of the representatives' trajectories exists, and every subsequential limit is 1-Lipschitz and satisfies $(\star)$.

Theorem 7.10 (Robust and non-uniform M*)**.** Lean: `theoremMStar_level_robust`, `theoremMStar_level_nonuniform`, `theoremMStar_excess_robust`, `theoremMStar_excess_nonuniform` [`Tape/Statements/Adaptive.lean`]

Let $d_i \to \infty$ and suppose that, eventually, M halts within cL_i steps with the transposed output on every input of a set W_i of co-entropy m_i, with $m_i/L_i \to 0$; inputs outside W_i are unconstrained. Then the popular trace classes inside W_i have co-entropy at most $m_i + Cd_ib(d_i)$, convergent subsequences of the representatives' trajectories exist, and every subsequential limit is 1-Lipschitz, satisfies $(\star)$, has total excess at least 1 and total range at most $kc_0 - 1$, and its paths are not all monotone; here $C = C(M, c)$ is the constant of Theorem 6.8. The same holds for machines M_i depending on i, with k work tapes, G bits per cell and $o(L_i)$ state bits (then C depends only on k, G and c).

Outline of the formal proofs of Theorems 7.8 to 7.10. All machine forms reduce to Theorem M* for code-carrying diagrams (Theorem 8.11), hence to Theorem M* for walks. For a trace class, the class code on the representative's diagram (Lemma 8.9) carries the state and head positions as $\lceil \log_2(q+1) \rceil + (k+1)\lceil \log_2(2T+1) \rceil = o(L)$ extra bits per memory edge, and the cut-set bound for codes gives the cut inequality. The oblivious machine is the case of one class with $D = 0$ (Lemma 6.2); the adaptive form is the robust form with W_i all inputs; the robust form extracts the popular class inside W_i (Theorem 6.8, in its form within a set of inputs, `exists_popular_traceClass_on`). Representative independence holds because the work-head blocks of two members of a class differ by at most 2 before halting and 4 after. □

Corollary 7.11 (Theorem M* under (T))**.** Lean: `adaptive_hyps_of_linearTime`, `linearTimeTransposable_popular_limits` [`Tape/Statements/Adaptive.lean`]

If (T) holds, then some machine M and constant c have the following property along every $d_i \to \infty$: there are trace classes of co-entropy $o(L_i)$ (the popular traces), a convergent subsequence of their representatives' rescaled trajectories, and every subsequential limit is 1-Lipschitz and satisfies $(\star)$ on $[0, (2c)G + 1]$.

Outline of the formal proof. A machine meeting Definition 2.1 with constant c halts within $2cL$ steps on every matrix input of every size $d \geq 1$ and transposes (Lemma 2.2); apply Theorem 7.9(c). □

So a linear-time transposer must move its work heads in a macroscopically folding way. The condition is far from sufficient: it constrains only the macroscopic shape of the head motion. It does exclude simple zig-zags: if every head follows the same triangle wave $P \geq 0$ of slope ± 1, the labelling $\beta(u) = \lfloor mP(u) \rfloor$ with m large has classes of measure at most $\frac{1}{4}$ and keeps every level set inside one class, so $\kappa_t \equiv 0$ and $(\star)$ fails at $\theta = \frac{1}{4}$, $S = \emptyset$. Its value is as a test that any candidate construction must pass, and as the limit object behind the coding results of Section 8.

Proposition 7.12 ($(\star)$ is a real constraint)**.** Lean: `theoremMStar_conclusion_can_fail` [`Tape/Statements/Core.lean`]

For the constant path $P = 0$, $S = \emptyset$, $\beta(u) = \max(0, \lfloor 4u \rfloor)$ and $\theta = \frac{1}{4}$, the hypotheses on S, β and θ hold and $(\star)$ fails, for every k and c_0.

Outline of the formal proof. $\kappa \leq \#\Lambda - 1$, and the excess of a constant path vanishes almost everywhere. □

8 Codes on time–space diagrams

The block-level lower bounds of the previous sections (the cut lemmas for unions of slots, Discrete Isoperimetry and Theorem M*) use the transition function only through block locality: what they use is that information flows along the block network of the run, with bounded capacity per edge. This section makes that structure explicit. A *time–space diagram* records only head motions and an output schedule; its block network is a directed acyclic graph with bounded degree; and a *causal network code* on it is an arbitrary assignment of functions to edges. A machine that is correct on a trace class yields such a code (Lemma 8.9), and conversely these block-level results hold for codes (Lemma 8.4, Corollary 8.5, and Theorem 8.11); the cell-level results (the Locality Lemma, Theorems P and P′) are not claimed for codes. Section 9 then asks whether such codes exist at all.

8.1 Diagrams and block networks

Definition 8.1 (Time–space diagram)**.** Lean: `TSDiagram`, `TSDiagram.Valid`, `runDiagram` [`Tape/DefsCE.lean`]

A k-tape *time–space diagram* consists of a length T, walks $h_t : \mathbb{N} \to \mathbb{Z}$ for $t < k$ (the work heads), a walk h_{in} (the input head) and an output schedule $\text{out} : \mathbb{N} \to \mathbb{N}$ ($\text{out}(\tau)$ is the number of output bits before step τ). It is *valid* for output length L if all walks start at 0 and have unit steps, and the output schedule starts at 0, grows by at most one per step, and equals L from time T on. The diagram of a run of M on w_0 up to time T consists of its head trajectories and output lengths.

Definition 8.2 (Block network)**.** Lean: `TSDiagram.blockSize`, `TSDiagram.nodes`, `TSDiagram.memEdges`, `TSDiagram.chordEdges`, `TSDiagram.edges`, `TSDiagram.rowsAt`, `TSDiagram.outNode`, `TSDiagram.visible` [`Tape/DefsCE.lean`]

Let $w = b(d)$, $L = d^2 w$, $G \geq 1$ and $r = \lceil dw/G \rceil$. The block network of a diagram has as nodes the slots $y < \lceil T/r \rceil$, in their causal order; *memory edges* $y \to y+1$; and a *chord edge* $x \to z$ whenever some work tape has a chord from x to z of its pair residency. The rows *attached* at node y (its sources) are the input rows that the input walk touches during slot y; the *sink* of output position o is the slot during which bit o is appended.

Every node has in-degree at most $k+1$: one memory edge, and at most one chord per tape by (P2) (Lemma 4.3). There are at most kN chord edges (`card_chordEdges_le`).

Definition 8.3 (Causal codes, complete exchange, capacities)**.** Lean: `NetCode`, `NetCode.inVals`, `NetCode.edgeVal`, `NetCode.outVal`, `TSDiagram.Solves`, `TSDiagram.CapProfile`, `TSDiagram.blockCap`, `TSDiagram.profileCap` [`Tape/DefsCE.lean`]

A *causal code* assigns to every edge $e = (u, v)$ a function enc_e computing a natural number from the values on the edges into u and the input bits visible at u (the bits of the rows attached at u), and to every output position o a function dec_o computing a bit from the same data at the sink of o. The code is evaluated along the spine. It *solves complete exchange* on a set F of inputs within capacities cap if for every input in F every edge value is below $2^{\mathrm{cap}(e)}$ and every output bit is the transposed input bit. The capacities are *within the block profile* if a memory edge $y \to y+1$ carries at most the blocks resident at slot y (summed over the tapes, at most $3k$) and a chord edge carries one block per tape having that chord, a block being rG bits. The *profile capacities* are the largest such capacities; with s extra bits on every memory edge we speak of the block capacities with s extra bits.

Being a solving code is monotone in the capacities (`solves_mono`), so ruling out codes at the profile capacities rules them out within every capacity in the profile.

The code may compute arbitrary functions. The only restrictions are causality, the capacities, and the locality of the inputs: a row is visible only at the nodes where the input walk touches it.

8.2 The cut-set bound

Lemma 8.4 (Cut-set bound for codes)**.** Lean: `cutset_bound_bitwise`, `cutset_bits_bitwise`, `cutset_bound`, `card_le_of_solves` [`Tape/Statements/Coding.lean`]

Let a causal code solve complete exchange on F within cap*, and let U be a set of nodes. Let B_U be the set of input bits whose row is attached at no node of U and whose transposed position has its sink in U. Then*

$$2^{|B_U|}\,|F| \;\leq\; 2^L \cdot 2^{\sum_{e \text{ entering } U} \mathrm{cap}(e)},$$

so if F has co-entropy at most D, then $|B_U| \leq D + \sum_{e \text{ entering } U} \mathrm{cap}(e)$. In particular $2^{w|S_U|}|F| \leq 2^L 2^{\sum \mathrm{cap}}$, where S_U is the set of pairs of a row not attached in U and a column all of whose positions have their sinks in U. Taking U to be all nodes: rows attached nowhere cost co-entropy, $2^{u\,dw}|F| \leq 2^L$ for u such rows.

Outline of the formal proof. Group the inputs of F by their bits outside B_U. Within a group, the values computed inside U are a function of the values on the edges entering U and of the visible bits in U, which do not depend on B_U; the outputs in U determine B_U. So each group injects into the values on the entering edges. □

Corollary 8.5 (Discrete Isoperimetry for codes)**.** Lean: `isoperimetry_code`, `sum_cross_blockCap_le`, `cutIneq_of_solves` [`Tape/Statements/Coding.lean`]

Let a causal code solve complete exchange on a set of co-entropy at most D, on the block network of a valid diagram with $r \geq 1$. For every labelling of the nodes with M_{b} label changes and at most m

nodes per label,

$$(d \dot{-} M_{\mathrm{b}})(d \dot{-} 2m)\, w \;\le\; (M_{\mathrm{b}}+1)D + \sum_{e:\ \text{labels of the ends differ}} \mathrm{cap}(e),$$

where $x \dot{-} y = \max(x-y,0)$. At the block capacities with s extra bits, the right-hand sum is at most $M_{\mathrm{b}}s + rG(3kM_{\mathrm{b}} + \chi_{\mathrm{cross}})$, so the walks of the diagram satisfy the cut inequality (Definition 7.4) with slack D and boundary cost s.

Outline of the formal proof. Sum the bits form of Lemma 8.4 over the label classes, bound the left side as in Corollary 4.6, and observe that an edge between different labels is a memory edge at a label change or a crossing chord. □

8.3 Distances, flows and the code gap

Definition 8.6 (Distances and routings)**.** Lean: `ball`, `netDist`, `TSDiagram.rowDist`, `IsWalk`, `Routing`, `TSDiagram.RoutesAt` [Tape/DefsCE.lean]

dist is the undirected hop distance in the block network. For an input bit α, rowDist(α) is the distance from the nearest node at which its row is attached to the sink of its transposed position (0 if the row is attached nowhere). A *routing at rate λ* within cap is a fractional multicommodity flow, one commodity per input bit whose row is attached somewhere, sending λ units of each commodity along walks from a node attaching its row to its sink, with total traversals of every edge at most its capacity.

Lemma 8.7 (Weak duality and volume)**.** Lean: `weak_duality`, `rowDist_sum_ge`, `card_chordEdges_le`, `sum_cap_le` [Tape/Statements/Coding.lean]

A routing at rate λ satisfies $\lambda \sum_\alpha \mathrm{rowDist}(\alpha) \le \sum_e \mathrm{cap}(e)$. For a valid diagram with $1 \le r \le dw$ and every j,

$$j\,L \;\le\; \sum_{\alpha} \mathrm{rowDist}(\alpha) + j\big(4N(2k+3)^{j-1}w + u\,dw\big),$$

where N is the number of nodes and u the number of rows attached nowhere. There are at most kN chord edges, and capacities within the profile sum to at most $4kNrG$.

Outline of the formal proof. Weak duality: each unit of commodity α traverses at least rowDist(α) edges. For the distance bound, the ball of radius $j-1$ around a node attaching a row has at most $(2k+3)^{j-1}$ nodes (degree at most $2k+2$), each node attaches at most two rows, and a sink receives at most $2w$ bits of a row, so few bits are within distance $j-1$ of their sinks. □

Theorem 8.8 (The code gap)**.** Lean: `volume_gap`, `flow_rate_lt`, `code_gap` [Tape/Statements/Coding.lean]

Fix k, $G \ge 1$, c and C, and let $E = 4\lceil \log_2(2k+3)\rceil (4k+1)G(c+1)$. For all large d, on every valid k-tape diagram of length at most cL with capacities within the profile and at most C rows attached nowhere,

$$w \sum_e \mathrm{cap}(e) \;<\; E \sum_\alpha \mathrm{rowDist}(\alpha),$$

so every routing has rate $< E/w$. In particular, if a causal code solves complete exchange on a set of co-entropy at most Cdw within such capacities, the inequality holds: the code (rate 1) beats multicommodity flow by a factor larger than $w/E = \Omega(\log d)$.

Outline of the formal proof. Take $j = \lfloor (w-1)/(2\lceil \log_2(2k+3)\rceil)\rfloor$, so that $(2k+3)^{2j} \le d-1$. By Lemma 8.7, all but $o(L)$ bits are at distance at least $j = \Omega(w)$ from their sinks, while $\sum \mathrm{cap} \le 4kG(c+1)L$. Weak duality bounds the rate. For a solving code, co-entropy Cdw leaves at most C rows attached nowhere (Lemma 8.4). □

So a code on a block network of a linear-time diagram, if it exists, beats routing by a factor $\Omega(\log d)$. The connection goes back to Adler, Harvey, Jain, Kleinberg and Lehman [1, Thm. 6.2], who turn an oblivious algorithm for transposition into a network code on its computation graph; here Lemma 8.9 and Theorem 8.10 turn every linear-time transposer into such a code on its own block network (one way only: a code need not come from a machine). The Tape Coding Conjecture says that no such code exists.

8.4 Machines give codes

Lemma 8.9 (Class codes)**.** `Lean: traceClass_code` [`Tape/Statements/Coding.lean`]

Let $d \ge 1$, let F be a trace class with representative w_0 (slot length $r = \lceil dw/G \rceil$, $G = \max(1, \lceil \log_2(g+1)\rceil)$) on which M is correct. Then the diagram of the run on w_0 up to T is valid, and its block network carries a causal code that solves complete exchange on F, where a memory edge $y \to y+1$ carries the state, the $k+1$ head positions and the contents of the resident blocks, at most $\lceil \log_2(q+1)\rceil + (k+1)\lceil \log_2(2T+1)\rceil + r\lceil \log_2(g+1)\rceil \sum_t |R_t(y)|$ bits, a chord edge carries $r\lceil \log_2(g+1)\rceil$ bits per tape having that chord, and no other pair carries anything.

Outline of the formal proof. This is the Block Locality Lemma in functional form. A memory edge $y \to y+1$ carries the state, the head positions and the contents of the resident blocks at time $(y+1)r$, and a chord edge the contents of its blocks at the time of its target slot. The code at a node chooses a member of F consistent with the values it receives and its visible input bits, and sends that member's payloads; the decoder outputs that member's bit. By Lemma 6.3 for the single slot, all consistent members have the same outgoing payloads and output bits, and an induction over the slots shows that on every input of F each edge carries that input's own payload. □

Theorem 8.10 (Fast machines give codes)**.** `Lean: exists_popular_traceClass_on, codes_of_fastOn_bits, solves_profileCap_addTapes, codes_of_fastOn` [`Tape/Statements/Coding.lean`]

Let M be correct within cL steps on every input of a set F_0 of co-entropy at most $C_0 dw$.

(a) If $s \ge \lceil \log_2(q+1)\rceil + (k+1)\lceil \log_2(2cL+1)\rceil$, then there are a valid k-tape diagram of length at most cL (in the proof, the diagram of M's run on the representative of a popular class) and a causal code on it at the block capacities with s extra bits ($G = \max(1, \lceil \log_2(g+1)\rceil)$, $r = \lceil dw/G \rceil$), solving complete exchange on a set of co-entropy at most $(C_0 + C_M)dw$, where C_M depends only on M and c.

(b) Extra bits become tapes: a code at the block capacities with $s \le a\,rG$ extra bits is a code at the profile capacities of the diagram with a additional stationary tapes.

(c) Hence, if $2\lceil \log_2(q+1)\rceil \le dw$ and d is large enough, there is such a code at the profile capacities on a diagram with $k+1$ tapes.

Outline of the formal proof. (a) The popular trace inside F_0 (Theorem 6.8, which holds within any set of inputs) is a class of co-entropy at most $(C_0 + C_M)dw$; apply Lemma 8.9. (b) A stationary tape changes no node, edge, source, sink or chord, and its single resident block carries the extra bits on every memory edge. (c) Combine (a) and (b) with $a = 1$. □

8.5 Theorem M* for code-carrying diagrams

Theorem M* holds for codes: its proof uses the machine only through the cut inequality.

Theorem 8.11 (Theorem M* for code-carrying diagrams)**.** Lean: `theoremMStar_level_diagram`, `theoremMStar_diagram`, `theoremMStar_diagram_profile` [`Tape/Statements/Coding.lean`]

Let $d_i \to \infty$, and let valid k-tape diagrams D_i of length at most cL_i carry causal codes solving complete exchange on sets of co-entropy m_i, at the block capacities with s_i extra bits, where $(m_i + s_i)/L_i \to 0$. Then the rescaled work walks of the D_i have a uniformly convergent subsequence on $[0, cG + 1]$, and every subsequential limit is 1-Lipschitz and satisfies $(\star)$. At the profile capacities ($s_i = 0$) every limit also has total excess at least 1, total range at most $k(cG+1)-1$, and its paths are not all monotone.

Outline of the formal proof. By Corollary 8.5 the walks satisfy the cut inequality with slack m_i and boundary cost s_i; apply Theorem M* for walks (Theorem 7.5). □

Corollary 8.12 (Counterexamples to the conjecture obey $(\star)$)**.** Lean: `mstar_of_robustNoCode_counterexamples`, `mstar_of_not_robustNoCode` [`Tape/Statements/Coding.lean`]

Every family of counterexamples to the Tape Coding Conjecture (Conjecture 9.1): valid diagrams of length at most cL with codes at the profile capacities on sets of co-entropy Cdw, along $d_i \to \infty$, has rescaled work walks whose subsequential limits are 1-Lipschitz, satisfy $(\star)$, have total excess at least 1 and total range at most $k(cG+1)-1$, and are not all monotone. If the conjecture fails, such a family exists, with a convergent subsequence.

Outline of the formal proof. Theorem 8.11 with $m_i = Cd_iw_i = o(L_i)$ and $s_i = 0$. □

9 The Tape Coding Conjecture

9.1 Statement

Conjecture 9.1 (Tape Coding Conjecture)**.** Lean: `RobustNoCode_TM` [`Tape/DefsCE.lean`]

For all k, $G \geq 1$, c and C, the following holds for all sufficiently large d (with $w = b(d)$, $L = d^2w$ and $r = \lceil dw/G \rceil$). No causal code solves complete exchange at the profile capacities on the block network of a valid k-tape time–space diagram of length at most cL, for a set of inputs of co-entropy at most Cdw.

In words: on the block network of any linear-length tape diagram, complete exchange cannot be realized by any coding scheme, even if the scheme only has to work for a $2^{-O(d \log d)}$ fraction of the inputs. By Theorem 8.8, a solving code would beat multicommodity flow by a factor $\Omega(\log d)$. The conjecture can therefore be read as "on tape networks, coding beats routing by $o(\log d)$", and this reading is one of its equivalent forms (Theorem 9.4). The internal name of the conjecture in the Lean library is `RobustNoCode_TM` ("no code for *complete exchange* on *tape-machine* networks, robust form").

The *plain* form, `NoCode_TM`, asks the same only for codes that are correct on all inputs. It refutes oblivious linear-time transposers (Theorem 9.9). Whether plain implies robust is the subject of Section 10.

9.2 The reduction

Theorem 9.2 (The Tape Coding Conjecture refutes (T))**.** Lean: `not_linearTimeTransposable_of_robustNoCode`, `not_linearTimeTransposable_of_robustCE_TM`, `not_linearTimeTransposable_of_robustCE_TM_dist`, `codes_of_linearTimeTransposable`, `not_robustCE_TM_of_linearTimeTransposable` [`Tape/Statements/Coding.lean`]

If the Tape Coding Conjecture holds (in any of its robust forms of Theorem 9.4), then ¬`Transpose.LinearTimeTransposable`*. In contrapositive form, and more strongly, a linear-time transposer yields constants k, $G \geq 1$, c, C such that for* every *large d there is a valid k-tape diagram of length at most cL whose block network carries a causal code, within capacities in the block profile, solving complete exchange on a set of co-entropy at most Cdw (a refutation of the conjecture needs such codes only for infinitely many d).*

Outline of the formal proof. The formal proof goes through Theorem 9.6. The conjecture rules out codes at co-entropy $Cd \leq Cdw$ for every C. A machine meeting Definition 2.1 with constant c is correct within $2cL$ steps on every matrix input of every size $d \geq 1$; after output buffering, the popular class of its coarse trace has co-entropy $O(d)$, and its class code (Lemma 8.9), with the state and the head positions carried by extra stationary tapes (Theorem 8.10(b)), solves complete exchange at the profile capacities at every large size. This contradicts the conjecture at a single large d. The flow and distance forms are equivalent (Theorem 9.4), and the second statement follows because $Cd \leq Cdw$. □

Concretely, a proof `h` of `RobustNoCode_TM` resolves (T) by the file

```
theorem Transpose.answer :  ¬ Transpose.LinearTimeTransposable :=
    Transpose.Tape.not_linearTimeTransposable_of_robustNoCode h
```

which `Verify.lean` (in `lean/`) accepts, once `Transpose/Answer.lean` also imports `Transpose.Tape.Statements.Coding`. The reduction is one-way. A refutation of the conjecture is a choice of constants k, G, c, C with codes for infinitely many d; nothing makes these codes uniform, finite-state, or local to slots, as a machine's code would be, and no converse of Theorem 9.2 is proved. So a refutation would remove this coding obstruction but would not by itself decide (T).

Theorem 9.3 (Consequences for every machine)**.** Lean: `linearTime_fails_of_robustNoCode`, `no_fast_transposer_of_robustNoCode`, `no_fast_transposer_of_robustNoCode_bits`, `linearTime_fails_of_robustNoCode_logBits`, `not_linearTimeTransposable_of_robustNoCode_logBits` [`Tape/Statements/Coding.lean`]

Assume the Tape Coding Conjecture.

(a) For every machine M and constant c, for all large d some $d \times d$ matrix is not transposed within $c(L+1)$ steps: every machine fails at every *large size.*

(b) (Non-uniform, average case.) For all k, g, c, C_0, C_1 and all large d, no machine with k work tapes, $g+1$ work symbols and at most $2^{C_1 dw}$ states (its transition table may depend on d) halts within cL steps with the transposed output on every input of a set of co-entropy at most $C_0 dw$.

(c) (Extra bits.) For every s, if no code exists at the block capacities with $s(k,d)$ extra bits on every memory edge (`RobustNoCode_TM_bits`), then for all k, g, c, C_0 and all large d no machine with k work tapes, $g+1$ work symbols and $\lceil \log_2(q+1) \rceil + (k+1)\lceil \log_2(2cL+1) \rceil \leq s(k,d)$ halts within cL steps with the transposed output on every input of a set of co-entropy at most $C_0 dw$. For $s(k,d) = (3k+4)\lceil \log_2 d \rceil$ this hypothesis also implies (a) and ¬(T).

Outline of the formal proof. Corollaries of the reduction at co-entropy $O(d)$ (Theorem 9.6), whose single-size form applies to each machine separately. The form with extra bits per memory edge is direct: a run's payload on a memory edge is exactly its resident blocks, its state and its head positions. □

9.3 Equivalent forms

Two further forms make the connection with network coding explicit. The *flow form* `RobustCE_TM` says: for every $\varepsilon > 0$ and all large d, whenever a causal code solves complete exchange within capacities cap in the block profile on a set of co-entropy at most Cdw, the demand is routable at rate $1/(\varepsilon w)$ within cap; that is, coding beats multicommodity flow by a factor of at most $\varepsilon \log d$. The *distance form* `RobustCE_TM_dist` replaces the flow rate by its volume bound: $\sum_\alpha \mathrm{rowDist}(\alpha) \le \varepsilon w \sum_e \mathrm{cap}(e)$.

Theorem 9.4 (Equivalent forms)**.** Lean: `robust_tfae`, `robustCE_TM_iff_robustNoCode`, `robustCE_TM_dist_iff_robustNoCode`, `robustNoCode_iff_attached`, `robustCE_TM_att_iff`, `robustCE_TM_att_iff_robustCE_TM`, `robustCE_TM_gamma_iff`, `robustCE_TM_gamma_per_iff`, `robustNoCode_of_gainBound`, `plain_tfae`, `ce_TM_iff_noCode`, `ce_TM_dist_iff_noCode`, `noCode_of_robustNoCode`, `ce_TM_of_robustCE_TM`, `ce_TM_dist_of_robustCE_TM_dist` [`Tape/Statements/Coding.lean`]

The following are equivalent: the Tape Coding Conjecture; its flow form; its distance form; the no-code and flow forms restricted to diagrams whose input walk attaches every row; the no-code form with the uniform profile of $3k$ blocks on every memory edge; and the no-code form with $(3k+4)\lceil \log_2 d \rceil$ extra bits on every memory edge. The γ-form "for some $\gamma(d) = o(\log d)$ with $\gamma > 0$ (one γ for all constants, or one per choice of constants), every solving code's demand is routable at rate $1/\gamma(d)$" is equivalent as well. The explicit gain bound "every code solving complete exchange at the profile capacities has $E \sum_\alpha \mathrm{rowDist}(\alpha) \le w \sum_e \mathrm{cap}(e)$", with E as in Theorem 8.8, implies the conjecture. The plain forms (flow, distance, no code, for codes correct on all inputs) are equivalent to each other, and are implied by the robust ones.

Outline of the formal proof. A solving code at the profile capacities makes the distance form fail by Theorem 8.8, and weak duality (Lemma 8.7) turns any routing at rate $1/(\varepsilon w)$ into the distance form. So each form is equivalent to "no code". The restriction to attached rows loses nothing: append a final input sweep, which keeps every original node, edge and sink and the residency of every complete slot, and let the code mask each node's input to its original rows (the time constant becomes $2c+3$). The profile forms are Theorem 9.5. □

Theorem 9.5 (The capacity profile is immaterial)**.** Lean: `robustNoCode_with_iff`, `robustNoCode_with_k_iff`, `exists_shift_iff_unbounded`, `robustNoCode_with_of_robustNoCode`, `robustNoCode_bits_iff`, `robustNoCode_logBits_iff`, `robustNoCode_iff_forall_cap`, `noCode_iff_forall_cap` [`Tape/Statements/Coding.lean`]

For every unbounded memory profile m (equivalently, $3k \le m(k+a)$ for some a, for every k; for instance $m(k) = 3k$, $3k+1$, or one block per tape), the no-code statement with $m(k)$ blocks on every memory edge is equivalent to the Tape Coding Conjecture. So is the no-code statement with $s(k,d)$ extra bits on every memory edge, for every s with $s(k,d) \le C(k)\,dw$; for instance $s = (3k+4)\lceil \log_2 d \rceil$. The conjecture stated at the profile capacities is equivalent to ruling out codes within every capacity in the profile, and the same holds for the plain form.

Outline of the formal proof. Extra bits and extra blocks are traded for stationary tapes (Theorem 8.10(b)): a stationary tape changes no node, edge, source, sink or chord, and its resident block

can carry any payload of at most one block. Conversely the per-node profile is at most any uniform profile with enough tapes. □

9.4 Weaker sufficient conditions

The conjecture asks for no codes at co-entropy Cdw, at all large d. Much less suffices to refute (T).

Theorem 9.6 (Co-entropy $O(d)$ suffices)**.** Lean: `bufTM_run`, `codes_coentLinear_of_linearTimeTransposable`, `not_linearTimeTransposable_of_coentLinear`, `linearTime_fails_of_coentLinear`, `no_fast_transposer_of_coentLinear`, `not_linearTimeTransposable_of_coentLinear_end`, `not_linearTimeTransposable_of_coentLinear_end_io`, `coentLinear_of_robustNoCode`, `noCode_of_coentLinear` [`Tape/Statements/Coding.lean`]

Write NoCode(m) *for the statement "for all k, $G \geq 1$, c and all large d, no causal code solves complete exchange at the profile capacities on a valid k-tape diagram of length at most cL, for a set of co-entropy at most $m(d)$" (*`NoCode_TM_coent`*).*

(a) (T) yields codes at co-entropy Cd (not Cdw), on diagrams that append their output in their last L steps.

(b) Hence, if NoCode(Cd) *holds for every C, then $\neg$(T); every machine fails at every large size; and the non-uniform average-case statement of Theorem 9.3(b) holds with co-entropy $C_0 d$. It suffices to rule out codes on diagrams with the output at the end, and for infinitely many d.*

(c) This hypothesis lies between the plain and the robust form: the Tape Coding Conjecture implies it, and it implies NoCode(0)*, the plain form.*

Outline of the formal proof. First make the machine write its output only at the end (`Impl.bufTM`): it writes the output on an extra tape, rewinds, and copies it out, halting at $T + 2L + 2$ with the same output. Then the output length at every slot is fixed by the halting time, and a *coarse trace* suffices: block and row moves in $\{-1, 0, 1\}$ together with the resident-block and touched-row patterns within ± 1, at most 24^{k+1} values per slot by (V1) and (V2) of Lemma 4.2 (for the input head: a slot is at most one row long), which determine the block-level trace. The popular coarse class therefore has co-entropy $O(d)$, and the class code applies unchanged. □

Theorem 9.7 (One superlinear level, infinitely many sizes)**.** Lean: `not_linearTimeTransposable_of_coent`, `not_linearTimeTransposable_of_coent_nw`, `not_linearTimeTransposable_of_coentLinear_io`, `not_linearTimeTransposable_of_coent_io`, `coent_io_of_coent`, `coentLinear_io_of_robustNoCode_io`, `coentLinear_io_iff_exists_coent_io`, `coent_io_nw_of_robustNoCode_io`, `not_linearTimeTransposable_of_robustNoCode_io`, `not_linearTimeTransposable_of_robustCE_TM_io`, `robustNoCode_io_of_robustNoCode`, `robust_io_tfae`, `no_fast_transposer_of_coent`, `traceClass_coent_lt` [`Tape/Statements/Coding.lean`]

If $f(d) \geq Cd$ eventually for every C (for instance $f(d) = dw$), then NoCode(f) *refutes (T); so does* NoCode(f) *for infinitely many d. The infinitely-often forms of the conjecture (flow, distance, no code) are equivalent to each other, formally weaker than the conjecture, and still refute (T). For infinitely many d, the single superlinear level and the linear family are equivalent:* NoCode(Cd) *holds for infinitely many d, for every C, if and only if* NoCode(f) *holds for infinitely many d for some f with $f(d) \geq Cd$ eventually for every C. Under* NoCode(m)*, for all large d: no machine with k work tapes, $g + 1$ symbols and $2^{O(dw)}$ states is correct within cL steps on a set of co-entropy m_0 with $m_0 + Kd \leq m(d)$, for a constant K depending only on k, g and c; and every trace class on which such a machine is correct within cL steps has co-entropy larger than $m(d)$.*

Outline of the formal proof. (T) gives codes at every large d, so a single failing size per choice of constants contradicts it. For a single level, $Cd \le f(d)$ from some point on. Of the equivalence of the single-level and the linear infinitely-often families, one direction is monotonicity in the level and the other a diagonal argument. □

Theorem 9.8 (Further equivalent and sufficient forms)**.** Lean: `coentLinear_of_robustCE_TM_lin`, `robustCE_TM_lin_iff`, `not_linearTimeTransposable_of_robustCE_TM_lin`, `hintCodes_linear_of_linearTimeTransposable`, `hintLinear_iff_coentLinear`, `not_linearTimeTransposable_of_noCode_hintLinear` [`Tape/Statements/Coding.lean`]

The flow form at co-entropy Cd is equivalent to NoCode(Cd) *for every C, and refutes (T). (T) gives codes indexed by an $O(d)$-bit oracle hint (Section 10), and ruling out codes with a Cd-bit hint for every C is equivalent to* NoCode(Cd) *for every C, so it refutes (T) as well.*

Outline of the formal proof. As in Theorem 9.4, at the linear rung; hints and co-entropy are interchanged by Theorem 10.1. □

9.5 Oblivious machines

Theorem 9.9 (The plain conjecture refutes oblivious (T))**.** Lean: `codes_of_obliviousLinearTimeTransposable`, `not_obliviousLinearTimeTransposable_of_noCode`, `not_obliviousLinearTimeTransposable_of_ce_TM`, `not_obliviousLinearTimeTransposable_of_ce_TM_dist`, `not_obliviousLinearTimeTransposable_of_noCode_io`, `noCode_io_of_robustNoCode_io` [`Tape/Statements/Coding.lean`]

Let `ObliviousLinearTimeTransposable` *be (T) for a machine that is oblivious at every size. An oblivious linear-time transposer gives codes for independent uniform sources (all inputs). Hence the plain form of the conjecture, in any of its forms, refutes oblivious (T); so does the plain no-code form for infinitely many d. The robust infinitely-often form implies the plain infinitely-often form.*

Outline of the formal proof. For an oblivious machine all inputs form one trace class with $D = 0$ (Lemma 6.2), so the class code works on all inputs, with the state and the head positions on one extra stationary tape (Theorem 8.10(b)). □

10 Plain versus robust

The plain conjecture rules out codes that are correct on *all* inputs; the robust one rules out codes correct on a $2^{-O(dw)}$ fraction of the inputs. Robust implies plain trivially. The plain form already refutes oblivious linear-time transposers (Theorem 9.9); if plain implied robust, it would refute (T). This section isolates the exact gap and proves the implication for several natural classes of codes and sets. The general case is open.

10.1 The gap is an oracle hint

A *hint code* with 2^s hint values consists of a function h of the whole input and a family of causal codes $(C_v)_v$ such that on every input x the code $C_{h(x)}$ solves complete exchange for x. It models an oracle that sees the whole input and tells every node s bits; the hint is uncharged (no edge carries it and no node computes it). Write $\mathrm{NoCode}^{\mathrm{hint}}(s)$ for "no hint code with $2^{s(d)}$ values exists" (`NoCode_TM_hintBits`), with the same quantifiers as the conjecture.

Theorem 10.1 (Robust is plain with an oracle hint)**.** Lean: `robustNoCode_iff_hint`, `not_linearTimeTransposable_of_noCode_hint`, `hintCodes_of_linearTimeTransposable`, `noCode_of_noCode_hint`, `solves_shiftCode`, `exists_shift_cover`, `hint_of_code`, `code_of_hint`, `robustNoCode_iff_coent`, `noCode_iff_coent_zero`, `coent_mono`, `hintBits_of_coent`, `coent_of_hintBits`, `noCode_bits_iff` [`Tape/Statements/Coding.lean`]

(a) On a fixed diagram and at fixed capacities: a hint code with 2^s values gives a code on a set of co-entropy s; a code on a set of co-entropy m gives a hint code with $2^m(L+1)$ values.

(b) The Tape Coding Conjecture is equivalent to: for every C, no hint code with a Cdw-bit hint exists. With a constant hint this gives the plain form. (T) yields, for every large d, a single valid diagram with codes indexed by an $O(dw)$-bit hint solving complete exchange on every input.

(c) The co-entropy ladder: NoCode(0) *is the plain form,* NoCode(Cdw) *for every C is the robust form, the rungs are monotone, and hints and co-entropy agree up to $3w+1$ bits.*

(d) A side channel whose contents the code computes itself is free: for $s(k,d) \le C(k)\,dw$, the plain form with s extra bits on every memory edge is equivalent to the plain form.

Outline of the formal proof. (a) In one direction take the most popular hint value. In the other, a code for F is a code for every shift $F \oplus z$ (each node XORs its visible rows with z, and each output bit is XORed back), and $2^m(L+1)$ shifts of F cover every input, by counting with Bernoulli's inequality. (b) and (c) follow from (a), from Theorem 9.6 for the statement about (T), and from monotonicity in the co-entropy (a set of co-entropy 0 is the set of all inputs). (d) Extra memory bits become stationary tapes, as in Theorem 9.5. □

So the entire gap between plain and robust is whether an $O(dw)$-bit oracle broadcast can be replaced by one that the network computes itself. For the application to (T), co-entropy $O(d)$ suffices (Theorem 9.6), so only hints of $O(d)$ bits matter; this is still $\omega(1)$ bits. The first rung of the ladder is already open: does "no code on all inputs" imply "no code on half of the inputs"? This is a one-shot question of zero error versus constant error for this class of networks.

10.2 Partial results

Throughout this subsection we assume the plain conjecture `NoCode_TM` and conclude, for all constants and all large d, that certain codes do not exist at the profile capacities. The constructions prepend or append computations to the diagram (an input sweep, a selection phase, copies of the work walks), so they change k and c by constant factors; this is harmless because the plain conjecture quantifies over all constants.

Theorem 10.2 (Affine sets and additive hints)**.** Lean: `PlainRobust.noCode_affine_of_noCode`, `PlainRobust.noCode_of_affine_subset`, `PlainRobust.noCode_additive_of_noCode`, `PlainRobust.noCode_additiveCover_of_noCode`, `PlainRobust.noCode_affine_exc_of_noCode` [`Tape/Statements/PlainRobust.lean`]

Under the plain conjecture, no code exists on a set F that contains a nonempty affine set $\{x : \sum_i g_i(\text{row } i \text{ of } x) = b\}$, where the g_i are $\mathbb{F}_2$-linear maps to $\mathbb{F}_2^s$ with $s \le Cdw$; more generally, on a set F such that, for an additive hint $\sum_i g_i(\text{row } i)$ of $s \le Cdw$ bits and $K = O(1)$ shift tables, every input is moved into F by one of the shifts; and on such an affine set from which a set E with $2^s \cdot 2|E|^2 < 2^L$ is removed.

Outline of the formal proof. Prepend an input sweep: the input head goes from 0 to $L-1$ and back while the work heads are parked, so every row is read in the prefix and the original diagram follows unchanged. The prefix computes the additive hint along its spine, and the original part runs the code shifted by the shift table entry of the hint, which it reads from the low s bits of its memory edges. This is a syndrome trick (decoding the coset of the input from its syndrome), in the language of tape diagrams. The version with K shifts adds a selection among the K shifted codes (Theorem 10.4), and the version with exceptions adds voting (Theorem 10.3). □

Theorem 10.3 (Codes failing on few inputs)**.** Lean: `PlainRobust.noCode_cosmall_of_noCode` [`Tape/Statements/PlainRobust.lean`]

Under the plain conjecture, no code is correct on all inputs outside a set E with $2|E|^2 < 2^L$.

Outline of the formal proof. Triple every work walk; the block network is unchanged and the profile capacities triple. Run three shifted copies of the code side by side, each clamped to its capacities (so a copy that errs still fits), with shifts $0, z_1, z_2$ such that $z_i \oplus z_j \notin E \oplus E$, and output the majority of the three decoded bits: on every input at most one copy errs. □

Theorem 10.4 (Selection among candidates)**.** Lean: `PlainRobust.noCode_hintConst_of_noCode`, `PlainRobust.noCode_iff_hintConst`, `PlainRobust.noCode_netCover_of_noCode`, `PlainRobust.noCode_netHint_of_noCode` [`Tape/Statements/PlainRobust.lean`]

Under the plain conjecture, no hint code with a constant number of hint bits exists; equivalently, the plain conjecture is unchanged by a constant oracle hint. Moreover, no code exists on a set F for which a network-computable *hint of $O(dw)$ bits (computed by a causal code within the profile capacities of a linear-length prefix diagram) together with $O(1)$ shift tables moves every input into F; and no family of codes indexed by a network-computable hint of $O(dw)$ bits solves complete exchange on every input.*

Outline of the formal proof. Run the K candidate codes side by side (clamped and packed). Each node compares the candidates' would-be output bits at its own output positions and, for every pair of candidates without a recorded difference, records one position where they differ, with the first candidate's bit; this state travels along the spine. An appended input sweep fetches the true bit at every recorded position, and a fresh copy of the diagram runs a candidate that never lost a comparison. A correct candidate never loses, and every incorrect one loses against it, so the winner is correct. This is discrimination, not verification: telling two candidates apart takes one fetched bit. For network-computable hints, concatenate the prefix computation with the diagram and carry the hint along the spine. □

Theorem 10.5 (Linear and nearly linear codes)**.** Lean: `PlainRobust.noCode_linear_of_noCode`, `PlainRobust.noCode_linearExcept_of_noCode`, `PlainRobust.noCode_linearPlus_of_noCode`, `PlainRobust.noCode_linearPlusImage_of_noCode` [`Tape/Statements/PlainRobust.lean`]

Under the plain conjecture, no $\mathbb{F}_2$-linear code solves complete exchange on a set of co-entropy at most Cdw. The same holds for codes that are linear outside a set of edges carrying $O(dw)$ bits, for codes that are linear plus nonlinear corrections of $O(dw)$ bits in total, and for codes that are linear plus corrections whose values on the inputs of F take at most $2^{O(dw)}$ joint values.

Outline of the formal proof. For a linear code the set of inputs on which it is correct is closed under XOR, hence a subspace; if it contains a set of co-entropy Cdw it has codimension at most Cdw, so it is the kernel of an additive syndrome map, and Theorem 10.2 applies. For the nearly linear codes, the nonlinear values form an internally computed hint of $O(dw)$ bits: replacing them by constants gives an affine code, a $2^{-O(dw)}$ fraction of the good inputs share their nonlinear values by pigeonhole, and XOR-ing these inputs with one of them gives inputs on which the linear part is correct. □

Theorem 10.6 (Bounded-degree constraints and decompositions)**.** Lean: `PlainRobust.noCode_lowDegree_of_noCode`, `PlainRobust.noCode_quadForm_of_noCode`, `PlainRobust.noCode_decomp_of_noCode`, `PlainRobust.noCode_crossQuad_of_noCode` [`Tape/Statements/PlainRobust.lean`]

Under the plain conjecture, no code exists on a set containing the zero set of a Boolean function of degree at most D (for constant D, and the zero set nonempty), in particular the zero set of a quadratic form. No code exists on a set that, for a partition of the input positions into $K = O(1)$ parts, contains every input supported on a single part; for instance the common zero set of any family of quadratic forms each of whose monomials joins two different parts.

Outline of the formal proof. If f vanishes identically, the empty list of directions will do. Otherwise let $D' \le D$ be the largest order of a discrete derivative of f that is not identically zero, along directions $u_1, \dots, u_{D'}$; it is constant (its own derivatives vanish), hence equal to 1, and $D' \ge 1$ because the zero set is nonempty. At every x the $2^{D'}$ values $f(x \oplus \sum_{i \in S} u_i)$ then sum to 1, so one of them is 0: the translates of the zero set by these $2^{D'} \le 2^D$ vectors cover all inputs, and selection among the 2^D shifted codes (Theorem 10.4) applies. For a decomposition, run K copies of the code, copy j on the input masked outside part j, which lies in F, and XOR the outputs: transposition is linear. □

Theorem 10.7 (Errors)**.** Lean: `PlainRobust.noCode_fewErrors_of_noCode`, `PlainRobust.noCode_sparseErrors_of_noCode`, `PlainRobust.noCode_sparseEntryErrors_of_noCode`, `PlainRobust.noCode_fewErrorsCover_of_noCode` [`Tape/Statements/PlainRobust.lean`]

Under the plain conjecture, every code that stays within the profile capacities on every input has more than 2^{Hdw} distinct error vectors $C(x)^{\mathsf{T}} \oplus x$, for every constant H. In particular, no such code gets all but at most Ad output bits right on every input (transposition with $O(d)$ wrong bits is as hard as exact transposition); the same holds for $O(d)$ wrong entries, and for a choice among $O(1)$ such codes per input.

Outline of the formal proof. Syndrome correction. Fix a check matrix Φ with $t = 2Hdw + 1$ rows that is injective on the possible error vectors (a random one is). Run the code; every output bit equal to 1 adds the corresponding column of Φ on the spine; an appended input sweep adds the column of every input bit equal to 1, so the spine carries the syndrome $\Phi e(x)$; a second run of the code XORs each output bit with the error bit decoded from the syndrome. The t extra bits are paid for by stationary tapes. □

Theorem 10.8 (Column covers)**.** Lean: `PlainRobust.noCode_colCover_of_noCode`, `PlainRobust.noCode_rowMapCover_of_noCode`, `PlainRobust.noCode_colAggregate_of_noCode` [`Tape/Statements/PlainRobust.lean`]

Under the plain conjecture, no code exists on a set F for which $K = O(1)$ locally computable maps Φ_κ into F exist, each keeping the input columns of its class κ (the classes partitioning the columns); in particular for maps acting on rows. For even d, no code exists on a set containing $\{x : \sum_j v(\text{column } j \text{ of } x) = 0\}$ for a fixed v with values in $\mathbb{F}_2^m$.

Outline of the formal proof. Run K copies, copy κ on $\Phi_\kappa(x)$ (each node computes the copy's visible input from its own), and read column j from the copy of its class. For the aggregate constraint, the inputs $[x_{\text{left}} \mid x_{\text{left}}]$ and $[x_{\text{right}} \mid x_{\text{right}}]$ satisfy it, since each term appears twice. □

10.3 Why the general case is hard

We record, without formal proofs, why these methods stop short of the general implication.

- *A general hint-removal theorem must use the code, not only the size of F.* A counting argument over (automaton, shift table) pairs shows that a random set of density 2^{-m} has no streaming covering hint of $O(dw)$ bits.
- *A hint is cheap in information but not in flow.* An s-bit broadcast tells at most s bits about all outputs together, but delivering it to $\Theta(d)$ nodes costs $\Theta(ds)$ bits of capacity in aggregate, which is $\Theta(L)$ for $s = \Theta(dw)$; edge-removal style arguments do not apply as they stand. The first rung of the ladder is a one-shot instance of zero error versus constant error, related to the edge-removal problem and to the continuity of network capacity under dependent sources [20].
- *The machine-side view.* Plain would refute (T) if (T) implied oblivious (T), or merely a linear-time transposer whose block-level trace and halting time do not depend on the input. Codes of trace classes are self-detecting: at every slot the simulation knows whether the run still follows the popular trace. A hint-removal argument restricted to self-detecting codes would suffice; none is known.

11 Barriers

This section explains why the Tape Coding Conjecture is hard to prove and why a proof must use the shape of tape networks. Its results are not formalized; each gets a complete proof.

11.1 Multiplication

By Harvey and van der Hoeven [17, Cor. 6.9], a multiplication machine running in time $M(m)$ yields a transposition machine for $d \times d$ matrices with b-bit entries running in time $O(\ell_0\, \overline{M}(m))$, where $m \geq d^2 b$, $\ell_0 = \max(1, \lg^* d - \lg^* b)$ and $\overline{M}(m) = m \max_{k \leq m} M(k)/k$. For $b = \lceil \log_2 d \rceil$ we have $\ell_0 = 1$, so a linear-time multiplication machine gives a linear-time transposer (their transposition machine receives d, b and m as part of its input; computing them from the input length takes linear time). Hence $\neg$(T), and in particular the Tape Coding Conjecture, implies that integer multiplication cannot be done in linear time on multitape Turing machines. No superlinear lower bound for off-line multiplication on multitape machines is known; the classical $\Omega(n \log n)$ bound of Paterson, Fischer and Meyer is for on-line multiplication [26].

11.2 General graphs: the circuit barrier

We use the following elementary model. Let H be an undirected graph with source vertices $a_0, \ldots, a_{n-1}$ and sink vertices $b_0, \ldots, b_{n-1}$, every edge of unit capacity. The *complete bipartite demand* asks to deliver, for every pair (i, j), a message from a_i to b_j.

- A *routing at rate* λ is a fractional multicommodity flow that sends λ units from a_i to b_j for every pair, with total flow at most 1 on every edge.
- A *causal code with N rounds* fixes an acyclic orientation of H and, in each round, sends one bit along every edge in its orientation, each bit being a function (which may depend on the round) of the bits that entered the tail in the same round and, at a source a_i, of a_i's messages. Each pair (i, j) is assigned a round in which b_j must output the message bit m_{ij}, a function of the bits entering b_j in that round; the messages m_{ij} are arbitrary bits, and the code must work for all of them. Its *rate* is $1/N$ per pair and round.

In Proposition 11.2 we use three extensions of this model. A source may be attached at a set of vertices: at each of them the code may use its messages, also in the output functions, a routing may start at any of them, and $\mathrm{dist}_H(a_i, b_j)$ is the distance from the nearest. The output functions, too, may depend on the round. And edges may carry symbols of κ bits per round, against routings with capacity κ per edge; then (1) below reads $\lambda \sum_{i,j} \mathrm{dist}_H(a_i, b_j) \le \kappa |E(H)|$. The ratio of the rate of a causal code to the largest routing rate is its *coding gain*. A routing needs one unit of capacity per edge traversed, so for every routing,

$$\lambda \sum_{i,j} \mathrm{dist}_H(a_i, b_j) \;\le\; |E(H)|. \tag{1}$$

Proposition 11.1 (Shift circuits are coding gains)**.** Not formalized; a complete proof is given below. *Let C be a Boolean circuit with s gates of fan-in at most two (arbitrary gate functions) whose inputs are $x \in \{0,1\}^n$ and a shift input, such that for every $k < n$ some value of the shift input makes the outputs equal the cyclic shift $x_{i-k \bmod n}$ at output i. Then there is a graph H of maximum degree at most 4 with $O(s+n)$ vertices and a causal code with n rounds for the complete bipartite demand on H whose coding gain is at least $c\, n \log n/(s+n)$, for an absolute constant $c > 0$ and all large n.*

For example, a circuit multiplying two n-bit integers, followed by n OR gates, is such a circuit: for $y = 2^k$ the $2n$-bit product $p = x\,2^k$ has p_j and p_{j+n} never both equal to 1, so the cyclic shift is $p_j \vee p_{j+n}$.

Proof (not formalized). The graph. Take the circuit as a directed acyclic graph on its gates, the inputs $x_0, \dots, x_{n-1}$ and the shift-input bits that feed some gate (at most $3s+n$ vertices; parallel edges allowed). Replace every vertex of fan-out $f > 2$ by a binary tree of $f-1$ copy vertices, so that in-degrees and out-degrees are at most 2; this adds at most $2s+n$ vertices. Let H be the underlying undirected graph, with a_i the input vertex of x_i and b_j the vertex of output j. Its maximum degree is at most 4 and it has $O(s+n)$ vertices.

One code per shift. Fix $k < n$ and fix the shift input to a value realizing the shift by k. Every vertex then computes a fixed function of its in-edges, so the circuit, read in topological order with one bit per edge, is a causal code on H (with the circuit's orientation) that delivers x_i from a_i to $b_{i+k \bmod n}$.

Time-sharing. Run n rounds; in round k use the code for shift k with the source a_i sending the message $m_{i,i+k \bmod n}$ as x_i. Every pair (i,j) is served in round $j - i \bmod n$, and every edge carries one bit per round. This is a causal code with n rounds, of rate $1/n$.

The flow bound. In a graph of maximum degree 4 at most 4^{t+1} vertices lie within distance t of a vertex. With $t = \lfloor \frac{1}{2} \log_4 n \rfloor - 1$, at most $\sqrt{n}$ sinks lie within distance t of a_i, so $\sum_{i,j} \mathrm{dist}(a_i, b_j) \ge n(n-\sqrt{n})(t+1) \ge \frac{1}{5} n^2 \log_4 n$ for large n. By (1), every routing has rate $\lambda \le 5|E(H)|/(n^2 \log_4 n) = O((s+n)/(n^2 \log n))$. The coding gain is therefore at least $(1/n)/\lambda = \Omega(n \log n/(s+n))$. □

So if, on every bounded-degree graph, complete bipartite exchange had coding gain at most $\gamma(n)$, then every circuit computing all cyclic shifts, and every multiplication circuit, would have size $\Omega(n \log n/\gamma(n))$. With $\gamma = o(\log n)$ this is a superlinear circuit lower bound for an explicit function, which is far beyond current techniques: the best lower bounds for explicit functions are $3.1n - o(n)$ over the full binary basis [21]. A proof of the Tape Coding Conjecture must therefore use the structure of tape networks: unless it also yields a superlinear circuit lower bound for an explicit function, it cannot be a statement about bounded-degree networks in general. The single-shift version of this argument, conditional on the network coding conjecture, is due to Afshani, Freksen, Kamma and Larsen [2], and the step from a small circuit to a coding gain is also observed by Braverman and He [7].

Tape networks are not simple either. The chords of one work tape form two non-crossing families along the time spine, excursions to the right and to the left, so a k-tape network has a book embedding with $2k$ pages, while some families of bounded-degree (bipartite) expanders have 3-page book embeddings [10]. We expect, but do not prove, that after spreading each vertex over a constant number of slots every page can be realized by the excursions of one work head, so that tape networks with a constant number of work tapes contain subdivided bounded-degree expanders, on which cut bounds would not exclude a code. This is the heuristic reason why we do not expect cut arguments alone to prove the conjecture.

11.3 Tape networks: an oblivious separation

Programmable rotation takes $x \in \{0,1\}^L$ and $\rho < L$ (in binary, $\lceil \log_2 L \rceil$ bits appended to x) and outputs x rotated by ρ. A non-oblivious machine computes it in time $O(L)$: copy x to a work tape, then count ρ down in binary while moving a head ρ cells (amortized $O(1)$ per decrement), and output the work tape cyclically from that head. Oblivious machines can rotate in time $O(L \log L)$, by a barrel shifter or by the oblivious simulation of Pippenger and Fischer [28].

The *bit-level network* of an oblivious machine M at input length $L + \lceil \log_2 L \rceil$ has a vertex for every step $\tau < T$; a spine edge $\tau \to \tau + 1$; and, for every work tape and every pair of consecutive visits $\sigma < \tau$ of its head to one cell, an edge $\sigma \to \tau$. The source of input bit x_i is attached at the steps where the input head reads cell i, and the sink of output bit o is the step that appends it. All of this is the same for every input, because M is oblivious.

Proposition 11.2 (Rotation codes on oblivious tape networks)**.** Not formalized; a complete proof is given below.

Let M be an oblivious machine with k work tapes that computes programmable rotation within $T(L) \le L \log_2 L$ steps. Then its bit-level network carries a causal code with L rounds for the complete bipartite demand between the input bits and the output bits whose coding gain over routing, with every edge of capacity $\kappa = \max(\lceil \log_2(q+1) \rceil, \lceil \log_2(g+1) \rceil)$, is at least $c\, L \log L / T(L)$ for a constant $c > 0$ depending only on k, q and g, and all large L.

Proof (not formalized). Fix $\rho < L$. With the bits of ρ fixed, the run of M is a causal code on its bit-level network: the spine edge out of step τ carries the state, and the edge from one visit of a cell to the next carries the symbol written at the first visit; every step computes its outgoing values from its incoming ones and the input bit it reads. It delivers x_i to output position $i - \rho \bmod L$. Time-sharing over the L values of ρ, as in Proposition 11.1, serves every pair (input bit, output bit) once in L rounds.

For the flow bound, every vertex has degree at most $D = 2k + 2$. Let μ_i be the number of steps at which the input head reads cell i; then $\sum_i \mu_i \le T$. Within distance t of the steps attaching x_i lie at most $\mu_i D^{t+1}$ steps, hence at most $\mu_i D^{t+1}$ sinks. With $t = \lfloor \frac{1}{2} \log_D L \rfloor - 1$, at most $T\sqrt{L} \le L^{3/2} \log_2 L = o(L^2)$ pairs are within distance t, so $\sum_{i,o} \mathrm{dist} \ge \frac{1}{2} L^2 (t+1)$ for large L. The network has at most $(k+1)T$ edges, each of the stated capacity κ, so a routing has rate $\lambda \le (k+1)T\kappa / (\frac{1}{2} L^2 (t+1)) = O(T/(L^2 \log L))$, while the code has rate $1/L$. The gain is $\Omega(L \log L / T)$. □

So a bit-level analogue of the Tape Coding Conjecture, asserting that the complete bipartite demand between input and output bits on bit-level networks of oblivious machines running in linear time, at the capacities of Proposition 11.2, has coding gain $o(\log L)$, would imply that oblivious machines need time $\omega(L)$ for programmable rotation, while non-oblivious machines need only $O(L)$. To our knowledge, no superlinear *off-line* separation of oblivious and non-oblivious multitape

machines is known: the optimality claimed by Pippenger and Fischer is for on-line simulation [28], and the known superlinear lower bounds specific to oblivious machines are on-line or for one head [31]. The block version used for transposition and the bit version used here are not known to imply each other. Independently of this calibration, by Theorem 9.9 a proof of the conjecture would show that oblivious machines cannot transpose in linear time, a lower bound that current techniques do not give. By contrast, Theorem P and Theorem M* are unconditional, and the network coding conjecture, which would have given such bounds for two-tape oblivious machines directly [1], is false [7, 32].

12 Limits of XOR-frame recursions

The only technique claimed to beat $O(L \log d)$ for transposition on tapes is an unrefereed construction of OpenAI [25], which claims $O(L(\log d)^{1-\kappa})$ for a tiny $\kappa > 0$. Its core is an address-interchange primitive that mixes data by XOR in sheared address frames, applied recursively. This section abstracts the recursion and shows that it cannot give linear time: every run realizing a gadget makes at least as many frame changes as the gadget has wires (Theorem 12.4), so every gadget has cost at least W (Proposition 12.9, a translation from gadgets to runs that we prove in full but have not formalized). The results are about an abstract model, so they rule out a class of constructions, not linear-time transposition.

12.1 The framework

Fix $m \geq 2$. Two address chunks H and D are to be interchanged; write each as a vector of m fields. For a rational $m \times m$ matrix M the *frame* Φ_M is the address map $(H, D) \mapsto (H + MD, D)$; frames compose additively, $\Phi_A \Phi_B = \Phi_{A+B}$. Changing the frame of a stream from A to B costs $\operatorname{rank}(B - A)$ recursive interchanges of narrower chunks.

Definition 12.1 (Gadget)**.** Lean: `Transpose.ShearFrame.Gadget`, `Transpose.ShearFrame.Gadget.cost` [`Frames/Gadget.lean`]

A *gadget* has W wires; wire w starts in a source frame S_w, passes through the frames of its gates in time order, and ends in a sink frame T_w. Gates are XOR operations $i \leftarrow i + j$ between two distinct wires in a common frame. The scalar network permutes the wires by a permutation ρ (G1), and $T_{\rho(w)} = S_w + I$ (G2): every wire ends sheared by the identity. The *cost* s of the gadget is the sum over wires of the ranks of consecutive frame differences along the wire, from source to sink. Formally, `Transpose.ShearFrame.Gadget` records ρ, the source frames and the frame paths subject to (G2); the gates and (G1) enter only through the run model of Definition 12.3 and the translation of Proposition 12.9.

The recursion built on a gadget distributes a stream over the W wires and runs the gadget, each frame change being a recursive interchange of narrower chunks. Its cost per unit of data satisfies $F_k \leq (s/W)F_{k-1} + O(1)$ at width m^k, and every implementation in the template also satisfies $F_k \geq (s/W)F_{k-1} + c_0$. A gadget with $s < W$ would therefore give linear-time transposition, a gadget with $s/W = m^\tau$, $0 < \tau < 1$, would give $O(L(\log d)^\tau)$, and $s \geq W$ excludes linear time within the template; $s = W$ would give $\Theta(L \log \log d)$, but for rational frames with $m \geq 2$ and $W \geq 1$ no gadget has $s \leq W$ (Proposition 12.9(b)). (This reading of the template is informal: it rests on the preprint's construction [25], which we use only to motivate the model.)

12.2 Cost bounds

Theorem 12.2 (Cycle bound and projection bound)**.** Lean:
`Transpose.ShearFrame.Gadget.invariant_cost_ge`,
`Transpose.ShearFrame.Gadget.cost_ge_of_invariant_family`,
`Transpose.ShearFrame.Gadget.cost_ge_m` [`Frames/Gadget.lean`]
Lean: `Transpose.ShearFrame.PGadget.two_mul_cost_ge` [`Frames/GadgetProjection.lean`]

Every nonempty ρ-invariant set of wires has total cost at least m, so $s \geq jm$ for j disjoint such sets, and $s \geq m$ if $W \geq 1$. If every wire passes through at least one gate and every gate frame is idempotent (a projection, as in the preprint), then $2s \geq Wm$.

Outline of the formal proof. Rank is subadditive, so the cost of a wire is at least $\operatorname{rank}(T_w - S_w)$, and over a nonempty ρ-invariant set C, $\sum_{w \in C}(T_w - S_w) = |C|I$ has rank m. For idempotent frames, $\operatorname{rank}(P_V - P_U) \geq |\dim V - \dim U|$ and $\operatorname{rank}(P_A + P_B) \geq \max(\dim A, \dim B)$; pairing the source term of w with the sink term of $\rho(w)$ and optimizing gives $2s \geq Wm$. □

The general lower bound uses a run model in which frames are arbitrary and only frame changes are counted.

Definition 12.3 (Runs)**.** Lean: `Transpose.ShearFrame.Cfg`, `Transpose.ShearFrame.Event`,
`Transpose.ShearFrame.init`, `Transpose.ShearFrame.Cfg.allValid`,
`Transpose.ShearFrame.moves` [`Frames/GadgetRuns.lean`]

A configuration assigns to each of W wires a frame (from any set F) and a vector in $\mathbb{F}_2^W$. An event is either a gate $i \leftarrow i + j$, allowed only for $i \neq j$ at a common frame, or a move of one wire to a frame (a move to the wire's current frame changes nothing; the others are *frame-changing*). A run is a sequence of events applied to the configuration in which wire u is at frame S_u holding the unit vector e_u; it is *valid* if every gate is allowed when it occurs. The number of moves of a run counts all its moves, frame-changing or not. For matrix frames, the *jump* of a move is its new frame minus its old frame.

Theorem 12.4 (At least W frame changes)**.** Lean: `Transpose.ShearFrame.moves_ge_W`,
`Transpose.ShearFrame.cost_ge_W`,
`Transpose.ShearFrame.moves_ge_W_matrix` [`Frames/GadgetRuns.lean`]
Lean: `Transpose.ShearFrame.vmoves_ge_W` [`Frames/GadgetCoarse.lean`]

Let $\operatorname{shift} : F \to F$ *have no fixed point. Every valid run that ends with wire $\rho(u)$ at frame* $\operatorname{shift}(S_u)$ *holding e_u, for every u, contains at least W frame-changing moves, hence at least W moves; so, if every move costs at least* 1*, its total cost is at least W. For rational $m \times m$ matrix frames with $m \geq 1$ this applies with* $\operatorname{shift}(S) = S + I$.

Outline of the formal proof. Let σ_f be the span of the vectors on the wires at frame f, and N_f the span of the e_u with $\operatorname{shift}(S_u) = f$. The potential $\sum_f \dim(\sigma_f \cap N_f)$ is unchanged by gates (they act within one frame by transvections), grows by at most one per frame-changing move (a move to the current frame changes nothing), is 0 initially (no unit vector starts at its target, since shift has no fixed point), and is W at the end. □

Theorem 12.5 (Mixing across frames does not help)**.** Lean:
`Transpose.ShearFrame.GroupAlgebra.changes_ge_W`,
`Transpose.ShearFrame.GroupAlgebra.cost_ge_W` [`Frames/GroupAlgebraRuns.lean`]

In the group-algebra model, where each wire holds a formal sum of sheared items (a function from an additive group of frames to $\mathbb{F}_2^W$), gates $i \leftarrow i + j$ are allowed between any wires, and a change shifts the content of one wire, every run from the configuration in which wire u holds exactly item u

at frame S_u to one in which wire $\rho(u)$ holds item u at frame $S_u + I$ and nothing else ($I \neq 0$) makes at least W changes.

Outline of the formal proof. The span of the vectors $w \mapsto$ (content of w at frame $I + S_u)_u$ has dimension 0 at the start and W at the end; gates do not increase it and each change increases it by at most one. □

12.3 Coarsening and integer frames

Theorem 12.6 (Coarsening)**.** Lean: `Transpose.ShearFrame.vmoves_ge_W`, `Transpose.ShearFrame.visible_ge_W_of_coarse`, `Transpose.ShearFrame.det_ne_zero_of_moves_le`, `Transpose.ShearFrame.rank_sum_of_moves_le` [`Frames/GadgetCoarse.lean`]

For every map $\pi : F \to F'$ with $\pi(\mathrm{tgt}(u)) \neq \pi(S_u)$ for all u, every valid run as in Theorem 12.4 (with targets $\mathrm{tgt}(u)$) makes at least W moves that change the π-class of the moved wire. For rational matrix frames with targets $S_u + I$, every jump (new frame minus old frame) of a run with at most W moves has nonzero determinant, and the total rank of its jumps is at least Wm.

Outline of the formal proof. The potential of Theorem 12.4 needs only a target frame per item and survives coarsening: a move that does not change the π-class of the wire does not change the coarsened run. For single jumps: if a jump J of a run with at most W moves were singular, choose $\ell \neq 0$ with $\ell^{\mathsf{T}} J = 0$; the coarsening $M \mapsto \ell^{\mathsf{T}} M$ separates every S_u from $S_u + I$ (they differ by $\ell^{\mathsf{T}} \neq 0$) but hides J, so fewer than W moves would be visible. So every jump has rank m, and there are at least W jumps by Theorem 12.4. □

Theorem 12.7 (Mod-2 bounds for integer frames)**.** Lean: `Transpose.ShearFrame.two_mul_le_rank_sum` [`Frames/GadgetMod2.lean`]

For integer 2×2 frames, the total rank of the frame jumps of every valid run that ends with wire $\rho(u)$ at frame $S_u + I$ holding e_u (as in Theorem 12.4) is at least $2W$, the baseline Wm for $m = 2$.

Theorem 12.8 (Mod-2 bounds, every m)**.** Lean: `Transpose.ShearFrame.one_sided_bound` [`Frames/GadgetOneSided.lean`]

Lean: `Transpose.ShearFrame.two_sided_bound` [`Frames/GadgetTwoSided.lean`]

For integer $m \times m$ frames, the total rank s (over $\mathbb{Q}$) of the jumps of every valid run that ends with wire $\rho(u)$ at frame $S_u + I$ holding e_u satisfies $2(2^m - 1)W \leq 2^m s$ and $2(2^m - 1)W \leq (2^{m-1} + 1)s$. In particular $s \geq 2.8W$ for $m = 3$ and $s \geq (4 - o(1))W$ as $m \to \infty$.

Outline of the formal proofs of Theorems 12.7 and 12.8. Average Theorem 12.6 over a family of coarsenings: $M \mapsto \ell^{\mathsf{T}}(M \bmod 2)$ for nonzero $\ell \in \mathbb{F}_2^m$ (one-sided), or $M \mapsto \ell^{\mathsf{T}}(M \bmod 2)v$ for pairs with $\ell^{\mathsf{T}} v = 1$ (two-sided). Each coarsening separates every source from its target, so it sees at least W jumps. A jump of rank r over $\mathbb{F}_2$ is seen by $2^m - 2^{m-r}$ of the one-sided coarsenings, and by a bounded number of the two-sided ones, computed from the left kernel and the fixed vectors of the jump; and the rank over $\mathbb{F}_2$ is at most the rank over $\mathbb{Q}$ (`Frames/RankMod.lean`). For $m = 2$ a finite check over $\mathbb{F}_2^{2\times 2}$ gives the constant. □

Proposition 12.9 (From gadgets to runs)**.** Not formalized; a complete proof is given below.

Every gadget has a valid run, in the sense of Definition 12.3, that ends with wire $\rho(u)$ at frame $S_u + I$ holding e_u for every u, has at most s moves, and whose jumps have total rank s. Consequently:

(a) every gadget has cost $s \geq W$;

(b) a gadget whose wires make at most W frame changes in total (pairs of distinct consecutive frames along their frame paths) has cost $s \geq Wm$; in particular, if $m \geq 2$ and $W \geq 1$, no gadget has $s \leq W$;

(c) a gadget with integer frames satisfies $s \geq 2W$ if $m = 2$, and $2(2^m - 1)W \leq 2^m s$ and $2(2^m - 1)W \leq (2^{m-1} + 1)s$ for every m.

Proof (not formalized). Start from the configuration in which wire u is at frame S_u holding e_u, and process the gates of the gadget in time order: before a gate between wires i and j at frame f, move each of the two wires to f unless it is already there, and then apply the gate event; after the last gate, move every wire to its sink frame unless it is already there. Each gate joins two distinct wires at a common frame, so the run is valid. Along each wire the run makes exactly one move for each pair of distinct consecutive frames of the wire's frame path (source, gate frames in time order, sink), and the jump of that move is their difference. The cost s is the total rank of these differences, so the jumps have total rank s; each has rank at least 1, so there are at most s moves. The gate events act on the vectors as the gates act on the data, so by (G1) wire $\rho(u)$ ends holding e_u, and by (G2) it ends at frame $T_{\rho(u)} = S_u + I$.

Part (a) follows from Theorem 12.4 with $\mathrm{shift}(S) = S + I$: W is at most the number of moves, which is at most s. For (b), a gadget whose wires make at most W frame changes has a run with at most W moves, so by Theorem 12.6 the total rank s of its jumps is at least Wm; and if $s \leq W$, the run has at most $s \leq W$ moves, so $s \geq Wm \geq 2W > W$ when $m \geq 2$ and $W \geq 1$, a contradiction. Part (c) is Theorems 12.7 and 12.8 applied to the run, whose frames are the integer frames of the gadget. □

Within the informal reading of the template above, (a) means that the framework cannot give linear time. The Lean structure `Transpose.ShearFrame.Gadget`, which omits the gates and (G1), satisfies only $s \geq m$ (Theorem 12.2).

12.4 Tight runs

A valid run as in Theorem 12.6, with $\mathrm{tgt}(u) \neq S_u$ for all u, that makes at most W frame-changing moves is *tight*; by Theorem 12.6 it makes exactly W such moves. Over a finite set of frames, tight runs are rigid.

Theorem 12.10 (Structure of tight runs)**.** Lean: `Transpose.ShearFrame.tight_potentials`, `Transpose.ShearFrame.tight_sigma_split`, `Transpose.ShearFrame.targetI_inf_eq_bot`, `Transpose.ShearFrame.cargo_rule`, `Transpose.ShearFrame.cargo_rule_bool` [`Frames/GadgetCargo.lean`]

Let the set F of frames be finite. In a tight run, let σ_f be the span of the vectors on the wires at frame f, N_f the span of the e_u with $\mathrm{tgt}(u) = f$, and N'_f the span of the e_u with $S_u = f$. After any prefix with t frame-changing moves the "arrived" potential $\sum_f \dim(\sigma_f \cap N_f)$ is t and the "not yet departed" potential $\sum_f \dim(\sigma_f \cap N'_f)$ is $W - t$, and at every frame $\sigma_f = (\sigma_f \cap N_f) \oplus (\sigma_f \cap N'_f)$. If moreover each item has only two frames (every frame other than S_u is $\mathrm{tgt}(u)$, as after a two-class coarsening), then every frame-changing move carries a vector lying in the span of the items whose target is its destination and in the span of the items born at its origin (the cargo rule).

Theorem 12.11 (Item itineraries)**.** Lean: `Transpose.ShearFrame.chain_exists_visible`, `Transpose.ShearFrame.itinerary_condition`, `Transpose.ShearFrame.chain_itinerary`, `Transpose.ShearFrame.itinerary_condition_matrix`, `Transpose.ShearFrame.chain_itinerary_matrix` [`Frames/GadgetItinerary.lean`]

In every valid run as in Theorem 12.6, every item has a chain of frame-changing hops (moves whose cargo contains the item) from its source to its target. Let π be a two-class coarsening with $\pi(\mathrm{tgt}(u)) \neq \pi(S_u)$ for all u, and suppose the run makes exactly W moves that change the π-class. Then every such move carrying an item starts on the item's birth side (π of its origin equals $\pi(S_u)$), and every item has a chain from its source to its target all of whose π-visible hops are admissible in this sense. For frames in $\mathbb{F}_2^{m\times m}$ with targets $S_u + I$ and a coarsening by a functional Λ with $\mathrm{tr}\,\Lambda = 1$ (the class of M is $\langle \Lambda, M\rangle$), admissibility of a hop of s from f_0 with jump J and $\langle \Lambda, J\rangle = 1$ reads $\langle \Lambda, S_s + f_0\rangle = 0$.

Theorem 12.12 (Every carrying move lies on an admissible chain)**.** Lean: `Transpose.ShearFrame.carrying_move_on_chain`, `Transpose.ShearFrame.carrying_move_on_admissible_chain`, `Transpose.ShearFrame.visible_move_on_admissible_chain` [`Frames/GadgetItineraryC.lean`]

In every valid run as in Theorem 12.6, every frame-changing move whose cargo contains an item lies on a chain of frame-changing hops of that item from its source to its target. If π is as in Theorem 12.11 (exactly W moves change the π-class), every frame-changing move lies on such a chain of some item all of whose π-visible hops are admissible.

Theorem 12.13 (Mixing lemma)**.** Lean: `Transpose.ShearFrame.mixed_move_forces_extra_hop`, `Transpose.ShearFrame.mixing_lemma`, `Transpose.ShearFrame.mixing_lemma_matrix` [`Frames/GadgetMixing.lean`]

Weight frames by a function $\delta : F \times F \to \mathbb{N}$ with $\delta(f, f) = 0$, $\delta(f, g) = 0 \Rightarrow f = g$ and the triangle inequality (for matrices, $\delta(A, B) = \mathrm{rank}(B - A)$), and let $\delta(S_u, \mathrm{tgt}(u)) \geq m$ for every item u. Suppose a valid run as in Theorem 12.6 (with targets $\mathrm{tgt}(u)$) contains a frame-changing move whose cargo contains two distinct items, and every other frame-changing move carries a single item. Then some item of that cargo has δ-weighted hop count at least $m + 1$: a mixed move forces an extra hop.

Outline of the formal proofs of Theorems 12.10 to 12.13. The vectors of a configuration reached by a valid run are linearly independent, since gates are transvections; so the spans at distinct frames are independent and their dimensions add up to W. The arrived and not-departed potentials each change by at most one per visible move and sum to at most W, so in a tight run both move in lockstep, which forces the split and the cargo rule. Chains are found by following an item backwards (and forwards) through the moves whose cargo contains it, using that an item is present at a frame only if some move carried it there. Admissibility (Theorem 12.11) is the cargo rule of the coarsened two-class run. For the mixing lemma, suppose every item of the mixed cargo x, moved into frame g, has hop count at most m. An item present at a frame after (before) a segment was (will be) present at a frame within δ-distance of it bounded by the segment's hop count; with the triangle inequality, no item of the support of x is present at g before the mixed move, and no later frame-changing move into g carries one. As the later moves out of g carry unit vectors, every wire at g stays constant on the support of x while some wire at g keeps coordinate s_0; at the end that wire holds e_{s_0}, which is not constant on a support of two items. □

Together with Proposition 12.9, these say that, within the XOR-frame template, every gadget costs at least W (so linear time is out of reach), gadgets with integer frames cost substantially more than W, and runs over a finite set of frames that make exactly W frame changes (tight runs, Theorem 12.10), as well as runs that make exactly W changes of class at a two-class coarsening (Theorems 12.11 and 12.12), are rigid; for rational matrix frames, a run with at most W moves has total jump rank at least the baseline Wm (Theorem 12.6). Whether some gadget has s/W substantially below m, which would give an improvement over $O(L \log d)$ much larger than the claimed κ, is open (Section 14).

13 The construction side

Our aim at the outset was a linear-time transposer. We did not find one. This section records what the results above require of any construction, describes the one architecture we know that passes every test, and summarizes the mechanisms that fail. It contains no new theorems: every claim is either one of the results above, a citation, or explicitly labelled as a heuristic.

13.1 Necessary conditions

A machine witnessing (T) must have all of the following properties.

(N1) *Coding.* It cannot only move data along an input-independent pattern: oblivious data movement needs time $\Omega(L \log d)$ (Stoß [30] for Turing machines, Aggarwal and Vitter [3] in external memory; both bounds concern a fixed permutation of distinct items). For data movement that depends on the entries, which repeat because $b = \lceil \log_2 d \rceil$, no such bound is known.

(N2) *Many passes, at every scale.* Its run has $\Omega(\log d)$ passes (Theorem P′, Theorem 5.12), even on a $2^{-O(db)}$ fraction of the inputs (Theorem 5.13); the grouped form of Theorem 5.12 gives the same at coarser column scales.

(N3) *Folding motion.* Along the popular trace, the macroscopic work-head trajectories satisfy ($\star$) (Corollary 7.11): some head makes a macroscopic U-turn, and the heads together waste at least one unit of range (Theorem 7.6).

(N4) *A large coding gain.* The block network of the run of a slightly modified machine (output buffered at the end; state and head positions carried on extra stationary tapes) carries a causal code for complete exchange, on a set of co-entropy $O(d)$, at the profile capacities (Theorems 9.2 and 9.6); by Theorem 8.8 this code beats routing by a factor $\Omega(\log d)$. For an oblivious machine the code works on all inputs (Theorem 9.9); for oblivious I/O machines the analogous statement, at the level of pages, is [1, Thm. 6.2].

(N5) *Not an XOR-frame recursion.* Within the abstract model of Section 12, every run realizing a gadget makes at least as many frame changes as the gadget has wires (Theorem 12.4), so every gadget has cost $s \geq W$ (Proposition 12.9, not formalized), while linear time would need $s < W$; the model captures the preprint's template only informally.

Condition (N4) is the heart of the matter. Coding gains over routing in undirected networks do exist: the multiple-unicast (network coding) conjecture is false [7, 32]. But the known growing gains are of order $(\log n)^{\varepsilon}$ on networks of size n, obtained by amplifying a constant gap by tensoring [6], and they concern sparse demands; we do not expect tensoring a constant gap to produce a gain of order $\log n$ for dense demands (a heuristic, not a theorem). Braverman and He also observe [7] that the unrefereed sub-$n \log n$ Fourier circuits of [24] would give a coding gain over routing; such gains would not come from tensoring, but they live on circuit graphs, not on networks shaped by tapes. A linear-time transposer needs a gain of order $\log d$ for the densest demand, on a network shaped by tapes, which by the heuristic above would have to be *native*, that is, not obtained by amplifying a constant gap.

13.2 An architecture that survives

We describe, as a design and not as a result, the one architecture that we could not rule out.

Geometric recursion. The transposer runs $\Theta(\log d)$ levels. Level ℓ works on data of volume $\theta^\ell L$ for a constant $\theta < 1$, with $O(1)$ passes of an encoder E_ℓ and a decoder D_ℓ: it computes its output as $D_\ell(X_\ell, f_{\ell+1}(E_\ell(X_\ell)))$, where $f_{\ell+1}$ is the next level's problem. The total time is $\sum_\ell O(\theta^\ell L) = O(L)$, and the pass count is $\Theta(\log d)$, as Theorem P requires.

Expanding choreography. The heads of each level are driven along prepared *rulers* that realize a constant number of monotone maps of the interval, so that the relation graph between the encoder's input and output rows is a bounded-degree expander. The monotone expanders of Bourgain and Yehudayoff [5] provide such maps (near-identity Möbius maps from a suitable free generating set, together with translations and the identity); at least three monotone maps are needed, since a union of two monotone matchings is planar [10]. The rulers can be written in linear time, since their marks are computed with $O(\log L)$-bit arithmetic (a design computation, not included here). Near-identity generators are needed: integer or dyadic Möbius pairs generate discrete groups whose relation graphs are not expanders (a heuristic supported by numerical spectral gaps). Repeating each choreography a constant number of times should make the macroscopic motion satisfy the quantitative inequality ($\star$); this is a cost constraint, not an obstruction (heuristic).

Two coding cores. The geometry is the easy part. Each level needs a constant-fraction coding saving at the encoder, so that $|E_\ell(X_\ell)| \le \theta |X_\ell|$, and, because the decoder writes the level's whole output from $O(1)$-pass views of the next level's output, also at the decoder. In the simplest linear form the encoder core is the following problem.

> *Locally encoded index coding with own-row side information.* Let Γ be a bounded-degree graph on $[d]$. A broadcast consists of linear functionals of a $d \times d$ matrix, each supported on the two rows of an edge of Γ. Receiver j sees the broadcast and knows row j, and must recover column j. Can the broadcast have size $(1-\varepsilon)d^2$ for a fixed $\varepsilon > 0$?

Without the degree bound, $\varepsilon = \frac{1}{2}$ is achievable: broadcast $A + A^{\mathsf{T}}$ above the diagonal. With the degree bound, routing-type codes save only $O(d^2/\log d)$ on expanders, and hyperfinite (amenable) graphs save $o(d^2)$ (heuristic arguments; not included here). A positive answer on expanders would be, to our knowledge, the first constant-fraction coded saving in a tape-relevant model. A negative answer, together with a negative answer to the analogous decoder problem, would rule out every geometric recursion of the form above with $O(1)$-pass levels (heuristic: the necessity of both savings is argued here only informally).

13.3 Mechanisms that fail

The following classes of constructions fail for reasons proved in this paper.

- *Few passes.* Any schedule with $o(\log d)$ passes, with any coding, including constant-pass choreographies along expanders (Theorems P and P′).
- *Regular or one-way motion.* Machines whose macroscopic head motions (along the popular trace) are all stationary or monotone (Theorems 7.6 and 7.9).
- *XOR-frame recursions* in the abstract model of Section 12, which captures the template of [25] informally: every run realizing a gadget makes at least as many frame changes as the gadget has wires (Theorem 12.4), so every gadget has cost at least W (Proposition 12.9, not formalized) and linear time is out of reach.

Some classes of codes are moreover no easier than plain codes: linear codes, codes correct off a small set, and codes that are linear outside $O(dw)$ bits of capacity exist only if plain codes exist (Theorems 10.3 and 10.5); a counterexample to the robust conjecture from these classes would refute the plain one. Many further attempts failed for reasons we could only argue informally: flat schemes that code across a constant number of whole streams (they save at most a constant factor), window-local recursions whose levels read only nearby data, static coding certificates on Ramanujan-type graphs that resist conversion into causal codes, and re-indexings that turn transposition into a cheap map only through a transposition-hard change of basis. We mention them as warnings, not as results.

14 Open problems

1. *Decide (T).* Either construct a machine transposing $d \times d$ matrices with $\lceil \log_2 d \rceil$-bit entries in time $O(d^2 \log d)$, or prove that none exists. A proof is accepted by the frozen checker in the form described in Section 2.

2. *The Tape Coding Conjecture* (Conjecture 9.1). A proof refutes (T) (Theorem 9.2). It suffices to rule out codes at co-entropy Cd for every C, for infinitely many d, on diagrams that write their output at the end (Theorems 9.6 and 9.7). By Section 11, a proof must use the shape of tape networks, and by Theorem 9.9 it would show that oblivious machines cannot transpose in linear time. A refutation would be a family of codes with a coding gain $\Omega(\log d)$ (Theorem 8.8) for complete exchange on tape networks; it would not by itself give a machine.

3. *Plain versus robust.* Does the plain conjecture (codes correct on all inputs) imply the robust one? Already the first rung is open: does "no code on all inputs" imply "no code on half of the inputs"? A positive answer for codes of trace classes, which are self-detecting, would suffice for the plain conjecture to refute (T) (argued informally in Section 10).

4. *Oblivious machines.* Is there an oblivious linear-time transposer? The plain conjecture says no (Theorem 9.9). We expect that Harvey and van der Hoeven's reduction [17] preserves obliviousness, so that a negative answer would also rule out oblivious linear-time multiplication; this requires checking that all control flow and data layout in their construction depend only on the sizes. Does (T) imply oblivious (T)?

5. *Locally encoded index coding* (Section 13). On bounded-degree expanders, can a broadcast of two-row functionals of size $(1-\varepsilon)d^2$ let every receiver, knowing its own row, recover its column?

6. *Sharper passes.* Remove the dependence on T from Theorem P′, whose bound gives $\Omega(\log d)$ passes only for $T \le \mathrm{poly}(L)$ (for example through the run-length lemma of Grohe and Schweikardt [15]), and extend Theorems P and P′ to passes that may move back by a bounded number of cells.

7. *Beyond levels.* Theorem M* concerns partitions of macroscopic time into intervals. Cuts of other shapes, or information inequalities that are not cut bounds, are needed to go further; is there a tape-specific non-cut inequality for oblivious machines?

8. *XOR-frame gadgets.* Is there a gadget (Definition 12.1) with $s/W \le m^\tau$ for some fixed $\tau < 1$? This would improve the exponent of the sub-$L \log d$ transposition claim substantially, though not to linear time (Proposition 12.9(a), not formalized). For integer frames the lower bounds of Proposition 12.9(c), which come from Theorems 12.7 and 12.8, stay below $4W$, while the trivial gadget costs mW; close this gap.

A Index of Lean declarations

Every declaration cited in this paper, grouped by the Lean module that contains it. All of them depend only on the axioms `propext`, `Classical.choice` and `Quot.sound` (Appendix B).

`Transpose.Frames.Gadget` `Transpose.ShearFrame.Gadget`, `Transpose.ShearFrame.Gadget.cost`, `Transpose.ShearFrame.Gadget.cost_ge_m`, `Transpose.ShearFrame.Gadget.cost_ge_of_invariant_family`, `Transpose.ShearFrame.Gadget.invariant_cost_ge`.

`Transpose.Frames.GadgetCargo` `Transpose.ShearFrame.cargo_rule`, `Transpose.ShearFrame.cargo_rule_bool`, `Transpose.ShearFrame.targetI_inf_eq_bot`, `Transpose.ShearFrame.tight_potentials`, `Transpose.ShearFrame.tight_sigma_split`.

`Transpose.Frames.GadgetCoarse` `Transpose.ShearFrame.det_ne_zero_of_moves_le`, `Transpose.ShearFrame.rank_sum_of_moves_le`, `Transpose.ShearFrame.visible_ge_W_of_coarse`, `Transpose.ShearFrame.vmoves_ge_W`.

`Transpose.Frames.GadgetItinerary` `Transpose.ShearFrame.chain_exists_visible`, `Transpose.ShearFrame.chain_itinerary`, `Transpose.ShearFrame.chain_itinerary_matrix`, `Transpose.ShearFrame.itinerary_condition`, `Transpose.ShearFrame.itinerary_condition_matrix`.

`Transpose.Frames.GadgetItineraryC` `Transpose.ShearFrame.carrying_move_on_admissible_chain`, `Transpose.ShearFrame.carrying_move_on_chain`, `Transpose.ShearFrame.visible_move_on_admissible_chain`.

`Transpose.Frames.GadgetMixing` `Transpose.ShearFrame.mixed_move_forces_extra_hop`, `Transpose.ShearFrame.mixing_lemma`, `Transpose.ShearFrame.mixing_lemma_matrix`.

`Transpose.Frames.GadgetMod2` `Transpose.ShearFrame.two_mul_le_rank_sum`.

`Transpose.Frames.GadgetOneSided` `Transpose.ShearFrame.one_sided_bound`.

`Transpose.Frames.GadgetProjection` `Transpose.ShearFrame.PGadget.two_mul_cost_ge`.

`Transpose.Frames.GadgetRuns` `Transpose.ShearFrame.Cfg`, `Transpose.ShearFrame.Cfg.allValid`, `Transpose.ShearFrame.Event`, `Transpose.ShearFrame.cost_ge_W`, `Transpose.ShearFrame.init`, `Transpose.ShearFrame.moves`, `Transpose.ShearFrame.moves_ge_W`, `Transpose.ShearFrame.moves_ge_W_matrix`.

`Transpose.Frames.GadgetTwoSided` `Transpose.ShearFrame.two_sided_bound`.

`Transpose.Frames.GroupAlgebraRuns` `Transpose.ShearFrame.GroupAlgebra.changes_ge_W`, `Transpose.ShearFrame.GroupAlgebra.cost_ge_W`.

`Transpose.Statement` `Transpose.ComputesWithin`, `Transpose.LinearTimeTransposable`, `Transpose.TransposesWithin`, `Transpose.encode`, `Transpose.entryBits`.

`Transpose.Tape.BufTM` `Impl.bufTM`.

Transpose.Tape.Defs AdmissibleRes, HaltsAt, IsChord, IsLastVisit, ObliviousOn, Transposes, blockOf, chordsInto, curBlock, cutPairs, cutSet, entryCells, entryPoints, entryTimes, matrixInputs, numSlots, pairRes, residentPair, run, slotOf, snkCols, srcRows, touchRes, tpos, transAt, writtenSym.

Transpose.Tape.DefsCE IsWalk, NetCode, NetCode.edgeVal, NetCode.inVals, NetCode.outVal, NoCode_TM, NoCode_TM_coent, NoCode_TM_hintBits, ObliviousLinearTimeTransposable, RobustCE_TM, RobustCE_TM_dist, RobustNoCode_TM, RobustNoCode_TM_bits, Routing, TSDiagram, TSDiagram.CapProfile, TSDiagram.RoutesAt, TSDiagram.Solves, TSDiagram.Valid, TSDiagram.blockCap, TSDiagram.blockSize, TSDiagram.chordEdges, TSDiagram.edges, TSDiagram.memEdges, TSDiagram.nodes, TSDiagram.outNode, TSDiagram.profileCap, TSDiagram.rowDist, TSDiagram.rowsAt, TSDiagram.visible, ball, netDist, runDiagram.

Transpose.Tape.DefsM ConsecIn, CutIneq, FinUnionIcc, PiecewiseConstOn, blockTraj, cellBits, crossPairs, horizon, levelCross, levelExcess, levelSet, rescaledTraj, slotLen.

Transpose.Tape.DefsP AllMono, MonoOn, PassPartition, addrCount, addresses, addrsBelow, childNode, combSet, forest, forestNode, greedyBound, greedyPasses, readSet, rootNode, sSet.

Transpose.Tape.DefsPP nodeCode, sideCode, structCount.

Transpose.Tape.DefsTrace SameTrace, TraceClass, blockTrace, rescaledTrajOf, traceConst, traceCount.

Transpose.Tape.Statements.Adaptive adaptive_hyps_of_linearTime, block_locality, block_locality_offsets_counterexample, cut_bound_trace, cut_bound_trace_bits, cut_bound_trace_bits_on, cut_bound_trace_on, exists_popular_traceClass, forest_agree, forest_comb_agree, forest_comb_agree_on, isoperimetry_trace, isoperimetry_trace_on, kraft_group, kraft_group_on, linearTimeTransposable_popular_limits, locality_anchored, locality_entry, mstar_discrete_class_on, no_stationary_limits_class_on, rescaledTrajOf_close, tendstoUniformlyOn_of_mem, theoremMStar_class_nonuniform, theoremMStar_excess_nonuniform, theoremMStar_excess_robust, theoremMStar_level_adaptive, theoremMStar_level_class, theoremMStar_level_class_on, theoremMStar_level_nonuniform, theoremMStar_level_robust, theoremPprime, theoremPprime_bits, theoremPprime_div, theoremPprime_gamma, theoremPprime_greedy, theoremPprime_grouped, theoremPprime_grouped_bits, theoremPprime_grouped_on, theoremPprime_grouped_on_bits, theoremPprime_hyps_of_oblivious, theoremPprime_hyps_tr2dd, theoremPprime_linearTime, theoremPprime_no_five_pass, theoremPprime_on, theoremPprime_on_bits, theoremPprime_on_div, theoremPprime_on_greedy, theoremPprime_on_pow, theoremPprime_on_robust, theoremPprime_on_robust_gamma, theoremPprime_pow, theoremPprime_ratio, traceBits_le, traceClass_of_mem, traceClass_of_oblivious, traceClass_of_obliviousOn.

Transpose.Tape.Statements.Coding blockTraj_limit_lipschitz, bufTM_run, card_chordEdges_le, card_le_of_solves, ce_TM_dist_iff_noCode, ce_TM_dist_of_robustCE_TM_dist, ce_TM_iff_noCode, ce_TM_of_robustCE_TM, code_gap, code_of_hint, codes_coentLinear_of_linearTimeTransposable, codes_of_fastOn, codes_of_fastOn_bits, codes_of_linearTimeTransposable, codes_of_obliviousLinearTimeTransposable, coentLinear_io_iff_exists_coent_io, coentLinear_io_of_robustNoCode_io, coentLinear_of_robustCE_TM_lin, coentLinear_of_robustNoCode, coent_io_nw_of_robustNoCode_io, coent_io_of_coent, coent_mono, coent_of_hintBits, cutIneq_of_solves, cutset_bits_bitwise, cutset_bound, cutset_bound_bitwise, exists_popular_traceClass_on, exists_shift_cover,

exists_shift_iff_unbounded, exists_subseq_blockTraj, flow_rate_lt, frozen_V2, hintBits_of_coent, hintCodes_linear_of_linearTimeTransposable, hintCodes_of_linearTimeTransposable, hintLinear_iff_coentLinear, hint_of_code, isoperimetry_code, linearTime_fails_of_coentLinear, linearTime_fails_of_robustNoCode, linearTime_fails_of_robustNoCode_logBits, mstar_of_not_robustNoCode, mstar_of_robustNoCode_counterexamples, noCode_bits_iff, noCode_iff_coent_zero, noCode_iff_forall_cap, noCode_io_of_robustNoCode_io, noCode_of_coentLinear, noCode_of_noCode_hint, noCode_of_robustNoCode, no_fast_transposer_of_coent, no_fast_transposer_of_coentLinear, no_fast_transposer_of_robustNoCode, no_fast_transposer_of_robustNoCode_bits, not_linearTimeTransposable_of_coent, not_linearTimeTransposable_of_coentLinear, not_linearTimeTransposable_of_coentLinear_end, not_linearTimeTransposable_of_coentLinear_end_io, not_linearTimeTransposable_of_coentLinear_io, not_linearTimeTransposable_of_coent_io, not_linearTimeTransposable_of_coent_nw, not_linearTimeTransposable_of_noCode_hint, not_linearTimeTransposable_of_noCode_hintLinear, not_linearTimeTransposable_of_robustCE_TM, not_linearTimeTransposable_of_robustCE_TM_dist, not_linearTimeTransposable_of_robustCE_TM_io, not_linearTimeTransposable_of_robustCE_TM_lin, not_linearTimeTransposable_of_robustNoCode, not_linearTimeTransposable_of_robustNoCode_io, not_linearTimeTransposable_of_robustNoCode_logBits, not_obliviousLinearTimeTransposable_of_ce_TM, not_obliviousLinearTimeTransposable_of_ce_TM_dist, not_obliviousLinearTimeTransposable_of_noCode, not_obliviousLinearTimeTransposable_of_noCode_io, not_robustCE_TM_of_linearTimeTransposable, numSlots_le_of, pairRes_congr, plain_tfae, robustCE_TM_att_iff, robustCE_TM_att_iff_robustCE_TM, robustCE_TM_dist_iff_robustNoCode, robustCE_TM_gamma_iff, robustCE_TM_gamma_per_iff, robustCE_TM_iff_robustNoCode, robustCE_TM_lin_iff, robustNoCode_bits_iff, robustNoCode_iff_attached, robustNoCode_iff_coent, robustNoCode_iff_forall_cap, robustNoCode_iff_hint, robustNoCode_io_of_robustNoCode, robustNoCode_logBits_iff, robustNoCode_of_gainBound, robustNoCode_with_iff, robustNoCode_with_k_iff, robustNoCode_with_of_robustNoCode, robust_io_tfae, robust_tfae, rowDist_sum_ge, solves_mono, solves_profileCap_addTapes, solves_shiftCode, sum_cap_le, sum_cross_blockCap_le, theoremMStar_diagram, theoremMStar_diagram_profile, theoremMStar_level_diagram, theoremMStar_level_walk, theoremMStar_walk, traceClass_code, traceClass_coent_lt, volume_gap, weak_duality.

Transpose.Tape.Statements.Core addrCount_mul, ae_finite_levelSet, card_addresses_le, card_le_entryData, charging_bound, computesWithin_iff, copyTM_obliviousOn, copyTM_transposes, countable_localExtr_values, coverage_grouped, coverage_per_address, crossChords_split, cutPairs_card_mul_le, cut_bound, cut_bound_bits, cut_bound_on, cut_bound_on_coentropy, cut_bound_pairs, cut_bound_sum, cut_bound_sum_bits, cut_bound_sum_on, entryCells_slotSteps_card, entryPoints_slotSteps_card, entryTimes_card_le, entryTimes_charged, excess_add_range_le, exists_slot_block_eq, exists_subseq_tendstoUniformlyOn, forestNode_disjoint, forest_entryCells_card_le, forest_entryCells_subset, forest_entryPoints_card_le, forest_entryPoints_subset, forest_info, forest_subset, greedyPasses_indep, greedyPasses_le, greedy_passPartition, haltsAt_unique, isoperimetry, isoperimetry_res, isoperimetry_res_on, isoperimetry_touch, levelChords_limsup_le, levelCross_le_levelExcess, levelSet_encard_majorant, lintegral_encard_levelSet_le, lintegral_levelExcess_eq_zero_of_monotone, locality, locality_core, locality_function, locality_interval, locality_oblivious, matrixInputs_eq, no_stationary_limits, not_all_monotone_of_star, numSlots_le_horizon, one_le_excess_of_star, outRev_reverse_eq, pairChange_spacing, pairRes_V2, pairRes_admissible, pairRes_begin_unique, pairRes_chord_source_unique, pairRes_chord_target_unique, pairRes_end_unique, pairRes_subset, passPartition_iff, passPartition_indep, pass_fresh, range_add_one_le_of_star,

read_chain, rescaledTraj_limit_lipschitz, slot_block_step, slot_cut_bound, slot_cut_bound_partition, srcRows_slotSteps_card_le, stepN_init_eq_some_iff, sum_cutPairs_ge, theoremMStar_conclusion_can_fail, theoremMStar_excess, theoremMStar_level, theoremMStar_level_sub, theoremMStar_level_subseq, theoremP, theoremP_bits, theoremP_bits_add_one, theoremP_div, theoremP_greedy, theoremP_grouped, theoremP_grouped_bits, theoremP_grouped_on, theoremP_grouped_on_bits, theoremP_hyps_tr2, theoremP_hyps_tr2k, theoremP_on, theoremP_on_bits, theoremP_on_div, theoremP_on_pow, theoremP_on_robust, theoremP_on_robust_gamma, theoremP_pow, touchRes_admissible, touchRes_begin_unique, touchRes_chord_source_unique, touchRes_chord_target_unique, touchRes_end_unique, tpos_spec, tr2_cutPairs, tr2_obliviousOn, tr2_transposes, tr2k_obliviousOn, tr2k_transposes, transposes_of_transposesWithin, uncovered_cols_le, uncovered_cols_le_len, uncovered_cols_le_slots, uncovered_cols_le_slots_len, unitSteps_inTraj, unitSteps_workTraj, work_eq_blank, work_eq_written.

Transpose.Tape.Statements.PlainRobust PlainRobust.noCode_additiveCover_of_noCode, PlainRobust.noCode_additive_of_noCode, PlainRobust.noCode_affine_exc_of_noCode, PlainRobust.noCode_affine_of_noCode, PlainRobust.noCode_colAggregate_of_noCode, PlainRobust.noCode_colCover_of_noCode, PlainRobust.noCode_cosmall_of_noCode, PlainRobust.noCode_crossQuad_of_noCode, PlainRobust.noCode_decomp_of_noCode, PlainRobust.noCode_fewErrorsCover_of_noCode, PlainRobust.noCode_fewErrors_of_noCode, PlainRobust.noCode_hintConst_of_noCode, PlainRobust.noCode_iff_hintConst, PlainRobust.noCode_linearExcept_of_noCode, PlainRobust.noCode_linearPlusImage_of_noCode, PlainRobust.noCode_linearPlus_of_noCode, PlainRobust.noCode_linear_of_noCode, PlainRobust.noCode_lowDegree_of_noCode, PlainRobust.noCode_netCover_of_noCode, PlainRobust.noCode_netHint_of_noCode, PlainRobust.noCode_of_affine_subset, PlainRobust.noCode_quadForm_of_noCode, PlainRobust.noCode_rowMapCover_of_noCode, PlainRobust.noCode_sparseEntryErrors_of_noCode, PlainRobust.noCode_sparseErrors_of_noCode.

Transpose.Tape.Witness Witness.tr2, Witness.tr2k.

Transpose.Tape.WitnessDD Witness.tr2dd.

B Building and verifying the formalization

The Lean project is the directory `lean/` next to this paper. It uses Lean and Mathlib, both at version v4.34.1 (`lean-toolchain`, `lakefile.toml`, `lake-manifest.json`).

Build. With `elan` installed, run in `lean/`:

```
lake exe cache get && lake build
```

The first command downloads the compiled Mathlib; the second compiles the library (86 modules).

The frozen files. `Transpose/Statement.lean` (the statement (T)) and `Verify.lean` (the checker of a future `Transpose/Answer.lean`) are byte-identical to the files fixed before this work began; `shasum -a 256 -c FROZEN.sha256` checks them. `Verify.lean` imports `Transpose/Answer.lean`, which does not exist because (T) is not decided; running it therefore fails with a missing-module error.

Axioms. The file `PaperAxioms.lean`, generated from the LaTeX sources of this paper, runs `#print axioms` on every declaration cited here (Appendix A):

```
lake env lean PaperAxioms.lean
```

Every declaration depends at most on `propext`, `Classical.choice` and `Quot.sound`. No Lean file uses `sorry`, `admit` or `native_decide`, or declares an `axiom` (the words occur only in comments of the frozen files).

Where things are. The public statements are in four files. They are proved from theorems in `Impl` namespaces, usually of the same name, which the other files of the library prove:

- `Tape/Statements/Core.lean`: Sections 2 to 4, Theorem P (Section 5) and Theorem M* for oblivious machines (Section 7);
- `Tape/Statements/Adaptive.lean`: Theorem P′, trace classes, and Theorem M* for adaptive, robust and non-uniform machines (Sections 5 to 7);
- `Tape/Statements/Coding.lean`: codes, Theorem M* for walks and for code-carrying diagrams, the Tape Coding Conjecture and its consequences (Sections 8 to 10);
- `Tape/Statements/PlainRobust.lean`: the partial results of Section 10.

The results on XOR-frame recursions (Section 12) are in `Frames/`. Definitions are in `Tape/Defs.lean`, `Tape/DefsP.lean`, `Tape/DefsPP.lean`, `Tape/DefsTrace.lean`, `Tape/DefsM.lean` and `Tape/DefsCE.lean`.

References

[1] Micah Adler, Nicholas J. A. Harvey, Kamal Jain, Robert D. Kleinberg, and April Rasala Lehman. On the capacity of information networks. In *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2006)*, pages 241–250, 2006.

[2] Peyman Afshani, Casper Benjamin Freksen, Lior Kamma, and Kasper Green Larsen. Lower bounds for multiplication via network coding. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, volume 132 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 10:1–10:12, 2019. doi:10.4230/LIPIcs.ICALP.2019.10.

[3] Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988. doi:10.1145/48529.48535.

[4] Stefan Banach. Sur les lignes rectifiables et les surfaces dont l'aire est finie. *Fundamenta Mathematicae*, 7:225–236, 1925.

[5] Jean Bourgain and Amir Yehudayoff. Expansion in $\mathrm{SL}_2(\mathbb{R})$ and monotone expanders. *Geometric and Functional Analysis*, 23(1):1–41, 2013. doi:10.1007/s00039-012-0200-9.

[6] Mark Braverman, Sumegha Garg, and Ariel Schvartzman. Coding in undirected graphs is either very helpful or not helpful at all. In *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*, volume 67 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:18, 2017. doi:10.4230/LIPIcs.ITCS.2017.18.

[7] Mark Braverman and Zhongtian He. The multiple unicast conjecture is false. arXiv:2610.10108, 2026. URL: https://arxiv.org/abs/2610.10108.

[8] Martin Dietzfelbinger and Wolfgang Maass. The complexity of matrix transposition on one-tape off-line Turing machines with output tape. *Theoretical Computer Science*, 108(2):271–290, 1993.

[9] Martin Dietzfelbinger, Wolfgang Maass, and Georg Schnitger. The complexity of matrix transposition on one-tape off-line Turing machines. *Theoretical Computer Science*, 82(1):113–129, 1991.

[10] Vida Dujmović, Anastasios Sidiropoulos, and David R. Wood. Layouts of expander graphs. *Chicago Journal of Theoretical Computer Science*, 2016(1), 2016.

[11] Alireza Farhadi, MohammadTaghi Hajiaghayi, Kasper Green Larsen, and Elaine Shi. Lower bounds for external memory integer sorting via network coding. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*. ACM, 2019. Also arXiv:1811.01313.

[12] Martin Fürer. Faster integer multiplication. *SIAM Journal on Computing*, 39(3):979–1005, 2009.

[13] Étienne Grandjean. A natural NP-complete problem with a nontrivial lower bound. *SIAM Journal on Computing*, 17(4):786–809, 1988. doi:10.1137/0217050.

[14] Martin Grohe, André Hernich, and Nicole Schweikardt. Lower bounds for processing data with few random accesses to external memory. *Journal of the ACM*, 56(3), 2009. Article 12.

[15] Martin Grohe and Nicole Schweikardt. Lower bounds for sorting with few random accesses to external memory. In *Proceedings of the Twenty-fourth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS 2005)*. ACM, 2005. doi:10.1145/1065167.1065197.

[16] David Harvey and Joris van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021.

[17] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition. arXiv:2503.22848, 2025. URL: https://arxiv.org/abs/2503.22848.

[18] F. C. Hennie. One-tape, off-line Turing machine computations. *Information and Control*, 8(6):553–578, 1965.

[19] Walter W. Kirchherr. Transposition of an $l \times l$ matrix requires $\Omega(\log l)$ reversals on conservative Turing machines. *Information Processing Letters*, 28(2):55–59, 1988.

[20] Michael Langberg and Michelle Effros. Edge removal in undirected networks. In *2021 IEEE International Symposium on Information Theory (ISIT)*, pages 1421–1426, 2021. doi:10.1109/ISIT45174.2021.9518243.

[21] Jiatu Li and Tianqi Yang. $3.1n - o(n)$ circuit lower bounds for explicit functions. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022)*, pages 1180–1193, 2022.

[22] Zongpeng Li and Baochun Li. Network coding: The case of multiple unicast sessions. In *Proceedings of the 42nd Annual Allerton Conference on Communication, Control, and Computing*, 2004.

[23] Wolfgang Maass. Combinatorial lower bound arguments for deterministic and nondeterministic Turing machines. *Transactions of the American Mathematical Society*, 292(2):675–693, 1985.

[24] OpenAI. Finite tensor savings and exact Fourier circuits. GitHub release, repository `openai/math`, family 130, September 2026. Preprint dated 25 September 2026. URL: https://github.com/openai/math/blob/main/preprints/Finite-tensor-savings-and-exact-Fourier-circuits-September-25-2026/main.pdf.

[25] OpenAI. Integer multiplication below $n \log n$. GitHub release, repository `openai/math`, family 109, September 2026. Preprint dated 23 September 2026. URL: https://github.com/openai/math/blob/main/preprints/Integer-multiplication-below-n-log-n-September-23-2026/paper.pdf.

[26] Michael S. Paterson, Michael J. Fischer, and Albert R. Meyer. An improved overlap argument for on-line multiplication. Technical Memorandum MIT-LCS-TM-040, Massachusetts Institute of Technology, Project MAC, January 1974.

[27] Wolfgang J. Paul, Nicholas Pippenger, Endre Szemerédi, and William T. Trotter. On determinism versus non-determinism and related problems. In *24th Annual Symposium on Foundations of Computer Science (FOCS 1983)*, pages 429–438, 1983.

[28] Nicholas Pippenger and Michael J. Fischer. Relations among complexity measures. *Journal of the ACM*, 26(2):361–381, 1979.

[29] Arnold Schönhage and Volker Strassen. Schnelle Multiplikation großer Zahlen. *Computing*, 7:281–292, 1971.

[30] Hans-Jörg Stoß. Rangierkomplexität von Permutationen. *Acta Informatica*, 2:80–96, 1973.

[31] Paul M. B. Vitányi. On the simulation of many storage heads by one. *Theoretical Computer Science*, 34(1–2):157–168, 1984.

[32] Xindan Zhang, Baochun Li, and Zongpeng Li. Network coding can beat routing in undirected multiple-unicast networks. arXiv:2610.09367, 2026. URL: https://arxiv.org/abs/2610.09367.